微机原理及应用 期末复习

第一篇 80C51 部分

第一章 绪论

单片机：中央处理器（CPU）、随机存储器（RAM）、只读存储器（ROM）、输入输出口（I/O口）、中断系统、定时器/计数器、串行口等外设

8 0 C 5 1

- 8：单位机位数
- 0：掩膜ROM　7：EPROM　9：E^2PROM
- C：CMOS　S：ISP
- 5：无内部ROM　5：有ROM
- 1：片内存储器容量　1*4K

OTP：一次性可编程

FLASH

掩膜ROM：	生产时写入
PROM：	供用户写入一次
EPROM：	紫外线可擦除
E^2PROM：	电可擦除

51和52系列区别 ☆

MOVC　MOVX

资源配置 / 子系列	片内ROM形式：无	ROM	EPROM	E^2PROM	片内ROM容量	片内RAM容量	定时器/计数器	中断源
51子系列	8031	8051	8751	8951	4KB	128B	2×16	5
52子系列	8032	8052	8752	8952	8KB	256B	3×16	6

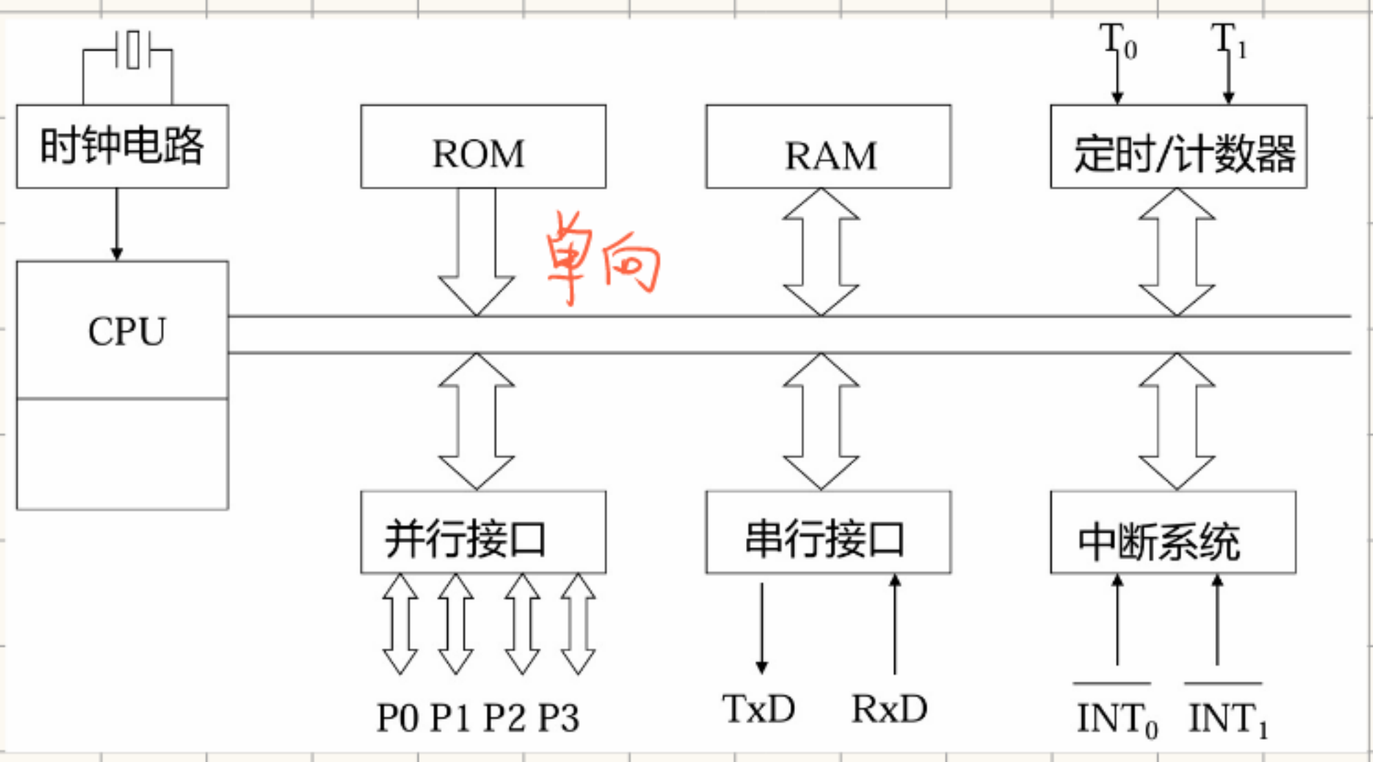
多T2定时器

第二章 80C51单片微机基本结构

2.1 结构组成：中央处理器（CPU）（运算器+控制器）+存储器（RAM、ROM）+输入/输出接口（并行、串行）

51单片机内部结构

① 中央控制器
② 程序存储器 ROM
③ 数据存储器 RAM
④ 并行 I/O 口
⑤ 串行 I/O 口　通信，构成多机系统
⑥ 定时器，计数器
⑦ 中断系统　内外5个中断，2个中断优先级
⑧ 定时器元件及电路



地址、数据、控制信号

2.2 引脚分类：

1. 电源和晶振：4根 V_{CC} V_{SS} XTAL1 XTAL2

2. I/O口 4个口，32根 I/O线

P0.P2可作扩展数据/地址线

P0：漏极开路双向I/O口 需外接上拉电阻

P1：8位 准双向I/O口

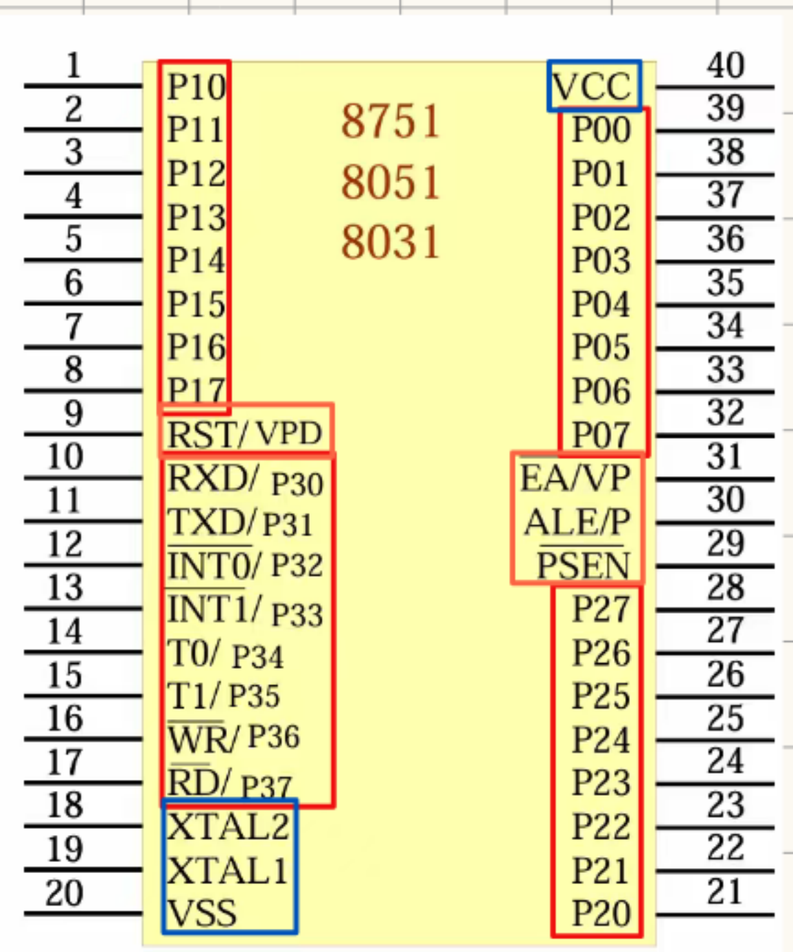
P2：8位、准双向I/O口，可用作总线，输出高8位地址

P3：8位、准双向I/O口，具有第二功能

IO口驱动：灌电流能力强于拉电流能力[上拉电阻大，输出电流能力弱]

Cortex 拉电流，灌电流能力相同

口线	第二功能	信号名称	口线	第二功能	信号名称
$P_{3.0}$	RXD 收	串行数据接收	$P_{3.4}$	T_0	定时器/计数器0计数输入
$P_{3.1}$	TXD 发	串行数据发送	$P_{3.5}$	T_1	定时器/计数器1计数输入
$P_{3.2}$	$\overline{INT_0}$	外部中断0申请	$P_{3.6}$	$\overline{WR}$ 写	外部RAM写选通
$P_{3.3}$	$\overline{INT_1}$	外部中断1申请	$P_{3.7}$	$\overline{RD}$ 读	外部RAM读选通



3. 控制线

RST 复位输入信号，高电平有效，2个机器周期以上高电平复位

$\overline{EA}$ / V_{PP}：片外程序存储器访问允许信号，低电平有效

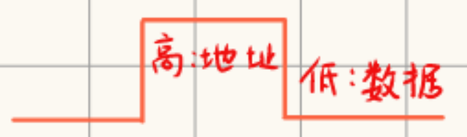
$\overline{EA}$=1 选择片内程序存储器

$\overline{EA}$=0 程序存储器都在片外

第二功能 EPROM编程使用，施加21V编程电压

ALE / $\overline{PROG}$ 地址锁存允许信号输出，访问外部存储器时，实现低8位数据与地址分开

第二功能 EPROM编程使用，施加编程用脉冲



$\overline{PSEN}$ 片外程序存储器读选通信号，低电平有效

2.3 CPU结构功能

2.3.1 中央控制器

PC：程序指针 不可直接访问

(不可访问) 16位寄存器 ⟷ 64KB ROM

PC存放的是下一条要从ROM中取出的指令地址

DPTR：数据指针 = DPH + DPL

(可被访问) 16位寄存器 可直接访问

数据存储器区

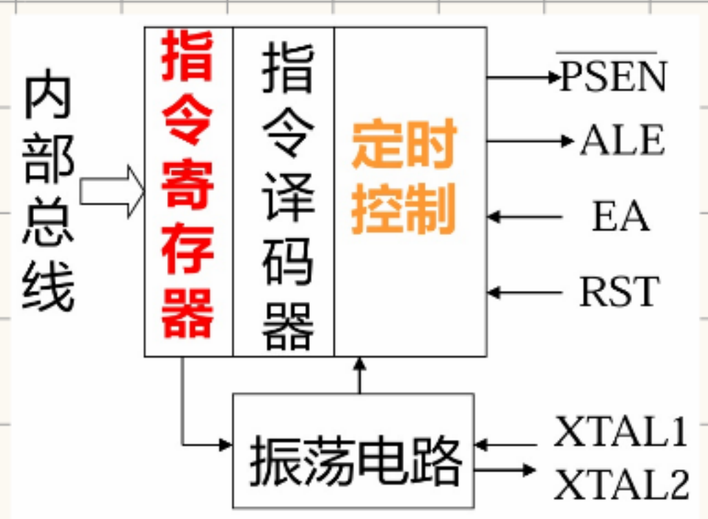
SP 8位 堆栈顶部 可操作 内部RAM

PC 16位 下一条指令地址 不可操作 ROM/FLASH

程序存储器区

片外I/O寻址用的地址寄存器

指令寄存器IR



2.3.2 运算器

1、ALU 算术逻辑运算单元 （本质全加器）

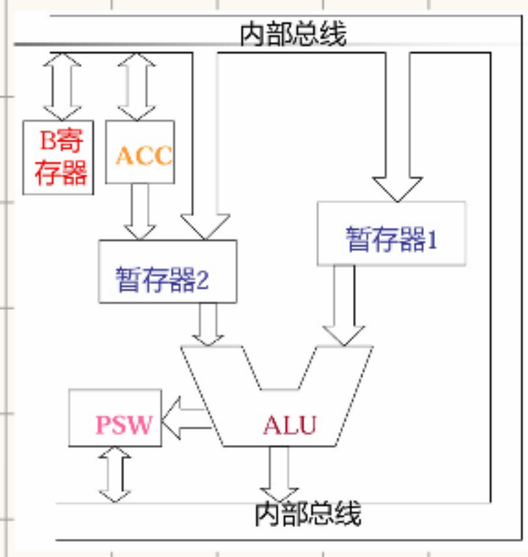
(1) 结果返回数据总线

(2) 输出PSW程序状态字

2、累加器 A （ACC） 8位

3、寄存器 B 8位

4、程序状态字 PSW



MUL AB；A × B = (B)(A)　　DIV AB；A / B 商(A)余(B)

PSW.7 CY位 进位标志 高位

PSW.6 AC位 辅助进位标志 低四位

PSW.5 用户标记位 F0

PSW.4 RS1

PSW.3 RS0

RS_1	RS_0	寄存器组	$R_0 \sim R_7$ 地址
0	0	0	00～07H
0	1	1	08～0FH
1	0	2	10～17H
1	1	3	18～1FH

PSW.2 OV 溢出标志 带符号数运算 （两正得负；两负得正） 乘法超过8位；除数为0

PSW.1 未用

PSW.0 奇偶校验位 分奇校验、偶校验

A寄存器中1的个数 { 奇，P为1 / 偶 P为0

2.3.3 时钟电路、CPU时序

晶振12分频得到机器周期

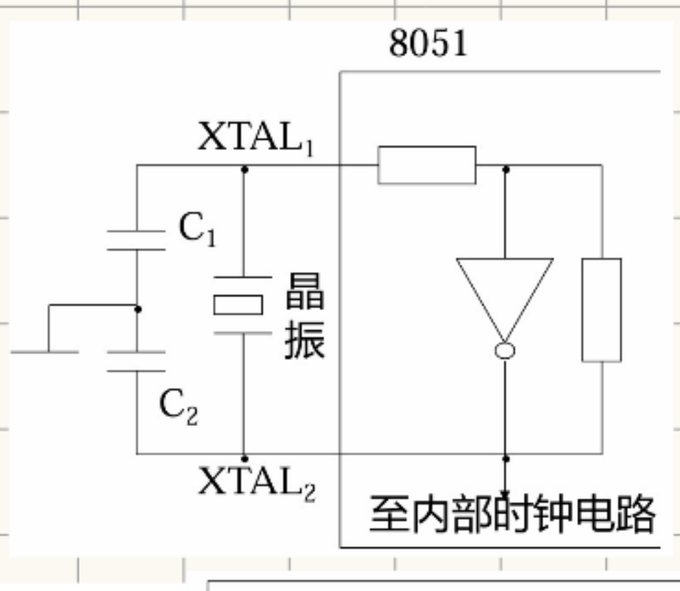
$\overline{PD}$ =1 振荡器停止工作，系统进入低功耗状态

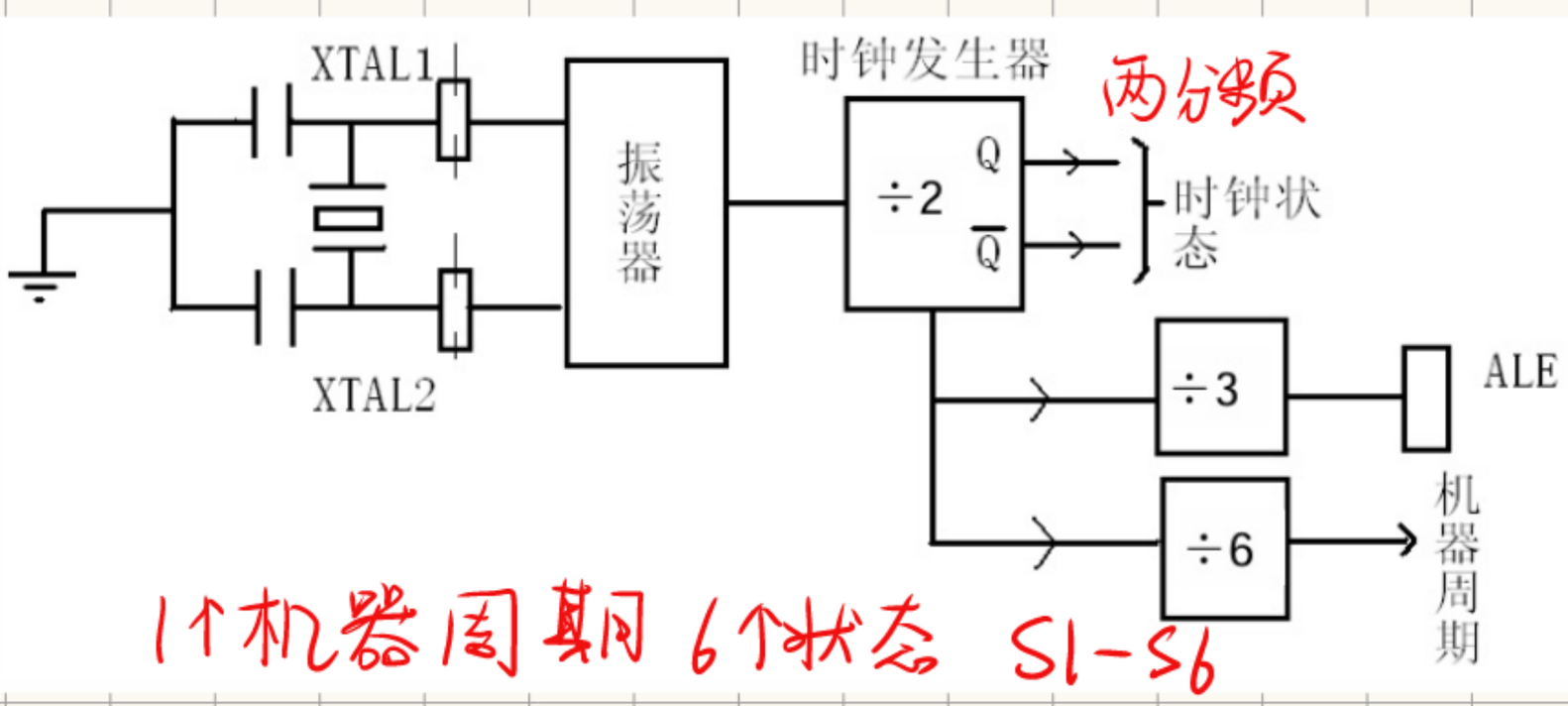
1个机器周期 = 12个振荡脉冲周期 = 1 us

↘ $\frac{1}{12}$ us 对应 12MHz

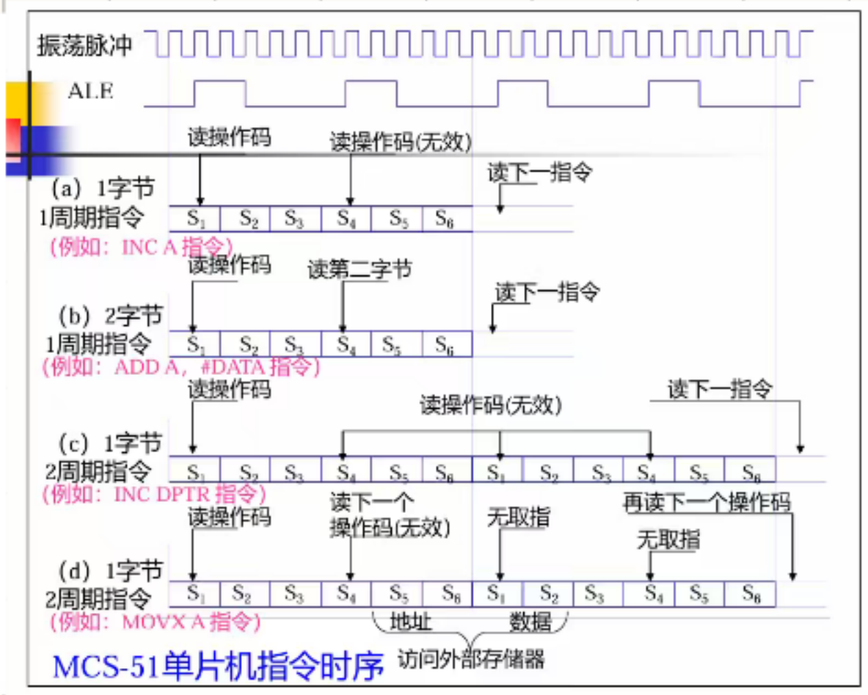
系统 ALE 信号周期 $\frac{1}{2}$ us

nop 1个机器周期，乘除 4个机器周期





1个机器周期 6个状态 S1-S6



单字节单周期 INC A

双字节单周期 ADD A，#data

单字节双周期 INC DPTR　RET、RETI、JMP @A+DPTR

单字节双周期 MOVX类 MOVX A，@DPTR

一个周期最多取2字节，因此三字节一定两周期

(1) 单字节单周期指令（例如INC A）
只需进行一次读指令操作。当第二个ALE有效时，由于PC没有加1，所以读出的还是原指令。

(2) 双字节单周期指令（例如ADD A，#data）
ALE的两次读操作都是有效的，第一次是读指令操作码，第二次是读指令第二字节。

(3) 单字节双周期指令（例如 INC DPTR）
两个机器周期共进行四次读指令的操作，但其中后三次的读操作无效。

(4) 单字节双周期指令（例如MOVX类指令）
第一机器周期时，与其它指令一样，第一次读指令（操作码）有效，第二次读指令操作无效。
第二周期时，进行外部RAM访问，此时与ALE信号无关，因此不产生读指令操作。

2.4 存储器结构和地址空间

普林斯顿　统一编码，地址空间重合（RAM、ROM）　也称冯诺依曼
　　数据　程序

哈佛结构　分别编码，地址空间独立　（51、Cortex 都是）

物理上：4个存储空间

程序存储器：片内、片外

数据存储器：片内、片外

3个存储器空间地址

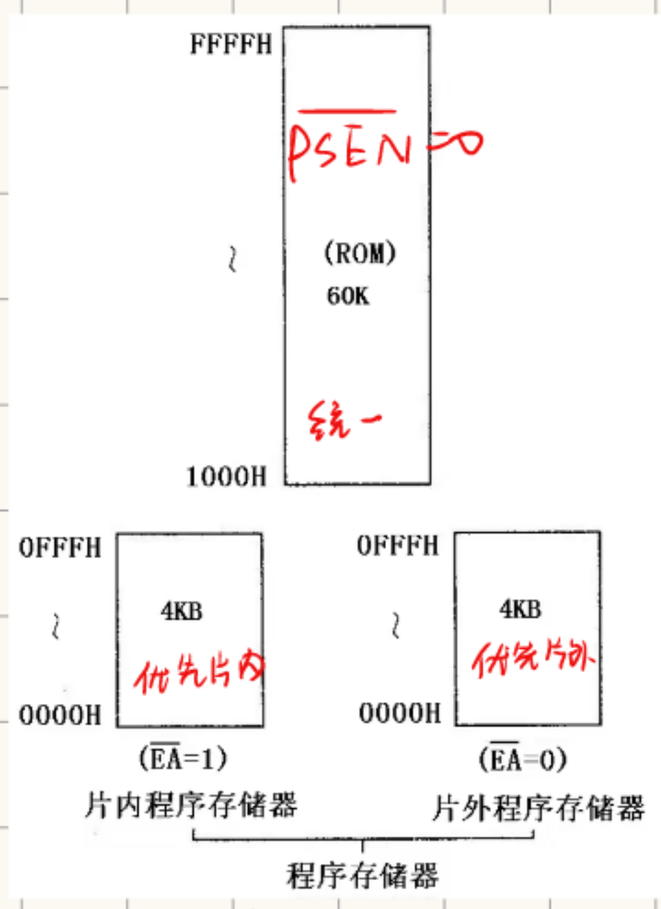
① 片内、片外统一 64KB ROM

② 片内 256B 数据存储空间 RAM

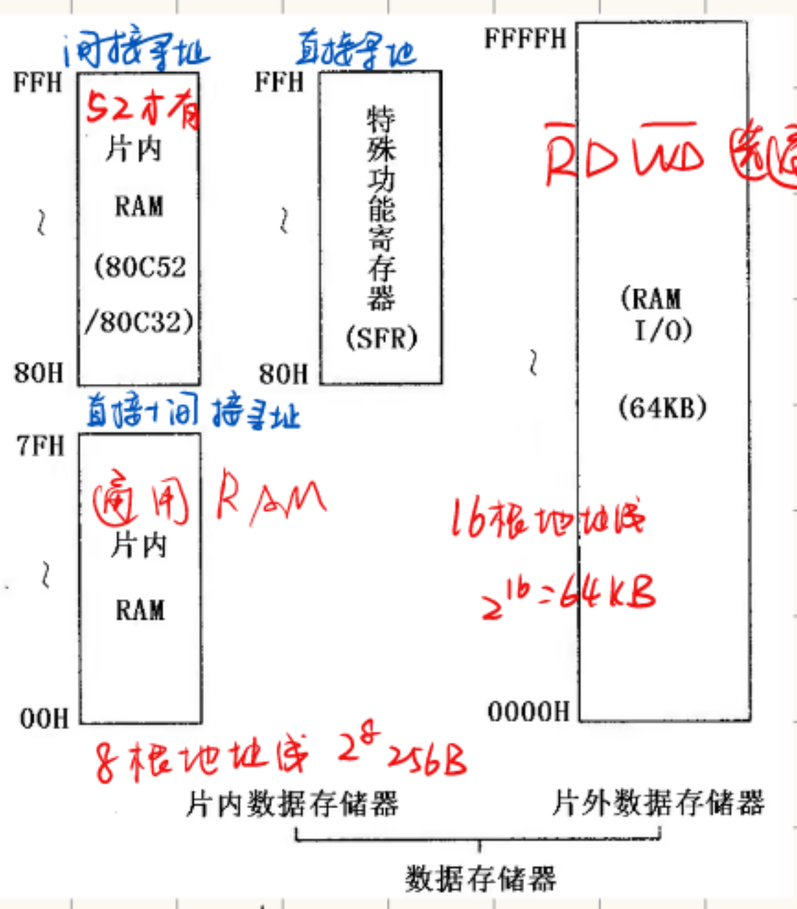
（0~127 通用 RAM，128-255 SFR）

52是 384B 数据存储空间

③ 片外 64kB RAM



统一指令　统一



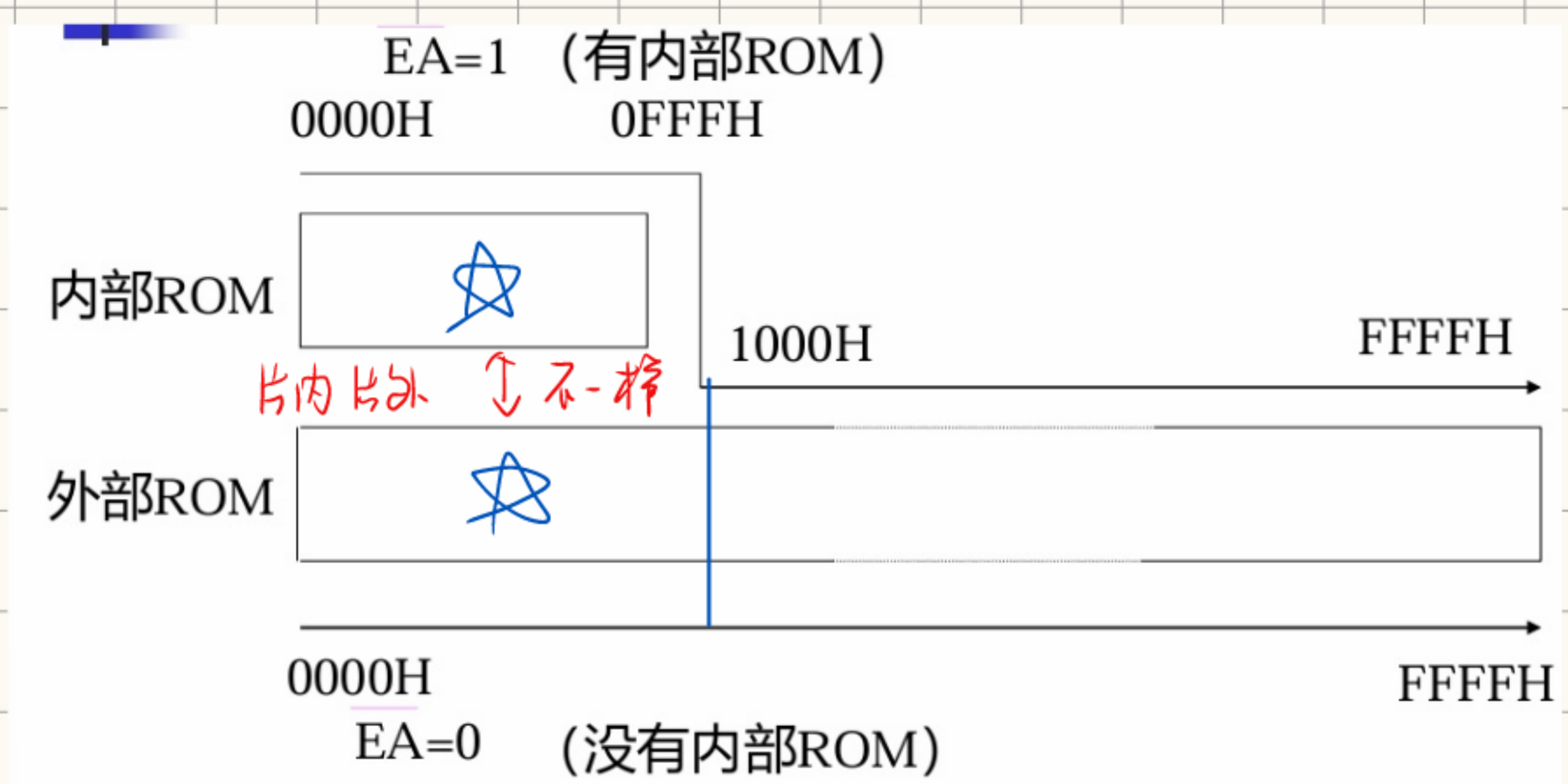
不同指令　逻辑不统一

	地址总线宽度	数据总线宽度	片外数据存储器	片内程序存储器
80C51	16位	8位	64KB ✗	4KB ✗
Cortex-M3	32位	32位		

6个特定地址 + 1个（52系列）

- 复位　0000H
- 外部中断0　0003H
- 计时器T0溢出　000BH
- 外部中断1　0013H
- 计时器T1溢出　001BH
- 串行口中断　0023H
- 计时器T2/T2EX　002BH　（52子系列）

	内部	外部
RAM 数据	MOV 指令	MOVX 指令 RD、WR选通
ROM 程序	MOVC 指令 EA=1	MOVC 指令 PSEN选通 EA=0



堆栈 SP：存放当前堆栈顶所指存储单元地址的8位寄存器

PUSH 进栈 向上生长

POP 出栈 恢复现场

后进的先出

SP+1 ←→

SP ↑↓

RS1	RS0	R0-R7
0	0	00-07H
0	1	08-0FH
1	0	10-17H
1	1	18-1FH

RAM

8051内部数据存储器配置图

	(低128单元)		(高128单元)	
7FH		FFH		
		F0H	B	
		E0H	ACC	
	用户RAM区	D0H	PSW	专用寄存器区 SFR
		B8H	IP	
	(堆栈、数据缓冲)	B0H	P3	
		A8H	IE	
		A0H	P2	
		99H	SBUF	
30H		98H	SCON	
2FH		90H	P1	
	位寻址区 (位地址00H~7FH)	8DH	TH1	
20H		8CH	TH0	
1FH		8BH	TL1	
18H	第3组通用寄存器区	8AH	TL0	
17H		89H	TMOD	
10H	第2组通用寄存器区	88H	TCON	
0FH		87H	PCON	
08H	第1组通用寄存器区	83H	DPH	
07H		82H	DPL	
00H	第0组通用寄存器区	81H	SP	
		80H	P0	

位寻址区 20H-2FH → 00H → 7FH

单元地址	MSB ← 位地址 → LSB							
2FH	7FH	7EH	7DH	7CH	7BH	7AH	79H	78H
2EH	77F	76H	75H	74H	73H	72H	71H	70H
2DH	6FH	6EH	6DH	6CH	6BH	6AH	69H	68H
2CH	67H	66H	65H	64H	63H	62H	61H	60H
2BH	5FH	5EH	5DH	5CH	5BH	5AH	59H	58H
2AH	57H	56H	55H	54H	53H	52H	51H	50H
29H	4FH	4EH	4DH	4CH	4BH	4AH	49H	48H
28H	47H	46H	45H	44H	43H	42H	41H	40H
27H	3FH	3EH	3DH	3CH	3BH	3AH	39H	38H
26H	37H	36H	35H	34H	33H	32H	31H	30H
25H	2FH	2EH	2DH	2CH	2BH	2AH	29H	28H
24H	27H	26H	25H	24H	23H	22H	21H	20H
23H	1FH	1EH	1DH	1CH	1BH	1AH	19H	18H
22H	17H	16H	15H	14H	13H	12H	11H	10H
21H	0FH	0EH	0DH	0CH	0BH	0AH	09H	08H
20H	07H	06H	05H	04H	03H	02H	01H	00H

字节位 → 位地址

位地址
=
(字节地址-20H)×8 + 位号

例如：

20.2 → 02H
21.5 → 0DH
2A.3 → 53H

位地址 减20 ☆

按字节访问

MOV A, 20H

MOV 21H, #55H

按位访问

SETB 00H

CLR 1FH

JB 35H LABEL

80H-FFH

序号	标识符	名 称	字节地址	位 地 址
1	ACC	累加器	E0H	E0H~E7H
2	B	B 寄存器	F0H	F0H~F7H
3	PSW	程序状态字	D0H	D0H~D7H
4	SP	堆栈指针	81H	
5	DPTR	数据指针(DPH、DPL)	83H、82H	
6	P0	P0 □	80H	80H~87H
7	P1	P1 □	90H	90H~97H
8	P2	P2 □	A0H	A0H~A7H
9	P3	P3 □	B0H	B0H~B7H
10	IP	中断优先级控制寄存器	B8H	B8H~BFH
11	IE	中断允许控制寄存器	A8H	A8H~AFH
12	TOMD	定时器 / 计数器方式控制寄存器	89H	
13	TCON	定时器/计数器控制寄存器	88H	88H~8FH
14	T2CON	定时器/计数器 2 控制寄存器	C8H	C8~CFH
15	TH0	定时器/计数器 0(高位字节)	8CH	
16	TL0	定时器/计数器 0(低位字节)	8AH	
17	TH1	定时器/计数器 1(高位字节)	8DH	
18	TL1	定时器/计数器 1(低位字节)	8BH	
19	TH2	定时器/计数器 2(高位字节)	CDH	
20	TL2	定时器/计数器 2(低位字节)	CCH	
21	RLDH	定时器/计数器 2 自动重装载（高位字节）	CBH	
22	RLDL	定时器/计数器 2 自动重装载（低位字节）	CAH	
23	SCON	串行口控制寄存器	98H	98H~9FH
24	SBUF	串行数据缓冲器	99H	
25	PCON	电源控制及波特率选择寄存器	97H	

80H-FFH

0H、8H 结果的有位寻址

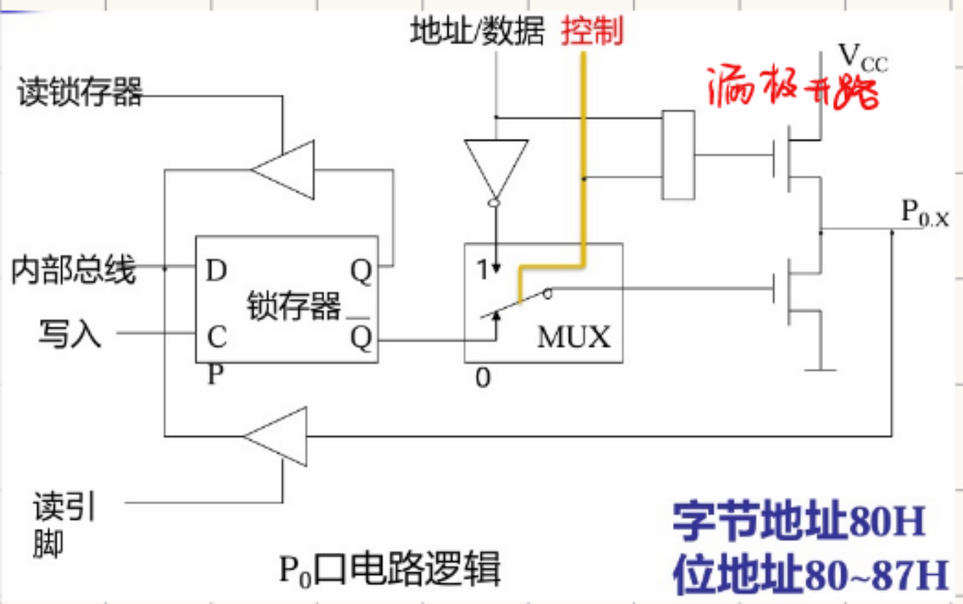
80H-FFH，部分有

数据指针 DPTR

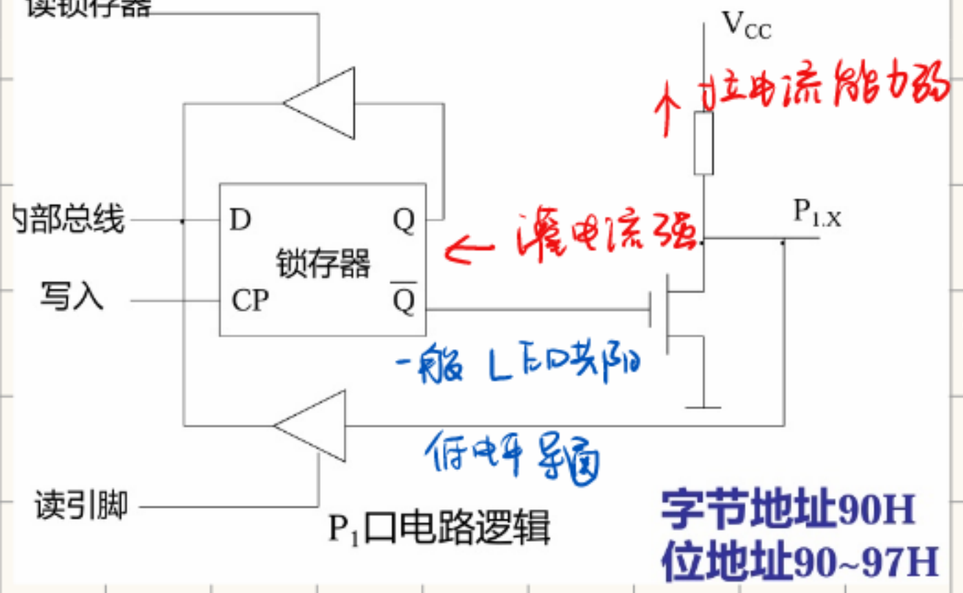
DPH 83H 高字节

DPL 82H 小字节

2.5 并行输入/输出端口



做通用 I/O 口要上拉电阻☆



80C51并行I/O接口的比较

I/O口	P_0口	P_1口	P_2口	P_3口
位数	8	8	8	8
性质	真正双向口	准双向口	准双向口	准双向口
功能	I/O口 替代功能	I/O口	I/O口 替代功能	I/O口 替代功能
SFR字节地址	80H	90H	A0H	B0H
位地址范围	80H~87H	90H~97H	A0H~A7H	B0H~B7H
驱动能力	8个TTL负载	4个TTL负载	4个TTL负载	4个TTL负载
拓展☆ 替代功能	地址总线低8位/8位数据总线		高8位地址总线	RXD、TXD、$\overline{INT0}$、T0、$\overline{INT1}$、T1、$\overline{WR}$、$\overline{RD}$

特殊功能寄存器设置

口线	第二功能	信号名称
$P_{3.0}$	RXD	串行数据接收
$P_{3.1}$	TXD	串行数据发送
$P_{3.2}$	$\overline{INT_0}$	外部中断0申请
$P_{3.3}$	$\overline{INT_1}$	外部中断1申请
$P_{3.4}$	T_0	定时器/计数器0计数输入
$P_{3.5}$	T_1	定时器/计数器1计数输入
$P_{3.6}$	$\overline{WR}$	外部RAM写选通
$P_{3.7}$	$\overline{RD}$	外部RAM读选通

MOVX

2.6 布尔处理器

2.7 工作方式

① 复位方式 RST 高电平 两个 机器周期

将 PC初始化为 0000H，ALE=0 $\overline{PSEN}$=1

② 低功耗方式

1° 待机

2° 掉电保护

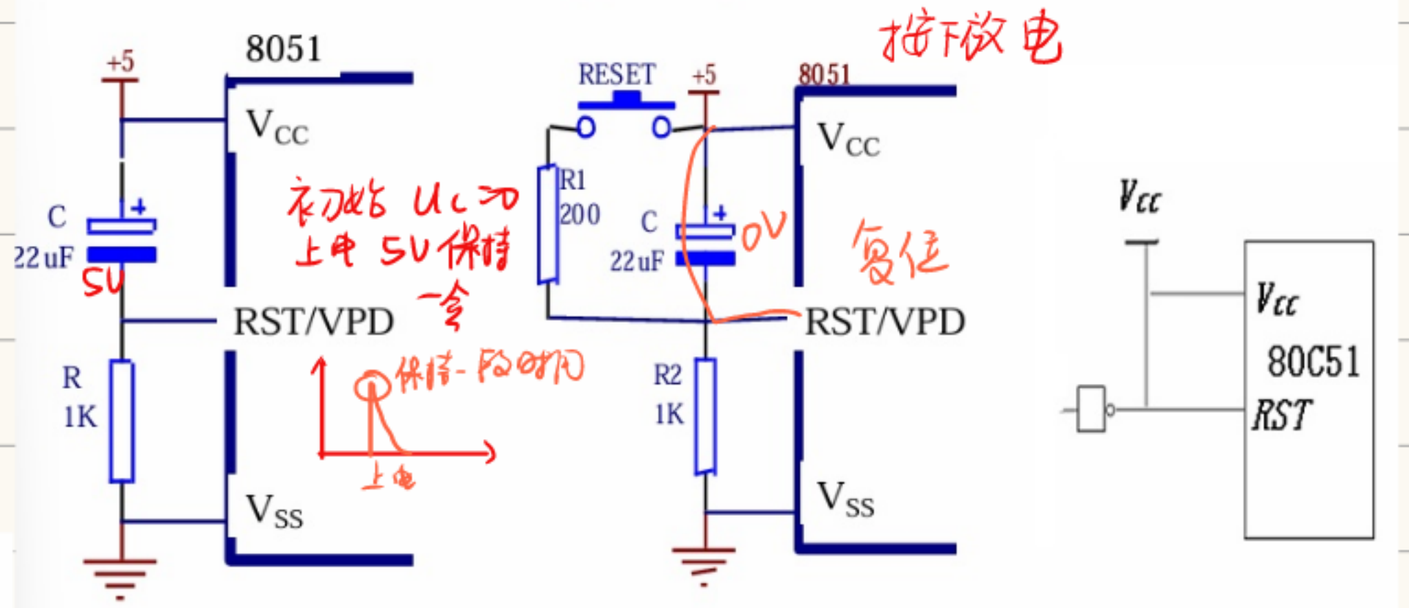
PCON寄存器

位序	D7	D6	D5	D4	D3	D2	D1	D0
位符号	SMOD	/	/	/	GF_1	GF_0	PD	IDL

SMOD 波特率倍增位，串行通信时使用
GF_0 通用标志位
GF_1 通用标志位
PD 掉电方式位，PD=1则进入掉电方式
IDL 待机方式位，IDL=1则进入待机方式

都打开：掉电模式

➢ 复位信号的产生: **三种方式**



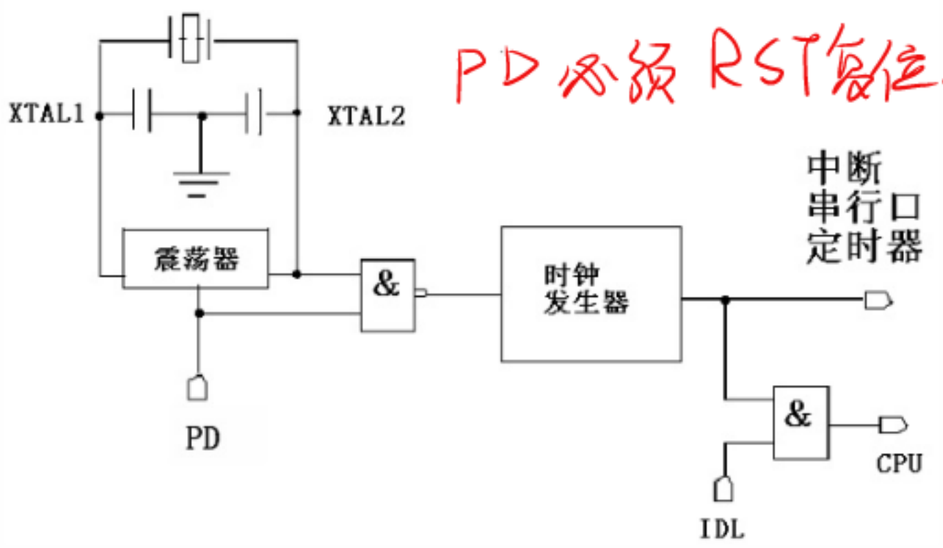
(a) 上电复位 (b) 按键电平复位 (c) 外部脉冲

➢ 待机方式以及掉电保护:

由PCON寄存器控制

PD=1 功耗低

PD必须RST复位恢复. IDL 要由一次中断才能恢复



第三章 80C51 单片微机指令系统

CISC 复杂指令集 指令长度不一

RISC 精简指令集 指令长度固定

指令 = 操作码 + 操作数 一个周期最多 2个字节 指令长度3字节，执行时间2个机器周期

3.1.1 寻址方式

① 立即寻址 MOV DPTR, #1234H

② 直接寻址 MOV A, 3AH

③ 寄存器寻址 INC R0

④ 寄存器间接寻址 MOV A, @R0 片内低128位只能 R0、R1 寻址

片外可用 DPTR 寻址；PUSH、POP 以 SP 作间接寻址寄存器

⑤ 相对寻址 JC 80H

⑥ 变址寻址 MOVC A, @A+DPTR

⑦ 位寻址 MOV C, 2BH

80C52高128字节RAM寻址必须用寄存器寻址，对特殊功能寄存器（位于高128字节地址）必须用直接寻址。

寄存器间接寻址只能用R0和R1。

CLR ADDR 是位寻址指令。

其它不合法指令，如

MOV R1,R0 R2不行

PUSH R0

寻址方式小结

序号	寻址方式	使用的变量	寻址空间
1	寄存器寻址	R0~R7、A、B、DPTR	
2	直接寻址		片内RAM低128B 专用寄存器
3	寄存器间接寻址	@R0、 @R1、SP	片内RAM
		@R0、 @R1、 @DPTR	片外RAM
4	立即寻址		程序存储器
5	位寻址		片内RAM的位寻址区 可以位寻址的专用寄存器位
6	变址寻址	@A+DPTR、 @A+PC	程序存储器
7	相对寻址	PC+偏移量	程序存储器

3.1.2 指令格式

单字节指令：

[操作码 | 地址码] MOV A, Rn

双字节指令：

[操作码 | 地址码] + [数据或地址码] MOV A, #data

三字节指令

[操作码] + [数据或地址码] + [数据或地址码] ANL direct, #data

3.1.4 符号说明

Rn：8个通用寄存器 R0~R7 n=0~7

Ri：间接寻址寄存器，只能 R0 R1 i=0,1

⋮

#data 8位立即数

#data16 16位立即数

addr16 16位目的地址

addr11 11位目的地址

rel 相对偏移量

3.2 数据传输类指令

通用格式：MOV <目的操作数>,<源操作数>

其他指令助记符：MOVX：外部RAM数据

MOVC：程序存储器数据

XCH/XCHD/SWAP：数据交换

PUSH/POP：堆栈数据

内部RAM数据传送指令组：16条

指令格式：MOV　<目的字节>，<源字节>

特点：复制性质

1）以累加器A为目的操作数的指令组

MOV　A，Rn　　；(A) ←(Rn)　　R0-R7　　1字节

MOV A，direct　；(A)←(direct)　　00—0FFH　　2字节

MOV A，@Ri　；(A)←((Ri))　　00—0FFH　　1字节

MOV A，#data；(A)←#data　　#00 - #0FFH　　2字节

2）以工作寄存器Rn为目的操作数的指令组

MOV　Rn，A　；(Rn)←(A)

MOV　Rn，direct　　；(Rn)←（direct)

MOV　Rn，#data　　；(Rn)←#data0

direct：RAM 0 ~ 127个地址单元和128 ~ 255地址的特殊功能寄存器

3）以直接地址direct（直接寻址）为目的操作数的指令组

MOV　direct，A　　；(direct)←(A)

MOV　direct，Rn　　；(direct)←(Rn)

MOV　direct，direct　　；(direct)←（direct)　　3字节　2周期

MOV　direct，@Ri　；(direct)←((Ri))

MOV　direct，#data　　；(direct)←#data

- 源操作数有寄存器寻址、直接寻址、寄存器间接寻址和立即寻址等寻址方式。
- direct：RAM 0 ~ 127个地址单元和 128 ~ 255地址的特殊功能寄存器

4）以间接寻址寄存器Ri为目的操作数的指令组

MOV　@Ri, A　　; ((Ri)) ← (A)

MOV　@Ri, direct　　; ((Ri)) ← (direct)

MOV　@Ri, #data　　; ((Ri)) ← #data

说明：

@Ri 表示寄存器间接寻址（Ri通常为R0或R1）。

第一条：将累加器A中的数据送到Ri所指向的RAM单元。

第二条：将直接地址单元中的数据送到Ri所指向的RAM单元。

第三条：将立即数data送到Ri所指向的RAM单元。

5）16位数据传输指令

MOV DPTR, #data16；（DPTR）← #data16　　3字节

DPTR是寄存器寻址的，

与MOV A, #data属于一类指令，只是数据是16位的

唯一的一条16位数据传输指令

外部RAM数据传送指令组：4条　　一定要MOVX

1）使用DPTR进行间接寻址

MOVX　A, @DPTR　　; A ← ((DPTR))

MOVX　@DPTR, A　　; (DPTR) ← (A)　　可读可读

2）使用Ri进行间接寻址

MOVX　A, @Ri　　; A ← ((Ri))

MOVX　@Ri, A　　; (Ri) ← (A)

读程序存储器数据指令：2条　　ROM

共2条，通常用于查表　　只读

MOVC　A, @A+PC

MOVC　A, @A+DPTR

DPTR为基址寄存器时，允许数表存放在程序存储器的任意单元，称为远程查表，编程比较直观；

而PC为基址寄存器时，数表只能放在该指令单元往下的256个单元中，称为近程查表。

数据交换指令:

1）整字节交换指令

XCH　A, Rn　　; (A) ↔ (Rn)

XCH　A, direct　; (A) ↔ (direct)

XCH　A, @Ri　　; (A) ↔ ((Ri))

特点：交换而非复制性质

例：设 (R0)=30H，(A)=3FH，片内 (30H)=BBH。

执行指令　XCH A,@R0

执行结果　(A)=BBH，(30H)=3FH

2）半字节交换指令

XCHD A, @Ri　; (A)3~0 ↔ ((Ri))3~0

将Ri间接寻址单元的低4位内容与累加器A的低4位内容互换，而它们的高4位内容均不变。此指令不影响标志位。

例：设 (R0)=20H，(A)=36H（00110110B），内部RAM中 (20H)=75H（01110101B）。

执行指令：XCHD A, @R0

结果为：(20H) = 01110110B = 76H　(A) = 00110101B = 35H

3）累加器高低半字节交换指令

SWAP A　; (A)3~0 ↔ (A)7~4

说明：SWAP A 用于将累加器 A 的高4位（D7~D4）与低4位（D3~D0）互换。

即：A = [高4位][低4位]

执行 SWAP A 后 A = [低4位][高4位]

堆栈操作指令组:

进栈指令

内部RAM低128个单元或专用寄存器内容送栈顶单元

PUSH direct　; SP ← (SP)+1，(SP) ← (direct)

出栈指令

栈顶单元内容送内部RAM低128个单元或专用寄存器

POP direct　; direct ← ((SP))，SP ← (SP)-1

PUSH POP示例

例：中断响应时(SP) = 30H，DPTR的内容为0123H，执行入栈指令其结果怎样?

PUSH　DPL　；低8位数据指针寄存器DPL内容入栈

PUSH　DPH　；高8位数据指针寄存器DPH内容入栈。

执行结果:

第一条指令(SP) + 1 = 31H→(SP),(DPL) = 23H→(31H)

第二条指令(SP) + 1 = 32H→(SP),(DPH) = 01H→(32H)

RAM:　(31H) = 23H，(32H) = 01H，(SP) = 32H

例：设（SP）= 32H，片内RAM的30H~32H单元中的内容分别为20H，23H，01H，执行下列指令的结果怎样?

POP DPH　;　((SP)) = (32H) = 01H→DPH

(SP) - 1 = 32H - 1 = 31H→SP

POP DPL　;　((SP)) = (31H) = 23H→DPL

(SP) - 1 = 31H - 1 = 30H→SP

POP SP　;　(SP) - 1 = 2FH→SP

((SP)) = (30H) = 20H→SP

以上第三条指令为特殊情况，先执行（SP）- 1 = 2FH，后装入由栈顶退出的值，所以执行后（SP）= 20H。

一般传送指令传送数据关系

目的操作数 \ 源操作数	A	Rn	direct	@Ri	#data
A		√	√	√	√
Rn	√		√		√
direct	√	√	√	√	√
@Ri	√		√		√

3.3 算术运算类指令

1.加法指令组　必须是累加器A

ADD　A, Rn　　; A ← (A) + (Rn)

ADD　A, direct　; A ← (A) + (direct)

ADD　A, @Ri　　; A ← (A) + ((Ri))

ADD　A, #data　; A ← (A) + data

Rn：工作寄存器寻址

direct：直接寻址

@Ri：寄存器间接寻址

#data：立即数寻址

A：累加器（Accumulator）

加法运算影响 PSW 位的状态。如果 3 有进位，AC 置 1；若 7 有进位，CY 置 1

带符号数运算有溢出问题，OV=1 表示有溢出状态出现 正正得负 负负得正 } OV=1 正负 OV=0

2.带进位加法指令组

ADDC A, Rn ; A ← (A) + (Rn) + (CY)

ADDC A, direct ; A ← (A) + (direct) + (CY)

ADDC A, @Ri ; A ← (A) + ((Ri)) + (CY)

ADDC A, #data ; A ← (A) + data + (CY)

其中:

ADDC：带进位加法指令（Add with Carry）

A：累加器（Accumulator）

Rn：工作寄存器

direct：直接寻址单元

@Ri：寄存器间接寻址

#data：立即数

CY：进位标志位（Carry Flag）

3. 加1指令组

INC A ; A ← (A) + 1

INC Rn ; Rn ← (Rn) + 1

INC direct ; direct ← (direct) + 1

INC @Ri ; (Ri) ← ((Ri)) + 1

INC DPTR ; DPTR ← (DPTR) + 1

加1操作不影响PSW

4. 十进制调整指令

DA A

➢ 十进制调整指令是紧接着加法指令用的，不能在其它运算后作十进制调整。

➢ 被加数和加数是十进制BCD码，执行ADD或ADDC二进制加法指令，再用DA A作十进制调整后，得到的结果是十进制BCD码。

- 累加器低4位大于9或辅助进位标志位（AC）=1，则进行低4位加6修正：

  A←（A）+06H

- 累加器高4位大于9或进位标志（CY）=1，则进行高4位加6修正：

  A←（A）+60H

- 累加器高4位为9，低4位大于9，则进行高低4位均加6修正：

  A←（A）+66H

例：设（A）=0101 0110=56 BCD，（R3）= 0110 0111=67 BCD，（CY）=1。

执行下述二条指令： ADDC A，R3 56+67+1=124

DA A

执行 ADDC A，R3 （A）0 1 0 1 0 1 1 0 (56 BCD)

(R3) 0 1 1 0 0 1 1 1 (67 BCD) BCD加法

\+ （CY） 1

10 1 1 1 1 1 0 (高、低四位均大于9)

再执行DA A 0 1 1 0 0 1 1 0 (加66H操作)

Cy=1 0 0 1 0 0 1 0 0 (124 BCD) BCD码

即BCD码数56＋67＋1＝124。经DA A指令校正后，答案正确。

5. 带借位减法指令组

SUBB A, Rn ; A ← (A) − (Rn) − (CY)

SUBB A, direct ; A ← (A) − (direct) − (CY)

SUBB A, @Ri ; A ← (A) − ((Ri)) − (CY)

SUBB A, #data ; A ← (A) − data − (CY)

6. 减1指令组

DEC A ; A ← (A) − 1

DEC Rn ; Rn ← (Rn) − 1

DEC direct ; direct ← (direct) − 1

DEC @Ri ; (Ri) ← ((Ri)) − 1

- 减1操作不影响PSW
- DPTR没有减1指令

7. 乘除指令组

MUL AB

DIV AB

乘法和除法操作都影响PSW

乘法和除法指令都是一字节指令，但执行时间都是4个机器周期。

A B 两个8位无符号数相乘，乘积16位，低8位放A，高8位放在B。 { 乘积小于等于FFH，OV=0 否则 OV=1

两个8位无符号数相除，A/B，商在A，余数在B，CY=0 { 除数0，OV=1 除数非0 OV=0.

加1和减1指令中的源操作数对象

助记符	A	Rn	direct	@Ri	DPTR
INC	√	√	√	√	√
DEC	√	√	√	√	

算术运算类指令中的操作数，**（一个是A）**

	源 操 作 数					
助记符	B	Rn	direct	@Ri	#data	DPTR
ADD		√	√	√	√	
ADDC		√	√	√	√	
SUBB		√	√	√	√	
MUL	√					
DIV	√					

3.4 逻辑运算类指令

- 各类逻辑运算：与、或、异或、清除、求反、移位等
- 助记符有ANL、ORL、XRL、RL、RLC、RR、RRC、CPL、CLR等九种。
- 只按位进行逻辑运算，结果不影响PSW中标志位。

1. 逻辑与运算指令组

```
ANL A, Rn          ; (A) ← (A) ∧ (Rn)
ANL A, direct      ; (A) ← (A) ∧ (direct)
ANL A, @Ri         ; (A) ← (A) ∧ ((Ri))
ANL A, #data       ; (A) ← (A) ∧ #data
ANL direct, A      ; (direct) ← (direct) ∧ (A)
ANL direct, #data  ; (direct) ← (direct) ∧ #data
```

按位与，用来做清零或屏蔽某些位

0FFH	1	1	1	1	1	1	1	1
0F0H	1	1	1	1	0	0	0	0
AND	1	1	1	1	0	0	0	0

置0

2. 逻辑或运算指令组

```
ORL A, Rn          ; A ← (A) ∨ (Rn)
ORL A, direct      ; A ← (A) ∨ (direct)
ORL A, @Ri         ; A ← (A) ∨ ((Ri))
ORL A, #data       ; A ← (A) ∨ data
ORL direct, A      ; direct ← (direct) ∨ (A)
ORL direct, #data  ; direct ← (direct) ∨ data
```

0A3H	1	0	1	0	0	0	1	1
45H	0	1	0	0	0	1	0	1
OR	1	1	1	0	0	1	1	1

置1

3. 逻辑异或运算指令组

```
XRL A, Rn          ; A ← (A) ⊕ (Rn)
XRL A, direct      ; A ← (A) ⊕ (direct)
XRL A, @Ri         ; A ← (A) ⊕ ((Ri))
XRL A, #data       ; A ← (A) ⊕ data
XRL direct, A      ; direct ← (direct) ⊕ (A)
XRL direct, #data  ; direct ← (direct) ⊕ data
```

10100011

⊕ 01000101

结果 11100110

4. 移位指令组

（1）循环左移

RL A

（2）循环右移

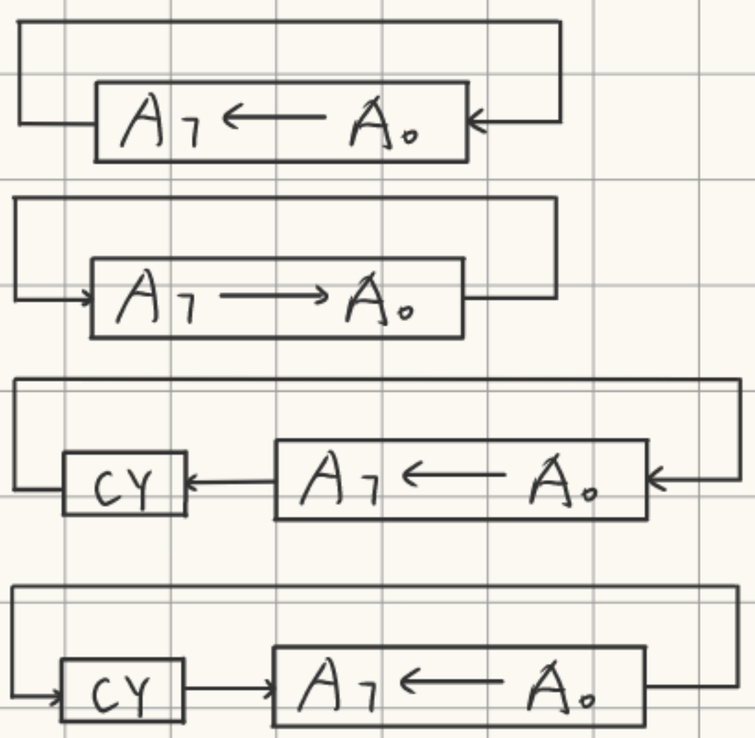
RR A

（3）带进位循环左移

RLC A

（4）带进位循环右移

RRC A



5. 累加器清0取反指令组

累加器清0指令：

CLR　A　　；A ← 0

累加器按位取反指令：

CPL　A　　；A ← $(\overline{A})$

双操作数逻辑运算指令
两个操作数之间的配合关系

		第二操作数				
		A	Rn	direct	@Ri	#data
第一操作数	A		√	√	√	√
	direct	√				√

3.5 控制程序转移类指令

➢ 共分4类：12种助记符

无条件转移指令：AJMP、LJMP、SJMP

条件转移指令：JZ、JNZ、

循环转移指令：CJNE、DJNZ、

子程序调用和返回指令：ACALL、LCALL、RET、RETI、

空操作指令：NOP

➢ 不包括布尔变量控制程序转移指令。

1、无条件转移指令

指令	机器码		
SJMP　rel	80	rel	
AJMP　addr11	a10 a9 a8 00001	a7---a0	
LJMP　addr16	02	addr15-8	addr7-0
JMP　@A+DPTR	73		

长度
2字节
2字节
3字节

2^8

➢ 功能：程序无条件地转移到各自指定的目标地址去执行

(1) 短转移指令　SJMP rel；　rel是带符号的8位二进制补码

目的地址 = (PC)+2 + rel　　短转范围 256字节地址

如 SJMP　7FH；正数　　-127~+127

目的地址 2100H+2+7FH = 2181H

SJMP　80H；负数

目的地址 2100H+2-80H = 2082H

例：0456H　AJMP　123H

PC ← (PC) + 2,
$PC_{10\sim0}$ ← addr11

(PC) + 2 = 0458H
目的地址 = 0123H

只改后面11位

0000 0100	0101 1000	PC+2
0000 0001	0010 0011	目标地址

(2) 绝对转移指令　AJMP addr11　　2^{11} = 2K字节

PC ← (PC)+2，PC_{10-0} ← addr11　　PC加2后替换后11位

(3) 长转移指令 LJMP addr16

设 (PC) = 0123H, ADR指向3456H

执行指令后 (PC) = 3456H

(4) 间接转移指令 JMP @A+DPTR

设 (A) = 5 (DPTR) = 4567H

(PC) = (A) + (DPTR) = 456CH

2、条件转移指令组

1) 累加器判零转移指令

2^{16} = 64K字节

例11：散转程序设计，根据A中的数值实现程序散转

```
        MOV     R1,A          ；(A)×3
        RL      A
        ADD     A,R1          A变3倍了
        MOV   DPTR ，#TABLE ；散转表首地址送DPTR
        JMP     @A+DPTR
TABLE:  LJMP    PM0           ；转程序PM0
TABLE+3: LJMP   PM1           ；转程序PM1
        ------
PM0:    -----
```

LJMP是一个三字节指令，因此转移指令入口地址相隔3个字节，A中内容需是3的倍数。

JZ rel ；若(A)=0，则PC ← (PC)+2+rel

；若(A)≠0，则PC ← (PC)+2

JNZ rel ；若(A)≠0，则PC ← (PC)+2+rel

；若(A)=0，则PC ← (PC)+2

特点:

- 偏移量rel是带符号的8位数，程序转移的目标地址为指令前128B或指令后127B。
- 指令执行后不影响任何操作数和标志位

2）数值比较转移指令

CJNE A, #data, rel ；累加器内容与立即数不等转移

CJNE A, direct, rel ；累加器内容与内部RAM单元内容不等转移

CJNE Rn, #data, rel ；寄存器内容与立即数不等转移

CJNE @Ri, #data, rel ；内部RAM前128单元内容与立即数不等转移

两操作数的比较不影响操作数的值，但影响CY标志，CY的值由无符号数目的操作数—源操作数有无借位决定。

相等 继续执行

不等 (PC)+3+rel

< 会借位

CJNE指令中源操作数与目的操作数的关系

目的操作数	源操作数 direct	源操作数 #data
A	√	√
Rn		√
@Ri		√

3）减1条件转移指令

寄存器减1条件转移指令

DJNZ Rn, rel

；Rn ← (Rn) - 1

；若(Rn) ≠ 0，则PC ← (PC) + 2 + rel

；(Rn) = 0，则PC ← (PC) + 2

减0转移

- 直接寻址单元减1条件转移指令

DJNZ direct, rel

；direct ← (direct) - 1

；若(direct) ≠ 0，则PC ← (PC) + 3 + rel

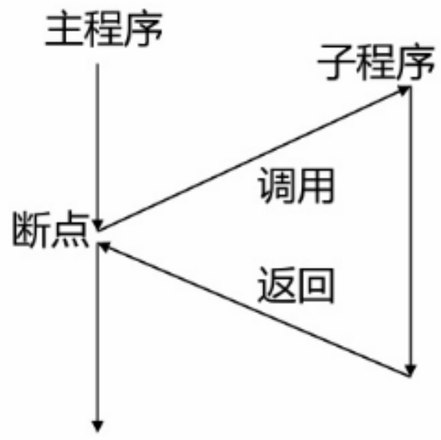
；(direct) = 0，则PC ← (PC) + 3

3、子程序调用和返回指令组

1）绝对调用指令

ACALL addr11

$a_{10}a_9a_81$	0 0 0 1
$a_7a_6a_5a_4$	$a_3a_2a_1a_0$



子程序调用示意图

① PC←(PC)+2
② SP←(SP)+1，(SP)←$(PC)_{7\sim0}$
③ SP←(SP)+1，(SP)←$(PC)_{15\sim8}$
④ $PC_{10\sim0}$ ←addr11

例：设（SP）=60H，（PC）=0123H，子程序SUBRTN的首地址为0456H。

执行指令：ACALL SUBRTN

100 1	0 0 0 1
0101	0110

机器码： 91 56

执行结果：

① (PC) + 2 = 0123H + 2 = 0125H→ (PC)

② ③ PC压栈：25H 压入(SP) + 1 = 61H，01H压入(SP) + 1 = 62H，此时（SP）=62H。

④ 构造目标地址：addr11 → PC10–0 PC=0456H

2）长调用指令

LCALL addr16 ；PC←(PC)+3
；SP←(SP)+1，(SP)←$(PC)_{7\sim0}$
；SP←(SP)+1，(SP)←$(PC)_{15\sim8}$
；PC ←addr16

3）返回指令

RET ；子程序返回指令
RETI ；中断服务程序返回指令
$(PC)_{15\sim8}$ ← (SP)，SP←(SP) – 1
$(PC)_{7\sim0}$ ← (SP)，SP←(SP) – 1

4、空操作指令

3.6 布尔(位)操作类指令

1. 位传送指令

MOV C, bit ; C ← (bit)
MOV bit, C ; bit ← (C)

- 无法对2个可寻址位直接进行传输 MOV bit, bit
- 对I/O端口操作：读-修改-写

例：设(C)=1，执行指令为 MOV P1.3, C，执行结果：P1.3=1
例：设P1口的内容为00111010B，执行指令如 MOV C, P1.3，执行结果为(C)=1。

2. 布尔状态控制指令

- 清零：

CLR C ; C ← 0 清零
CLR bit ; bit ← 0 清零

- 置位：

SETB C ; C ← 1
SETB bit ; bit ← 1

- 取反：

CPL C ; C ← $\overline{(C)}$
CPL bit ; bit ← $\overline{(bit)}$

3. 布尔(位)逻辑操作指令

与，或，（取反）

ANL C, bit　　; C ← (C) ∧ ($\overline{bit}$)

ANL C, $\overline{bit}$　　; C ← (C) ∧ (bit)

ORL C, bit　　; C ← (C) ∨ ($\overline{bit}$)

ORL C, $\overline{bit}$　　; C ← (C) ∨ (bit)

4. 布尔（位）条件转移指令

1）布尔累加器C条件转移指令　　直接取C

• (C)=1转移指令:　　看CY位

JC　rel　　; 若(CY)=1，则PC ← (PC)+2+rel

　　; 若(CY)≠1，则PC ← (PC)+2

• (C)=0转移指令:　　指令长度

JNC rel　　; 若(CY)=0，则PC ← (PC)+2+rel

　　; 若(CY)≠0，则PC ← (PC)+2

2）位测试条件转移指令　　看选中bit位

JB　bit, rel　　; 位状态为1转移

JNB　bit, rel　　; 位状态为0转移

• 操作不影响标志位。

• 三字节指令

满足条件:

　　PC ← (PC) + 3 + rel

不满足条件:

　　PC ← (PC) + 3

3）位测试条件转移并清0指令

JBC bit, rel　　C代表 clear bit

　　; 位状态为1转移，并使该位清0

• 操作不影响标志位。

• 三字节指令

满足条件:　　PC跳转

　　PC ← (PC) + 3 + rel

不满足条件:

　　PC ← (PC) + 3

判断普通字节地址还是位地址：

关键要看：这条指令要求的是字节地址还是位地址。

不要看数字:

02H

8CH

20H

88H

而要看指令。

位操作指令

后面必须跟位地址:

SETB bit

CLR bit

CPL bit

JB bit,label

JNB bit,label

JBC bit,label

MOV C,bit

MOV bit,C

例如:

SETB TR0

CLR TF0

JB RI,RECV

都是位寻址。

字节操作指令

后面必须跟字节地址:

MOV A,direct

MOV direct,A

ANL direct,#data

ORL direct,#data

INC direct

DEC direct

例如:

MOV A,20H

MOV 30H,A

INC 40H

都是字节寻址。

第四章 程序设计

4.1 概述

计算机语言种类：机器语言，汇编语言，高级语言

4.1.2 伪指令语句

(1) ORG <表达式>

(2) <标号> END <表达式>

(3) <标号> EQU <表达式>

(4) <标号> DL <表达式>

(5) <标号> DB <表达式或表达式表> 将数据存入标号开始连续单元

```
        ORG 2000H
TABLE:  DB  73H, 04H, 100, 32, 00, -2
        DB  "ABC"
```

2000H	73H
	04H
	64H
	20H
	00H
	FEH
41H	= A
42H	= B

取A,B ASCII码

(6) <标号> DW <表达式或表达式表>

(7) <标号> DS <表达式表>

(8) <字符名称> BIT <位地址>

Little endian：一个Word中的低位的Byte放在内存中这个Word区域的低地址处。Cortex系统一般采用该方式。

Big endian：一个Word中的高位的Byte放在内存中这个Word区域的低地址处。80C51系统16位表格存放（DW）采用该方式。

```
        ORG     8100H
        DW  100H, 3456H, 814
8100H 01H           ; 高位低地址
8101H 00H
8102H 34H
```

4.2 结构

顺序结构、分支结构、循环结构、子程序设计

第五章 中断系统

5.1 概述

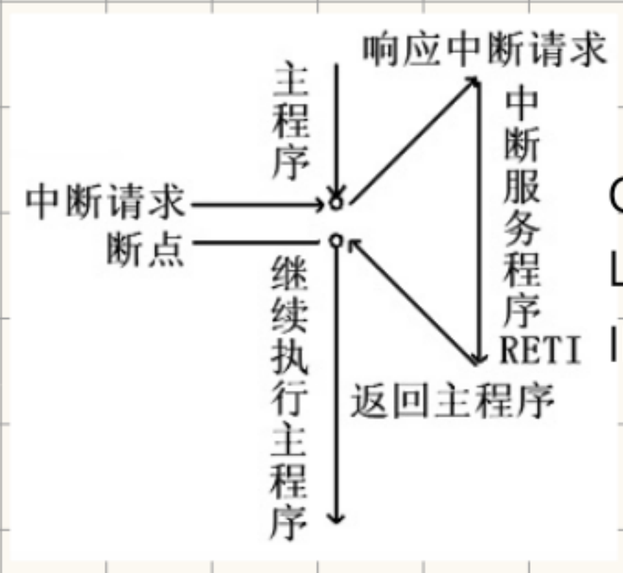
中断程序不能和主程序冲突

定义：程序执行过程中，允许外部或内部事件通过硬件打断程序的执行，使其转向为处理外部或内部事件的中断服务程序中去；完成中断服务程序后，CPU继续原来被打断的程序，这样的过程称为中断响应过程。

② 中断源

③ 中断优先级 (80C51 为 2 级)

④ 中断嵌套




```
ORG    0013H      ;;中断地址
LJMP   INT_1
INT_1:
       PUSH   PSW
       PUSH   ACC
       SETB   RS0
       CLR    RS1
       .....
       POP    ACC
       POP    PSW
       RETI
```

PSW里的

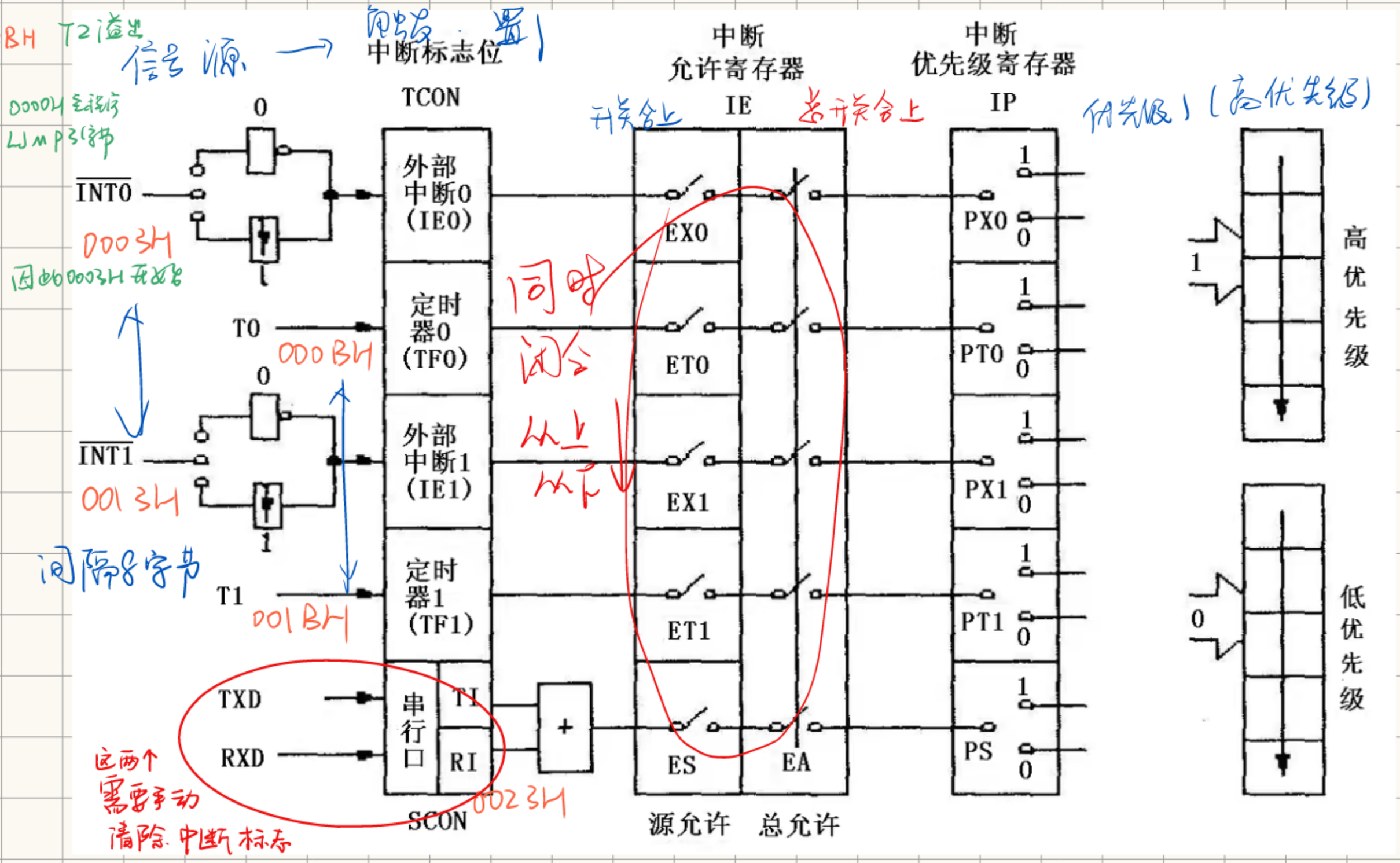
POP同时恢复了RS0,RS1

5.2 中断系统结构

5.2.1 中断源 (5个中断源, 2个外部, 3个内部)

0003H	$\overline{INT0}$ (P3.2)	IT0=0，低电平有效	IT0=1 下降沿有效
0013H	$\overline{INT1}$ (P3.3)	IT1=0，低电平有效	IT1=1 下降沿有效
000BH	TF0 (P3.4)	(T0溢出)	
001BH	TF1 (P3.5)	(T1溢出)	
0023H	RXD (P3.0)	TXD (P3.1)	串行中断
002BH	T2溢出		

信号源 → 中断标志位 (自动置1)



5.3 中断控制

☆ 定时控制器寄存器 TCON ($\overline{INT0}$、$\overline{INT1}$、T0、T1)

位地址	TCON.7	…	TCON.5	…	TCON.3	TCON.2	TCON.1	TCON.0
符号	TF1		TF0		IE1	IT1	IE0	IT0

T1溢出 硬件自动复位 T0溢出

外部中断1 IE=1中断中断

外部中断0

0低电平有效发
1下降沿有效发

串行口控制寄存器 SCON (RXD TXD)

位地址 ... SCON.1 SCON.0

位符号 TI 需手动清零 RI

串行口发送 串行口接收

中断允许控制寄存器 IE

位地址	IE.7	...	IE.4	IE.3	IE.2	IE.1	IE.0
位符号	EA	...	ES	ET1	EX1	ET0	EX0
	CPU中断允许		串行中断允许	T1中断允许	外部中断1	T0中断允许	外部中断0

中断优先级控制寄存器 IP

位地址	...	IP.4	IP.3	IP.2	IP.1	IP.0
位符号		PS	PT1	PX1	PT0	PX0

PX0、PX1 外中断优先级设定位

PT0、PT1 定时中断优先级设定位

PS 串行中断优先级设定位

0 优先级低 1 优先级高

未设置时：在主程序运行中，同级的多个中断请求同时出现时，响应次序为：
外中断0→定时中断0 →外中断1→定时中断1 →串行口中断

从上到下

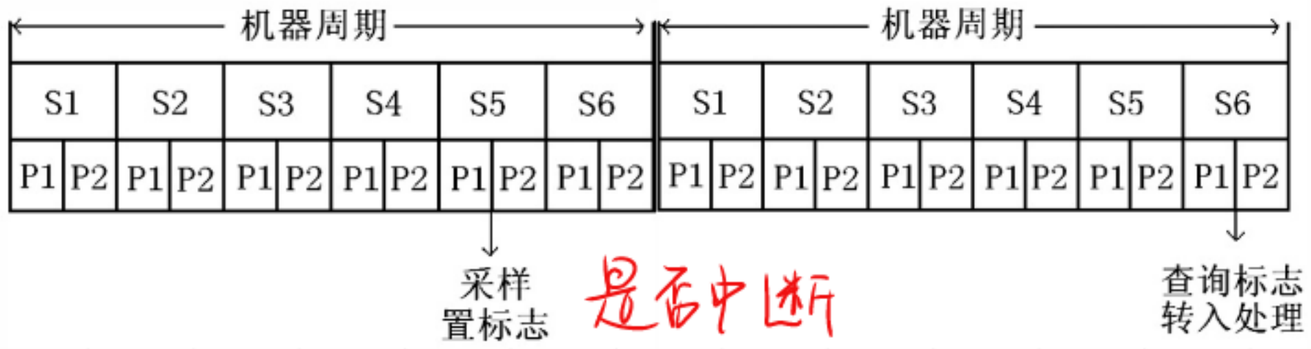
响应优先级

抢断优先级

只能 RETI返回，RET 是子程序返回

5.4 中断响应时间

每个机器周期的S5P2(第五状态的第二节拍)期间，各中断标志采样相应的中断源，并置入相应标志



是否中断 优先级

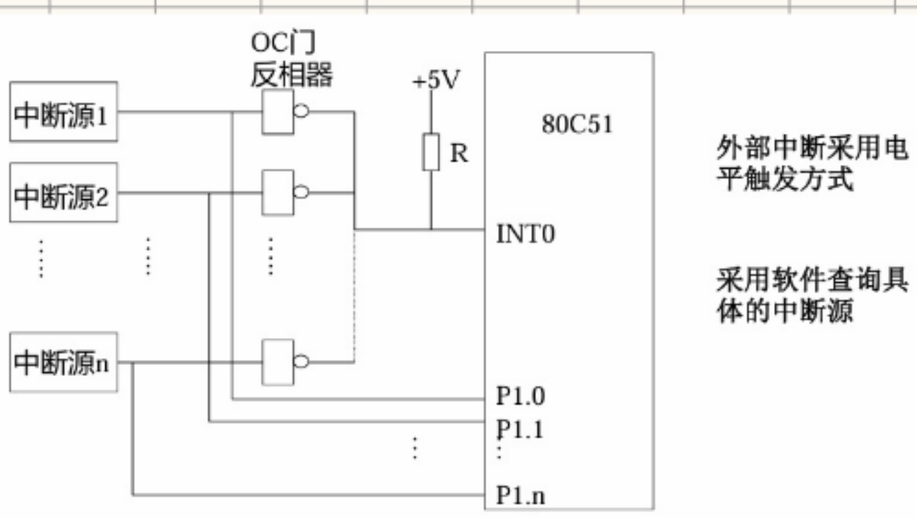
中断响应时间

最短：3个机器周期 (中断请求标志位查询 1个 + LCALL 2个)

最长：8个机器周期 (正在执行 RETI 或访问 IP,IE：2个；MUL,DIV：4个；LCALL 2个)

5.5 外部中断源拓展

通过OC门实现



5.7 中断服务程序设计

中断现场保护

PUSH PSW

PUSH ACC

⋮

POP ACC

POP PSW

执行完后必须再执行一条指令

不立即响应

例. 试编写设置外部中断INT0和串行接口中断为高优先级，外部中断INT1为低优先级。屏蔽T0 和T1中断请求的初始化程序段。

根据题目要求，只要能将中断请求优先级寄存器IP的第0、4位置“1”。其余位置“0”，将中断请求允许寄存器的第0、2、4、7位置“1”，其余位置“0”就可以了。

```
MAIN: …
      MOV IP, #00010001B ; 设外部中断INT0和
                         ; 串行口中断为高优先级
      MOV IE. #10010101B; 允许INT0、INT1、
                         ; 串行口中断，开CPU中断
```

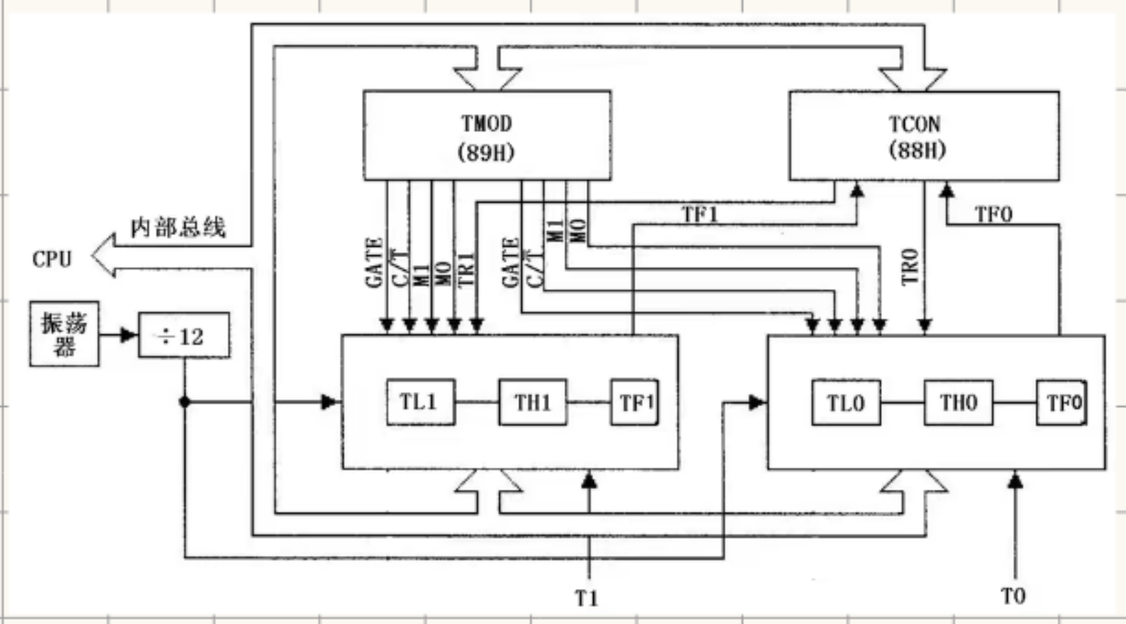
```
ORG   0000H
SJMP  MAIN
ORG   0003H
LJMP  INT0INT      ; 设外部中断中断矢量
ORG   0013H
LJMP  INT1INT      ; 设外部中断中断矢量
ORG   0023H
LJMP  SIOINT ; 设串行口中断矢量
ORG   0030H
```

第六章 定时器/计数器原理

6.1 概述

定时实现方法 (1) 硬件法 (2) 软件法 (3)可编程定时器/计数器

6.2 定时器/计数器 T0、T1 (核心是加1的计数器)



计数器 TH0、TL0 和 TH1、TL1;

特殊功能寄存器 TMOD、TCON;

时钟分频器;

输入引脚 T0、T1。

TMOD: 工作方式 (不能位寻址)

位号	B_7	B_6	B_5	B_4	B_3	B_2	B_1	B_0
位符号	GATE	$C/\overline{T}$	M_1	M_0	GATE	$C/\overline{T}$	M_1	M_0
		T_1					T_0	

(1) GATE: 门控位

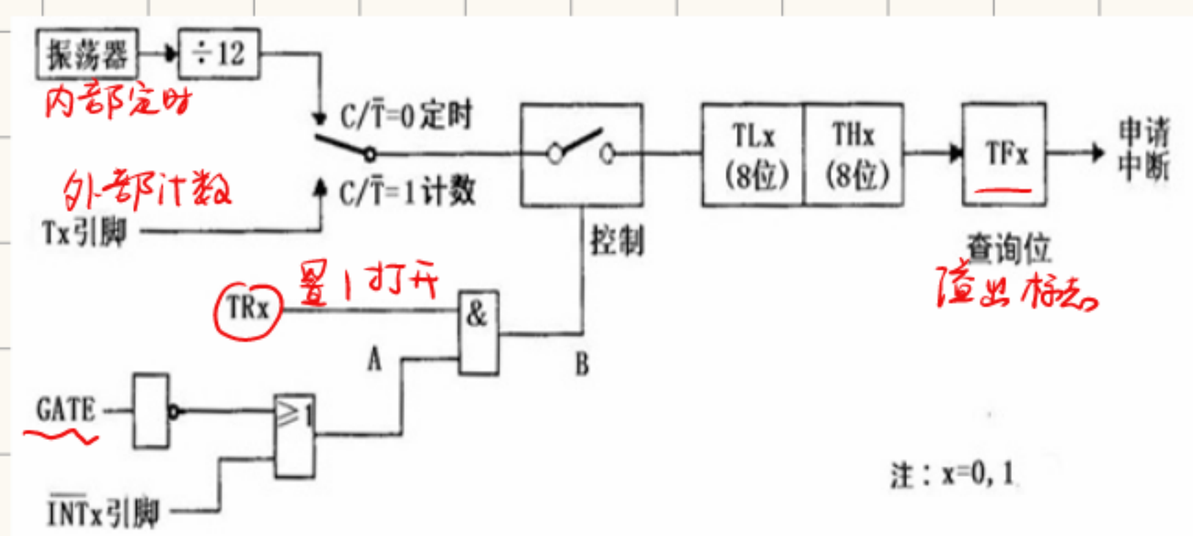
GATE=1 由 $\overline{INT0}$、$\overline{INT1}$ 和 TR0、TR1 共同启动定时器 (外部输入)

GATE=0 由 TR0、TR1 启动

(2) $C/\overline{T}$: 功能选择位

$C/\overline{T}=1$ 计数

$C/\overline{T}=0$ 定时



(3) M1, M0 工作方式选择位

M1	M0	方式	功能
0	0	方式0	13位计数器
0	1	方式1	16位计数器
1	0	方式2	自动重装载8位
1	1	方式3	T0分为两个8位计数器, T1停止工作

TCON 控制寄存器 (可以位寻址)

位地址	8FH	8EH	8DH	8CH	8BH	8AH	89H	88H
符号	TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0
	T1溢出	T1运行控制 1启动	T0溢出	T0运行控制 1启动	外部中断1		外部中断0	

方式0: 13位定时,计数器 定时时间 = $(2^{13}-$ 计时初值$)$ 个机器周期

方式1: 16位定时,计数器 定时时间 = $(2^{16}-$ 计时初值$)$ 个机器周期

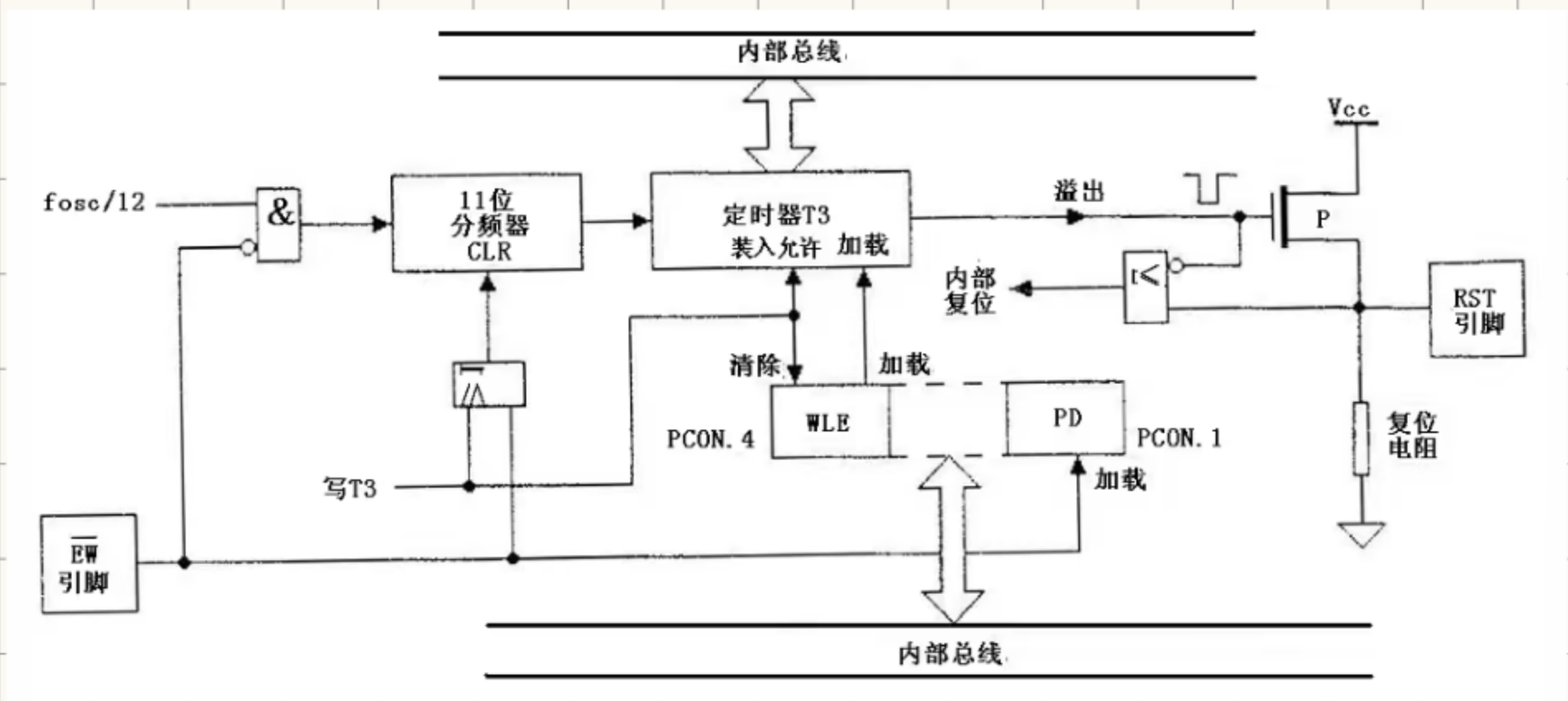
} 可由方式1实现 (TH×8位 TL×5位 ☆)

方式2: TLx 8位计数寄存器 THx 常数寄存器 TLx溢出重装THx 不用中断

方式3: TL0 8位定时/计数器 TH0 8位定时器

6.3 T3 监视定时器

$\overline{EW}=0$ 允许　$\overline{EW}=1$



PCON 专用寄存器

位序	D7	D6	D5	D4	D3	D2	D1	D0
位符号	SMOD	/	/	WLE	GF1	GF0	PD	IDL

PD(PCON.1) 掉电方式位，PD=1则进入掉电方式

WLE(PCON.4) 看门狗定时器允许重装标志

WLE置位，定时器T3只能被软件装入，装入后WLE自动清除

例 1：要求在 P1．0引脚上产生周期为2 ms的方波输出。

已知晶体振荡器的频率为fosc=6MHz。用T0作定时器，设为方式0，设定 1ms的定时，每隔1ms使P1.0引脚上的电平变反。

第一步：定时常数计算

方式0计算！

fosc = 6MHz → Tc = 2μs，

方式0计数器长度L = 13 ($2^{13}=8192$)，

$$t = T_c \times (2^L - TC) = \frac{12}{f_{osc}}(2^L - TC)$$

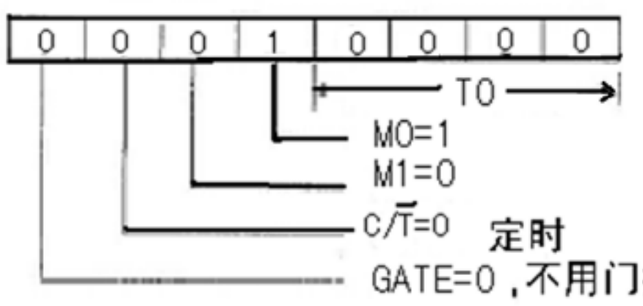
定时时间 t = 1ms = 0.001s

注意13位截取方法

计算出定时常数： TC = 7692 = 1E0CH = 0001 1110 0000 1100B，

取低13位，其中高8位TCH = F0H，低5位为TCL = 0CH

第二步：TMOD的设定（即控制字）



方式1

控制字为10H

例2．使用**T1的方式1**，设定1ms的定时。同样，在P1.0引脚上产生周期为2 ms的方波输出。晶体振荡器的频率为fosc = 6 MHz。

第一步：定时常数计算

定时常数:

fosc = 6MHz → Tc = 2μs，

方式1计数器长度L = 16 ($2^{16}=65\,536$)，

定时时间 t = 1ms = 0.001s

计算出定时常数：TC = 65536 – 1ms/2us = 65036 = FE0C H

= 1111 1110 0000 1100B，

其中高8位TCH = FEH，低8位为TCL = 0CH

```
        ORG   0000H
        AJMP  MAIN
        ORG   001BH              ；T1中断矢量
        AJMP  INQP
        ORG   0100H
MAIN:   MOV   TMOD，＃10H；写控制字，设T1为定时器方式1
        MOV   TH1，＃0FEH ；写定时常数(定时1ms)
        MOV   TL1，＃0CH
        SETB  TR1         ；启动 T1
        SETB  ET1         ；允许T1中断
        SETB  EA          ；开放CPU中断
        AJMP  $           ；定时中断等待
        ORG   XXXX H      ；T1中断服务程序
INQP:   MOV   TH1，＃0FEH ；重写定时常数
        MOV   TL1，＃0CH
        CPL   P1.0        ；P1.0变反输出
        RETI              ；中断返回
```

第七章 串行通信

7.1 概述

- 同步传输：数据块（连续字符）之间无空隙　一定要有 CLK 线
- 异步传输：以字符为单位传输，字符用起始位/停止位包装起来　起始 终 止位，要定时器

- 单工　只沿一个方向传输
- 双工　可以同时双向传输　UART
- 半双工　同一时刻只能一个方向传输　I^2C　1-wire

波特率：每秒传输离散信号事件的个数（单位 baud 波特）二进制下，波特率=比特率　bit/s

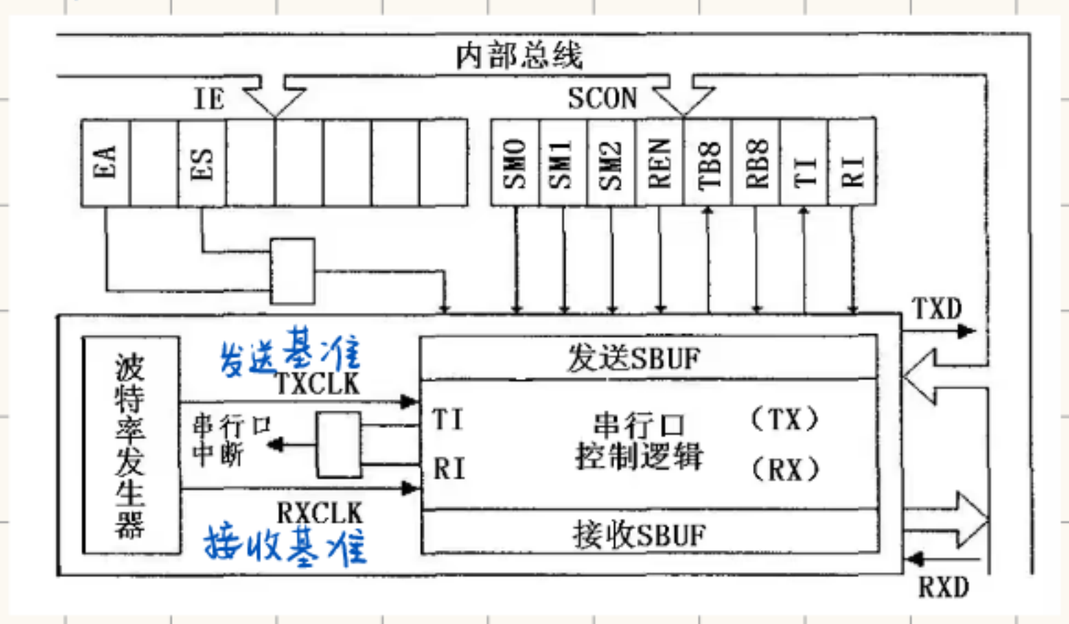
7.2 传输帧

起始位 D0 D1 D2 D3 D4 D5 D6 D7 停止位

低位 ⟶ 高位

MOV SBUF, A　发送数据

MOV A, SBUF　接收数据



SCON：串行口状态控制寄存器

位地址	9FH	9EH	9DH	9CH	9BH	9AH	99H	98H
符号	SM0	SM1	SM2	REN	TB8	RB8	TI	RI

工作方式↓选择　多机通信　允许串行接收　0：主机发数据　1：主机发地址　停止位（第9位）　发送中断标志　接收中断标志

方式2.3中 TB8. RB8为发送/接收的第9位数据

1）SM_0、SM_1——串行口工作方式选择位

SM_0	SM_1	工作方式	特点	波特率
0	0	工作方式0	8位移位寄存器	$f_{osc}/12$
0	1	工作方式1	10位异步接收发送	$\frac{2^{smod}}{32}\times\frac{f_{OSC}}{12\times(256-X)}$
1	0	工作方式2	11位异步接收发送	$f_{osc}/32$或$f_{osc}/64$
1	1	工作方式3	11位异步接收发送	$\frac{2^{smod}}{32}\times\frac{f_{OSC}}{12\times(256-X)}$

方式0：$\frac{f_{osc}}{12}\times\frac{1}{(2^{13}-TC+X)}$

方式1：$\frac{f_{osc}}{12}\times\frac{1}{(2^{16}-TC+X)}$

方式2：$\frac{f_{osc}}{12}\times\frac{1}{(2^{8}-TH1)}$

TC：计数初值　X：中断程序所需机器周期数

PCON：电源控制及波特率选择

位地址：	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
符号：	SMOD	/	/	/	GF_1	GF_0	PD	IDL

=1 波特率倍增

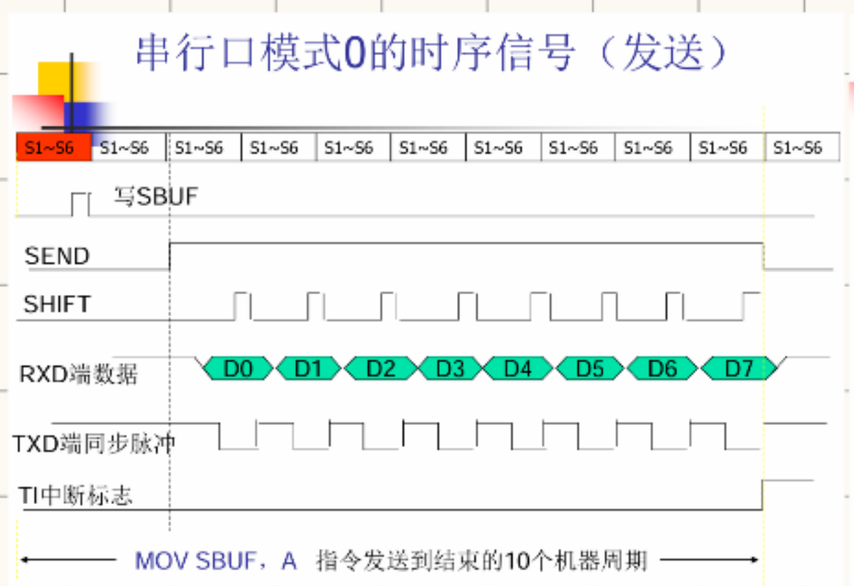
SBUF：串行数据寄存器

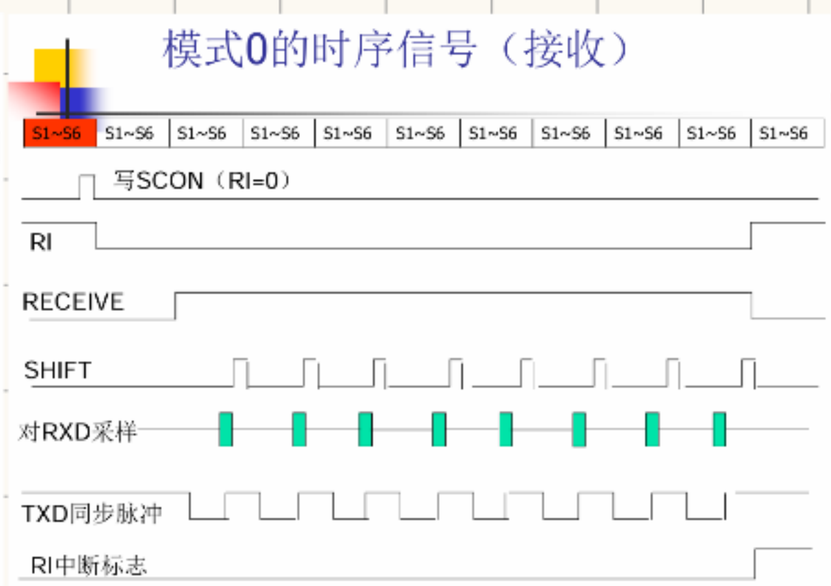
IE：中断允许寄存器　ES 串行中断允许位 0禁止. 1允许

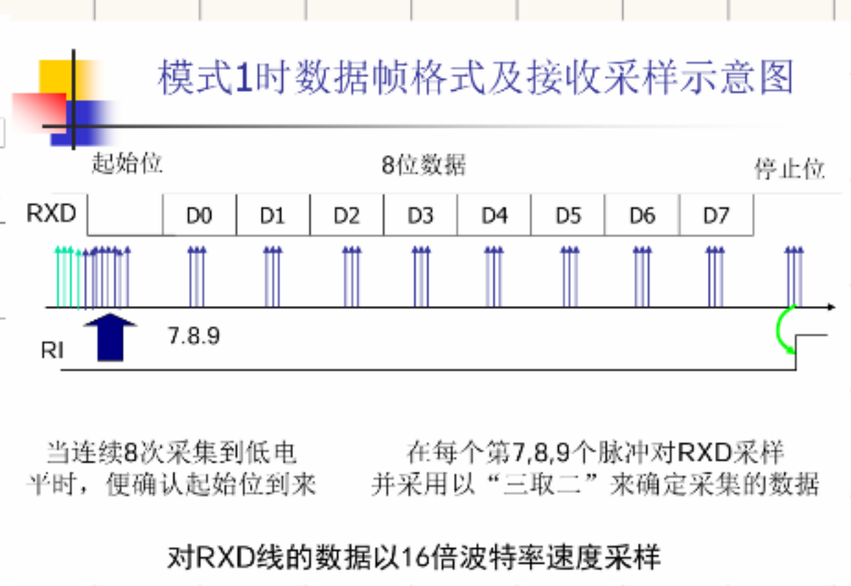
7.3 串行口工作方式

时序考吗？　波特率计算

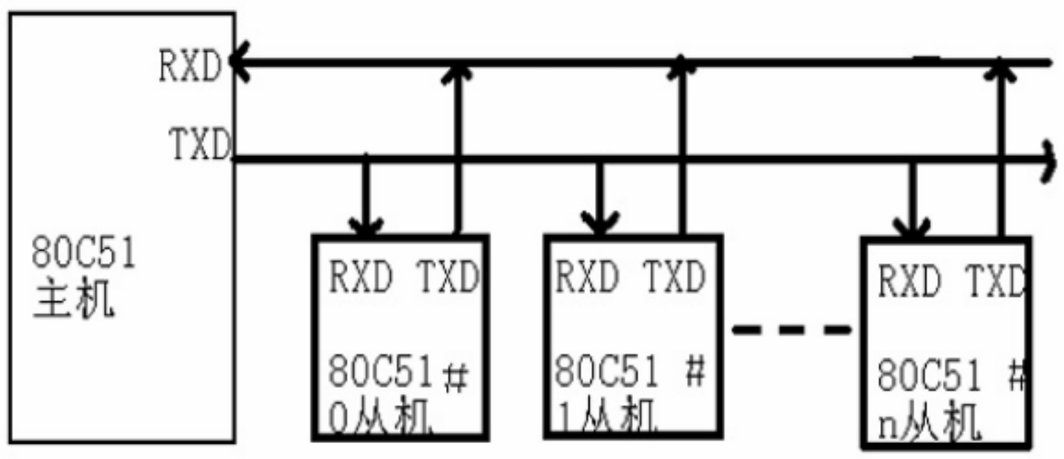
T1 工作于方式2，能自动重装，适合作波特率发生器　方式2 波特率 $=\frac{2^{smod}}{64}\times f_{osc}$

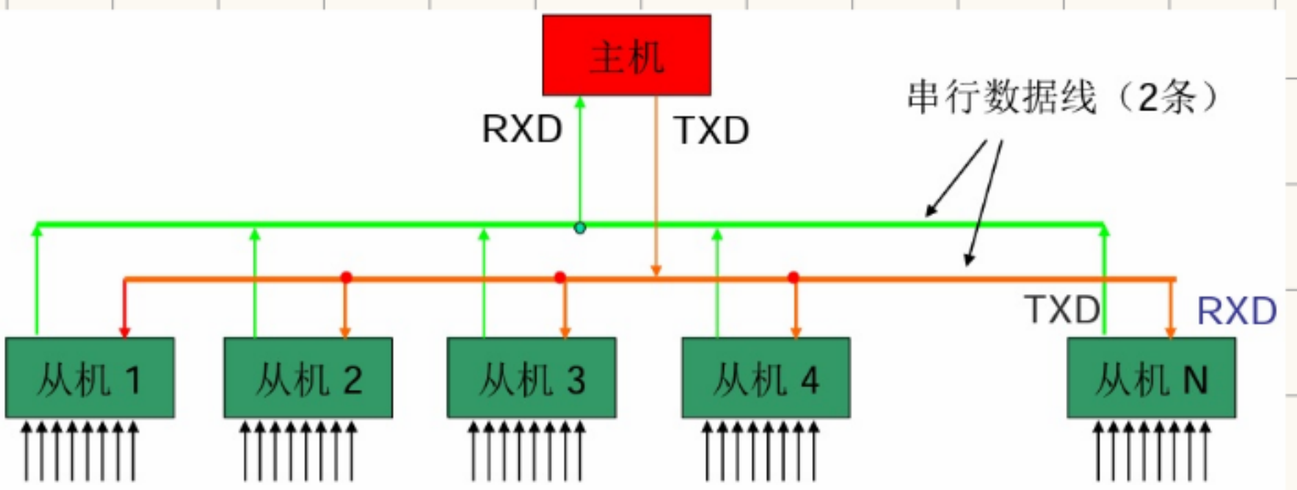






7.4 多机通信





1、每个从机都有各自的地址（一个字节）

2、每次通信都由主机发起，格式

地址	数据
1Byte	NByte

3. 从机处于接收状态 SM2=1 ☆

主机发送地址（TB8=1），所有从机都进入中断

4、地址相等那个从机 SM2=0，不等保持 SM2=1

注：TB8=0 发数据

7.5 串行口的应用

- 串行口的波特率发生器及波特率计算
  - 方式0：固定，fosc/12
  - 方式2：由SMOD位(PCON.7)决定

$$\text{波特率} = \frac{2^{smod}}{64} \times f_{osc}$$

  - 方式1 &3

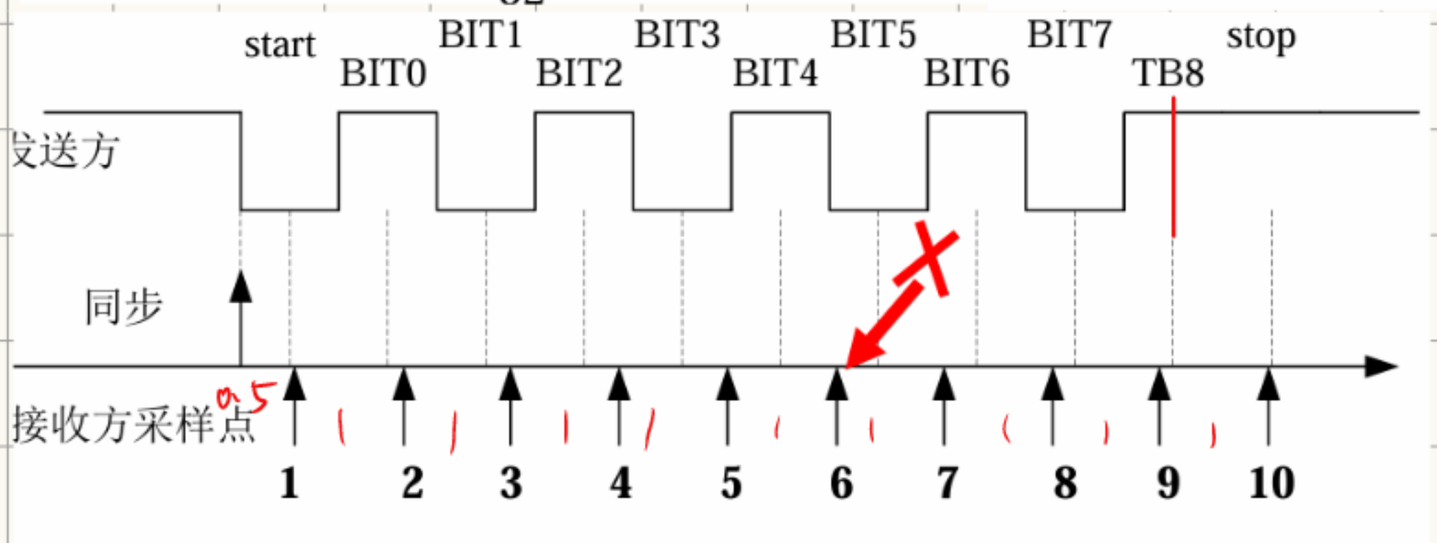
T1/T2的溢出率以及SMOD决定

$$\text{波特率} = \frac{2^{smod}}{32} \times (\text{定时器1溢出率})$$

定时器1模式2

$$\text{溢出率} = \frac{f_{osc}}{12} \times \frac{1}{(2^8 - TH1)}$$

$$\text{波特率} = \frac{2^{smod}}{32} \times (\text{定时器溢出率})$$



位周期：T　　发送与接收波特率误差：a%

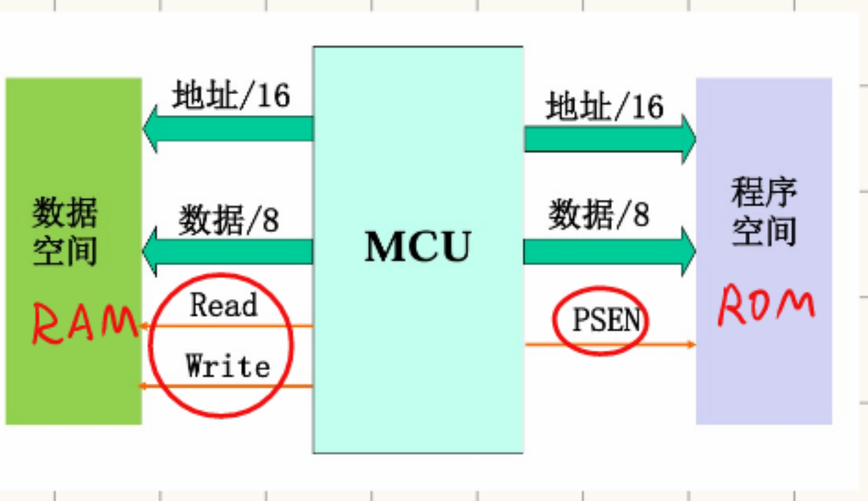
第10位累积误差：$\Delta T_{10} = 9.5 \times a\% \times T$

$\Delta T_{10} < T/2$ → $a < 5\%$

第八章 单片微机的系统扩展原理与接口技术

8.1 系统拓展原理

8.1.1 外部并行拓展原理

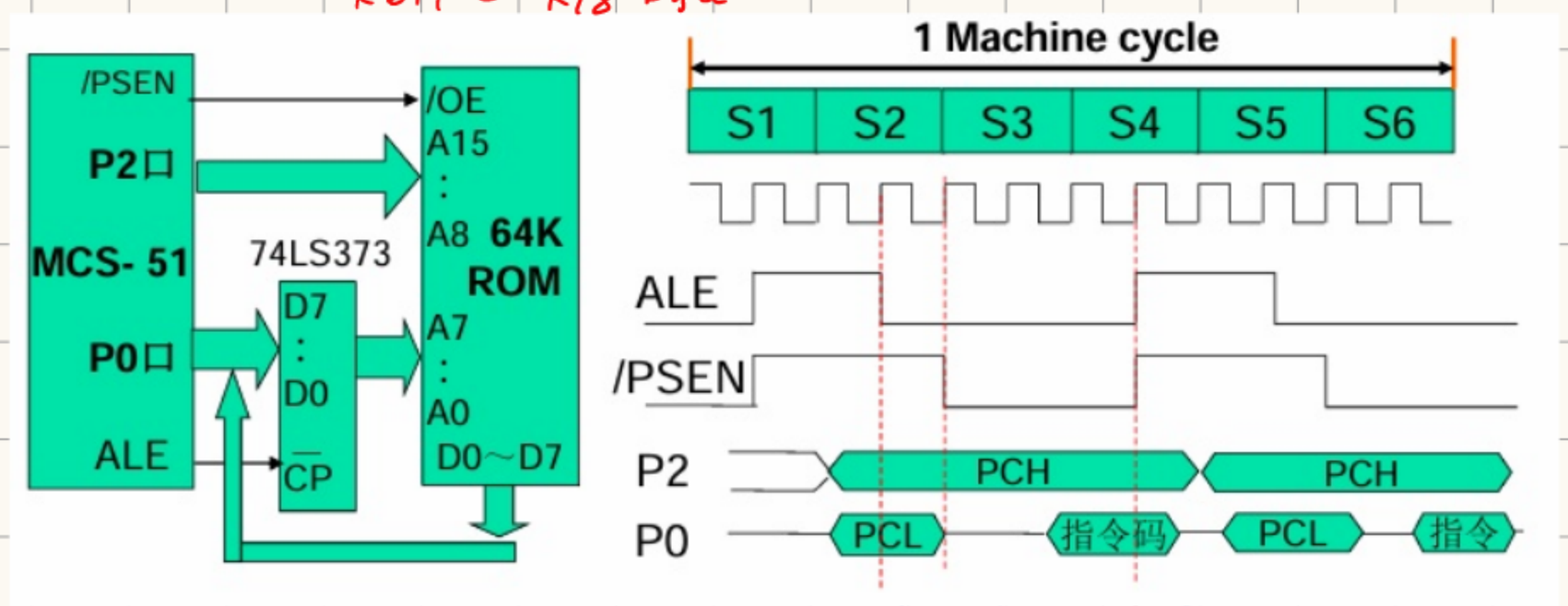


- 地址总线(AB)　P2：高8位　P0：低8位（与DB复用）
- 数据总线(DB)　P0
- 控制总线(CB)
  - ALE　下降沿 锁存地址数据
  - $\overline{PSEN}$　输出，读外存储器的数据
  - $\overline{EN}$　0，只访问外部存储器；1 先访问片内，再访问片外
  - $\overline{RD}$　读控制
  - $\overline{WR}$　写控制

命名规则　27C　64　ROM

27C：类型　64：容量　64 kbit = 8kB

kbit = k/8 byte

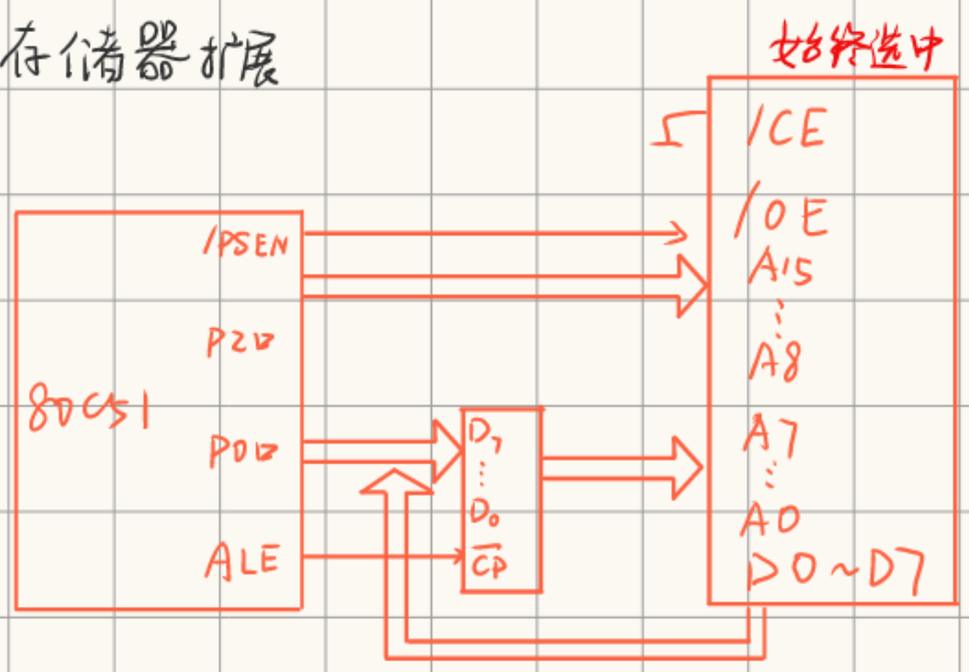


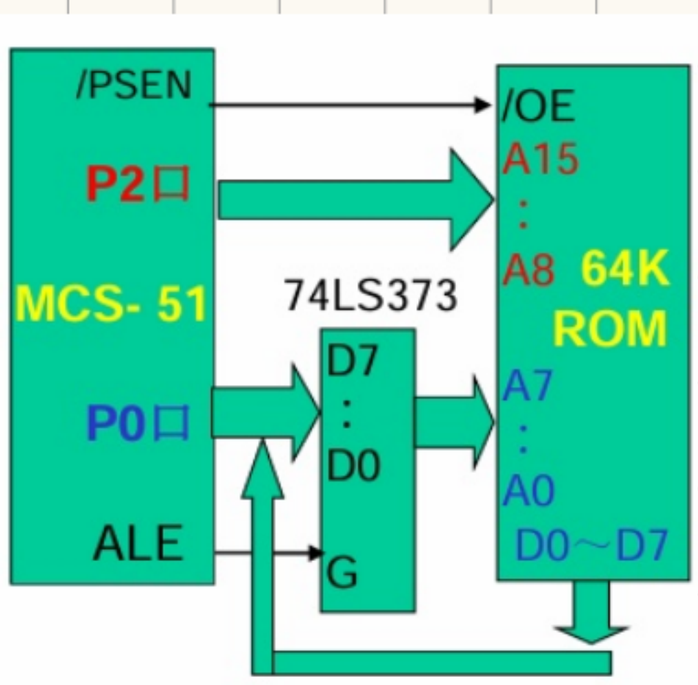
每个机器周期包括6状态（S1-S6），每状态包括2节拍

△CPU读入指令（$\overline{PSEN}$上升沿）

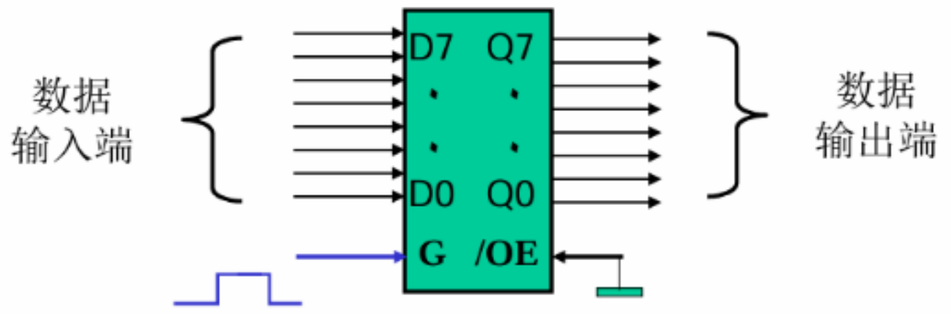
8.1.2 程序存储器扩展

始终选中





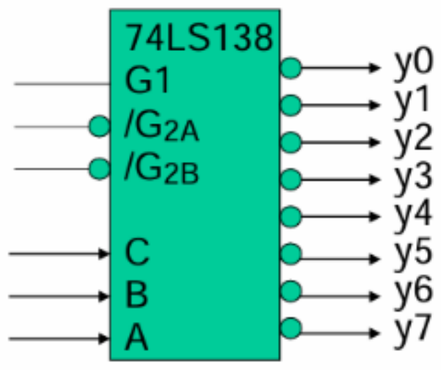
（2） 相关的芯片
（锁存器：74LS373，74LS573）



- /OE：三态控制。低电平：允许数据输出，三态门通；
  高电平：三态门关闭，输出高阻。
- G（LE）：数据锁存控制。
  高电平时，输出随输入变化；（变化）
  低电平时，Q端将被锁存（低电平锁存）。（不变化）

下降沿锁存

相关的芯片
（译码器：74L138）



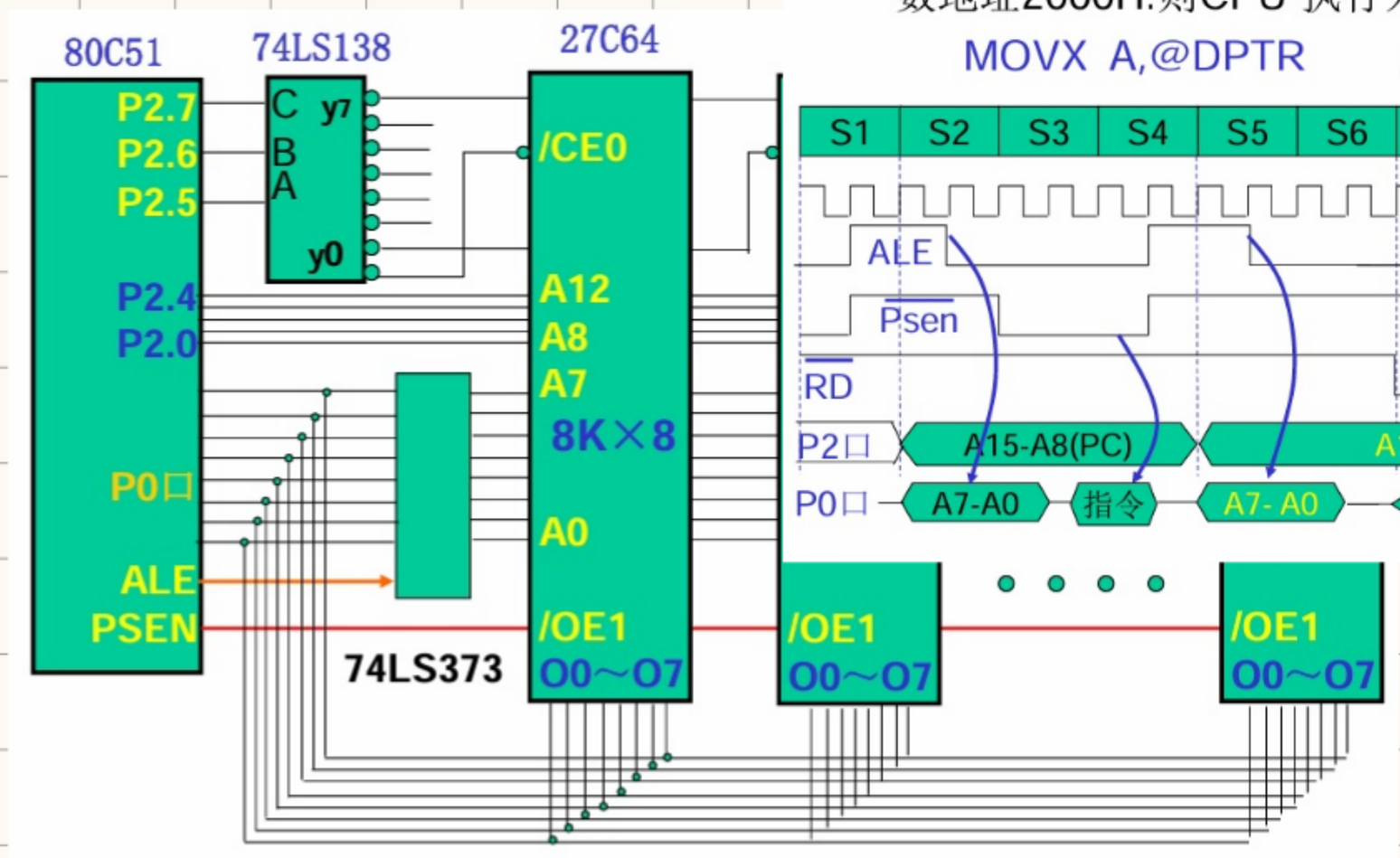
C B A	y7	y6	y5	y4	y3	y2	y1	y0
0 0 0	1	1	1	1	1	1	1	0
0 0 1	1	1	1	1	1	1	0	1
0 1 0	1	1	1	1	1	0	1	1
0 1 1	1	1	1	1	0	1	1	1
1 0 0	1	1	1	0	1	1	1	1
1 0 1	1	1	0	1	1	1	1	1
1 1 0	1	0	1	1	1	1	1	1
1 1 1	0	1	1	1	1	1	1	1

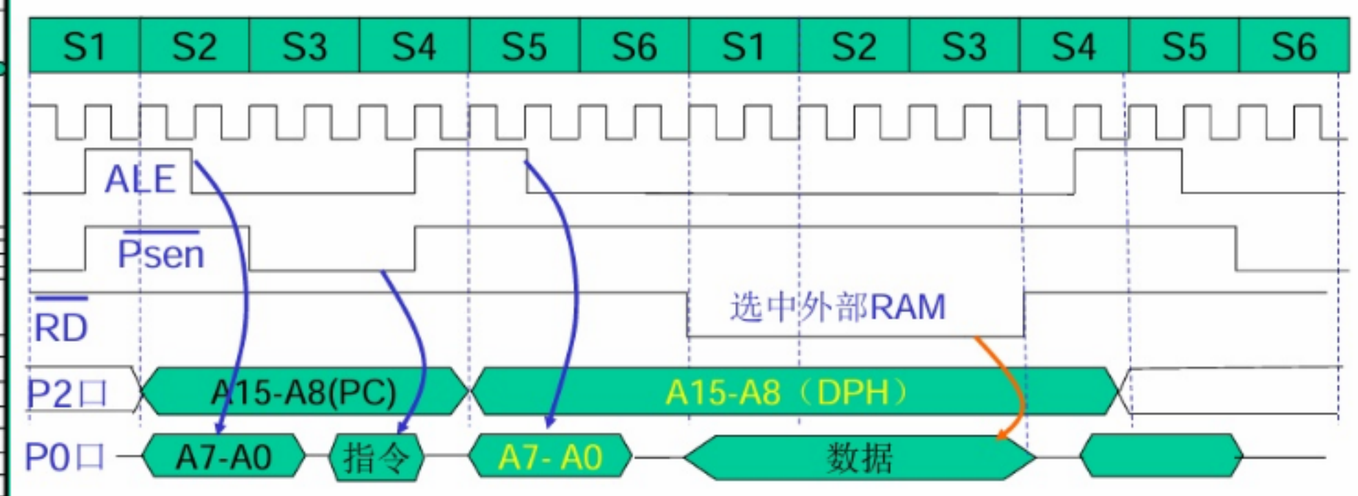
- A、B、C：选择输入。作为与输出对应的二进制编码输入；
- G_1、$/G_{2A}$、G_{2B}：使能输入，作为片选控制输入；
- y7～y0：译码器输出，低电平有效。（选中为低电平）

多个ROM

设外部RAM2000H单元中有一个数x,且DPTR中已存有该数地址2000H.则CPU 执行外部ROM中的指令:

MOVX A,@DPTR ;将外RAM的x送A



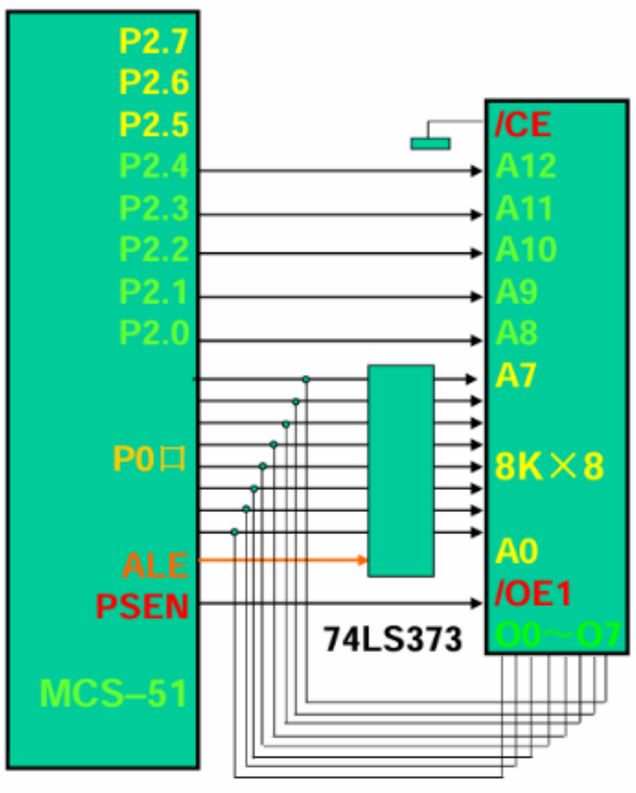


误连

P2.7～P2.5 138输入	片选/CEi 138 输出	选中 ROM	P2.4～P0.0	有效地址范围
0 0 0	Y0=0	第1片	0000H～1FFFH	0000H～1FFFH
0 0 1	Y1=0	第2片	0000H～1FFFH	2000H～3FFFH
0 1 0	Y2=0	第3片	0000H～1FFFH	4000H～5FFFH
0 1 1	Y3=0	第4片	0000H～1FFFH	6000H～7FFFH
1 0 0	Y4=0	第5片	0000H～1FFFH	8000H～9FFFH
1 0 1	Y5=0	第6片	0000H～1FFFH	A000H～BFFFH
1 1 0	Y6=0	第7片	0000H～1FFFH	C000H～DFFFH
1 1 1	Y7=0	第8片	0000H～1FFFH	E000H～FFFFH

P2.7～P0.0

16位地址 ☆

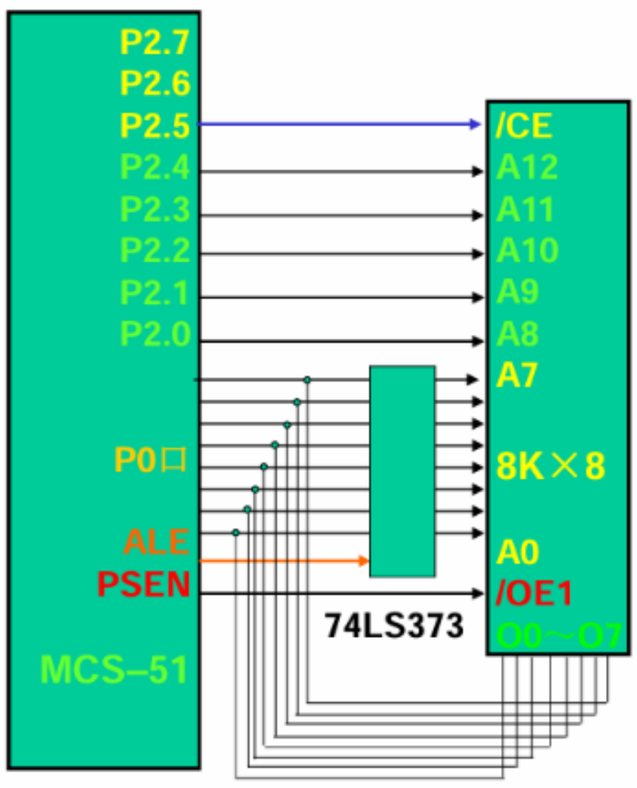


P2.7～P2.5	P2.4～P2.0	P0.7～P0.0	有效地址范围
0 0 0	00000/11111	00H/FFH	0000H～1FFFH
0 0 1	00000/11111	00H/FFH	2000H～3FFFH
0 1 0	00000/11111	00H/FFH	4000H～5FFFH
0 1 1	00000/11111	00H/FFH	6000H～7FFFH
1 0 0	00000/11111	00H/FFH	8000H～9FFFH
1 0 1	00000/11111	00H/FFH	A000H～BFFFH
1 1 0	00000/11111	00H/FFH	C000H～DFFFH
1 1 1	00000/11111	00H/FFH	E000H～FFFFH

地址空间重叠

在这里读出内容会一样

线选法



造成了相互重叠的8个地址存储区域

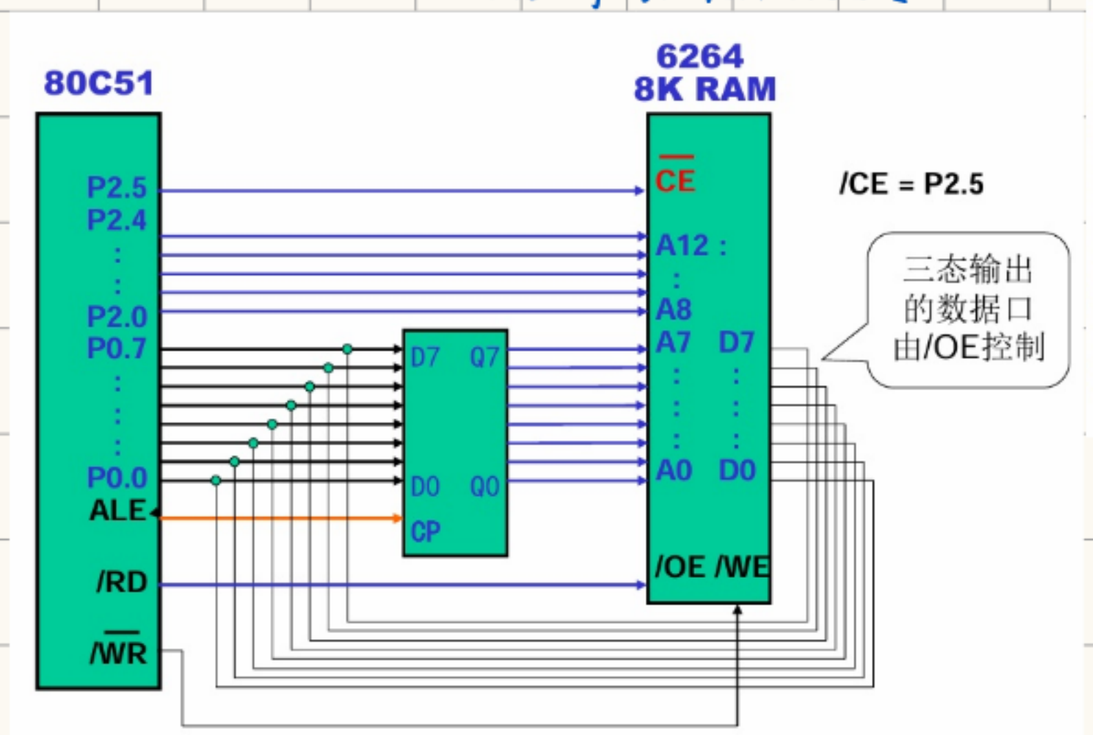
P2.7	P2.6	P2.5	P2.4 ～ P2.0	P0.7 ～ P0.0	有效地址范围
0	0	0	00000/11111	00H/FFH	0000H～1FFFH
0	0	1	00000/11111	00H/FFH	2000H～3FFFH
0	1	0	00000/11111	00H/FFH	4000H～5FFFH
0	1	1	00000/11111	00H/FFH	6000H～7FFFH
1	0	0	00000/11111	00H/FFH	8000H～9FFFH
1	0	1	00000/11111	00H/FFH	A000H～BFFFH
1	1	0	00000/11111	00H/FFH	C000H～DFFFH
1	1	1	00000/11111	00H/FFH	E000H～FFFFH

粉色为无用地址（/CE=1），红色为重叠存储区域

8.2.3 RAM 拓展

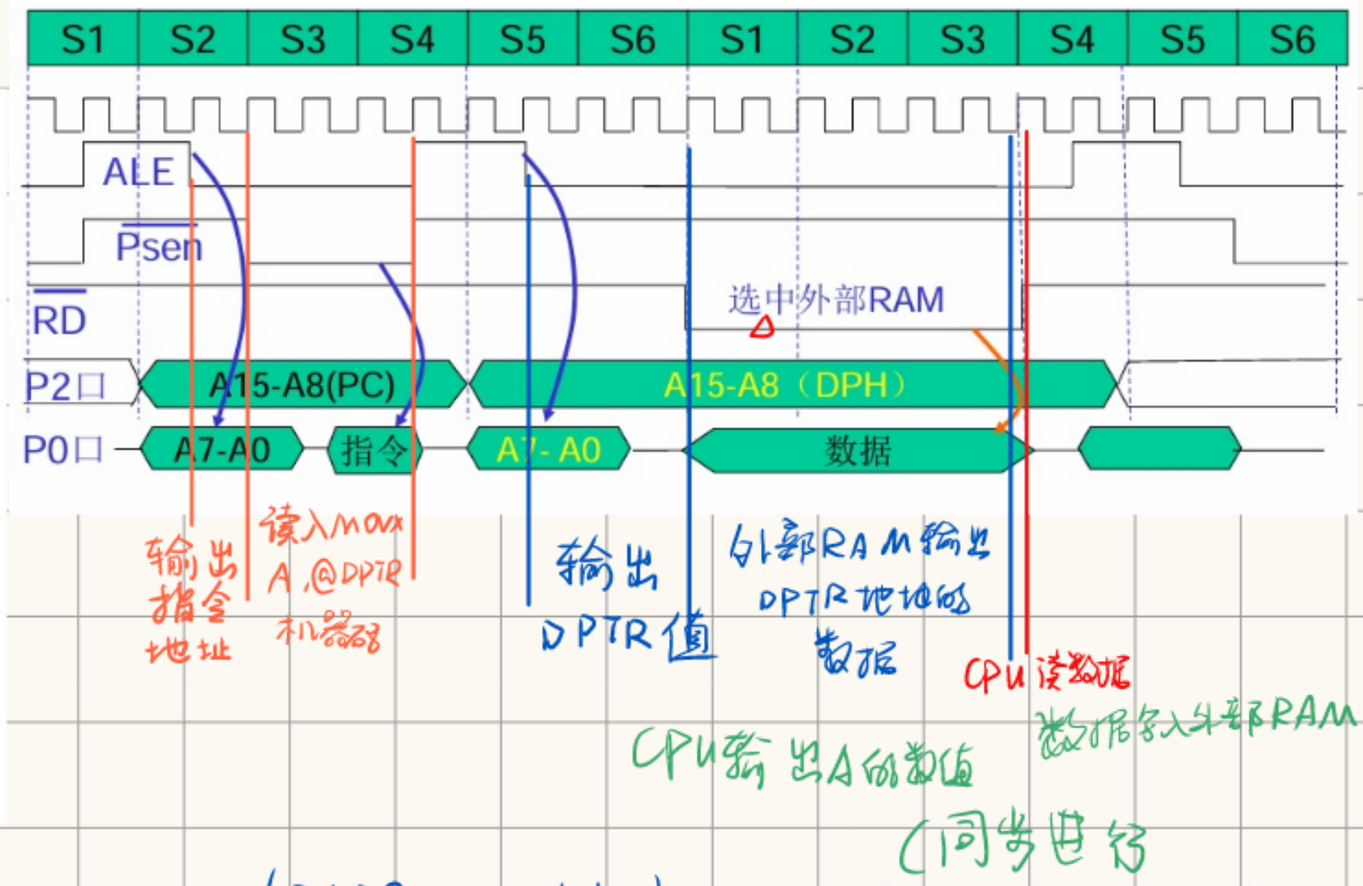
$\overline{RD}$ $\overline{WD}$ —— 读写控制信号

MOVX 访问 外 RAM指令



设外部RAM2000H单元中有一个数x,且DPTR中已存有该数地址2000H.则CPU 执行外部ROM中的指令:

MOVX A,@DPTR ;将外RAM的x送A

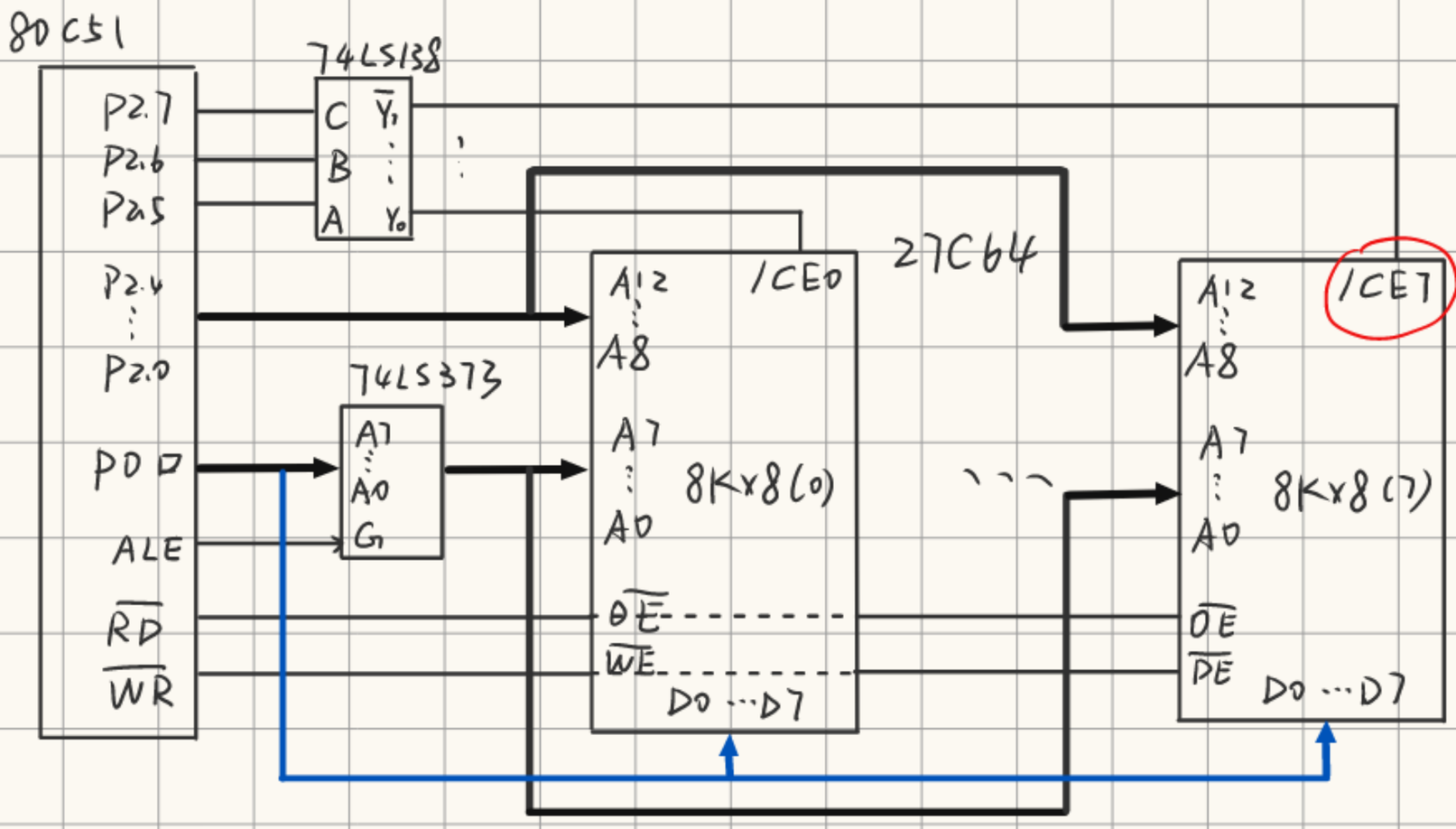


命名规则 62 64 8K Byte 62128 16K byte 62256 32K byte

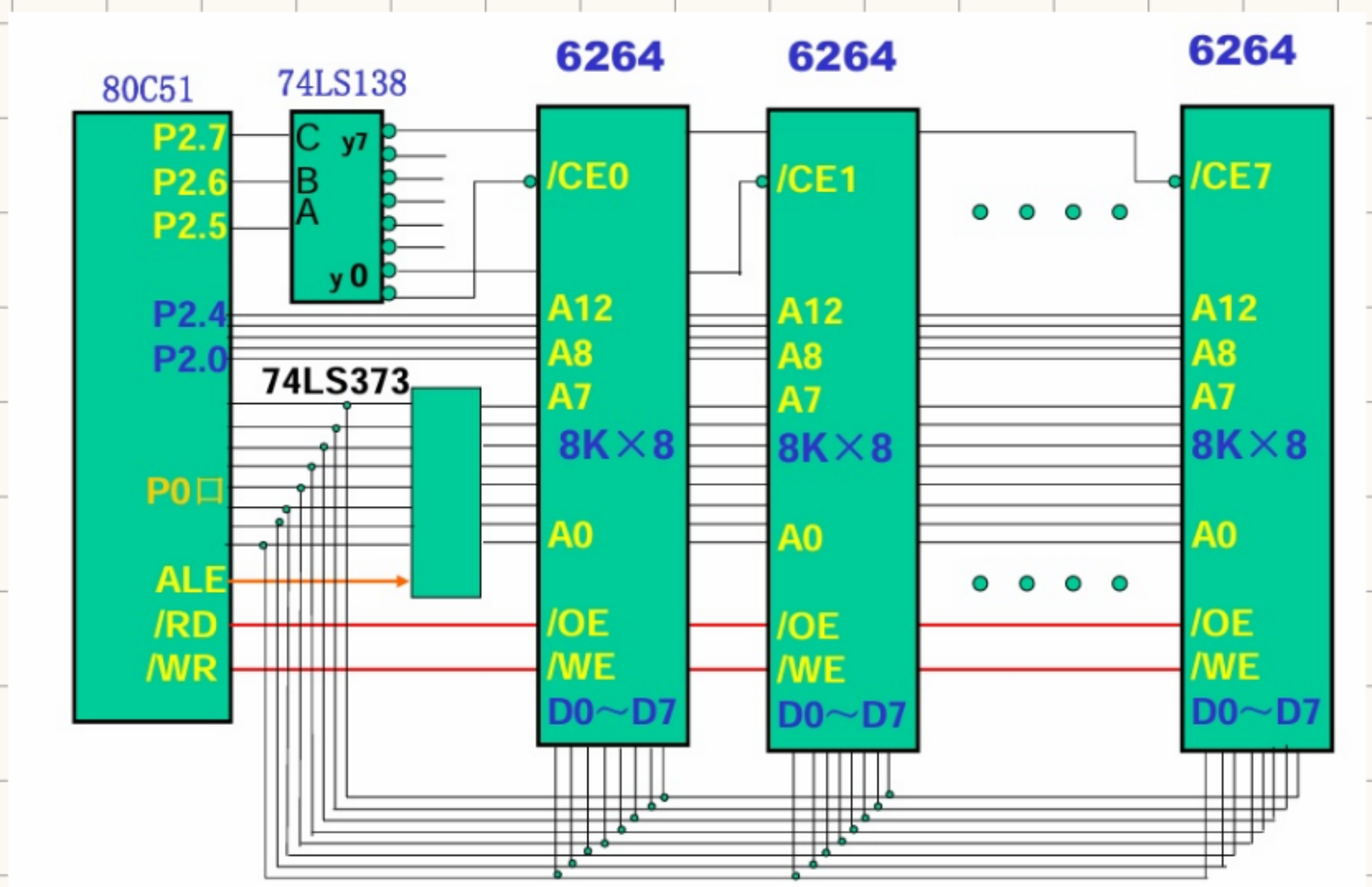
固定 位 (同步世行)

并行扩展

(1) 线选法 (只能接3个 27C64) (2)译码法



因为80C51外部地址总线16位 A15－A0，因此最大扩展2^{16}为64KB



总结：(1)程序、数据通过P2.P0扩展

(2)程序扩展用 PSEN引脚

(3)数据扩展用 $\overline{WR}$、$\overline{RD}$ 引脚

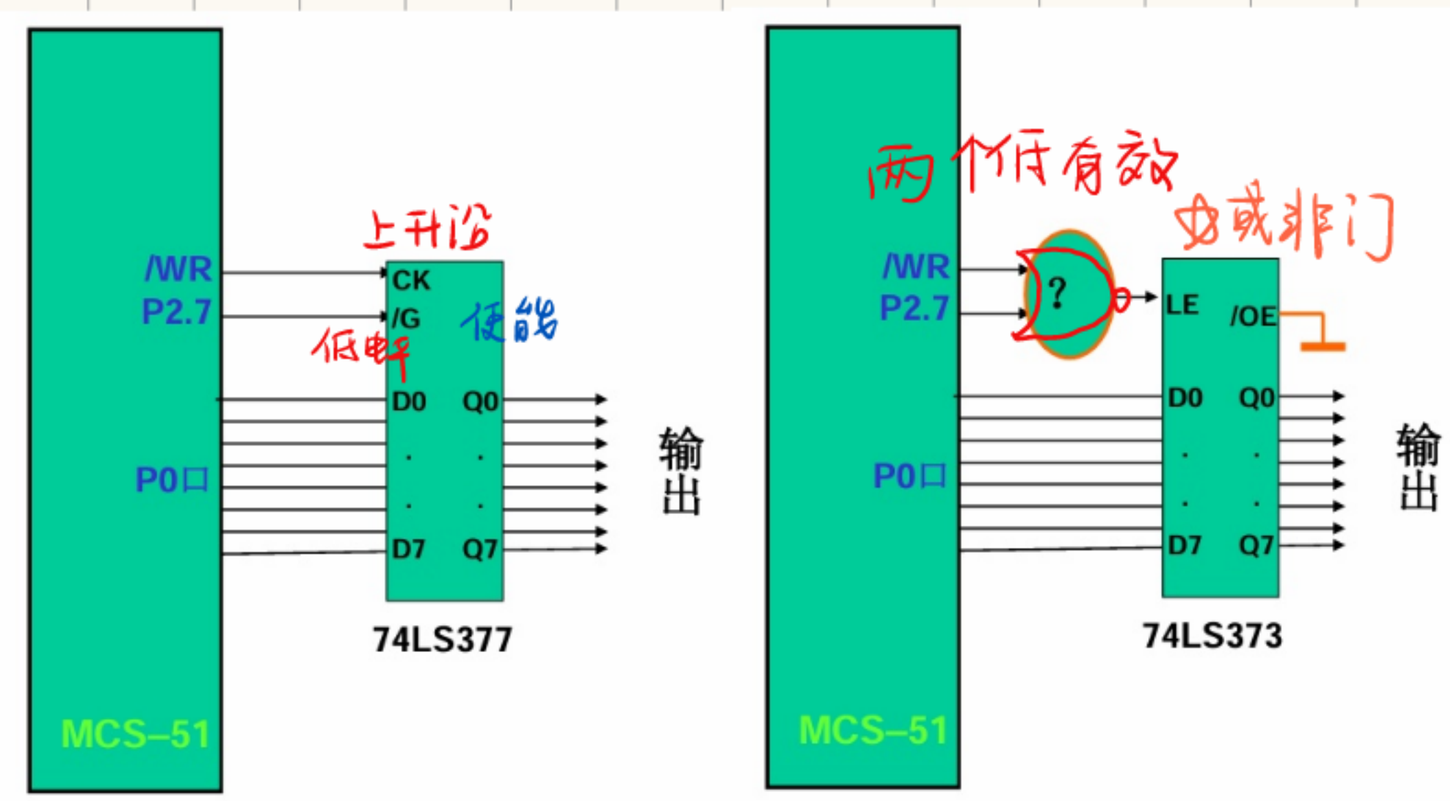
(4)程序和数据空间完全独立

8.2.4 外部 I/O 口扩展

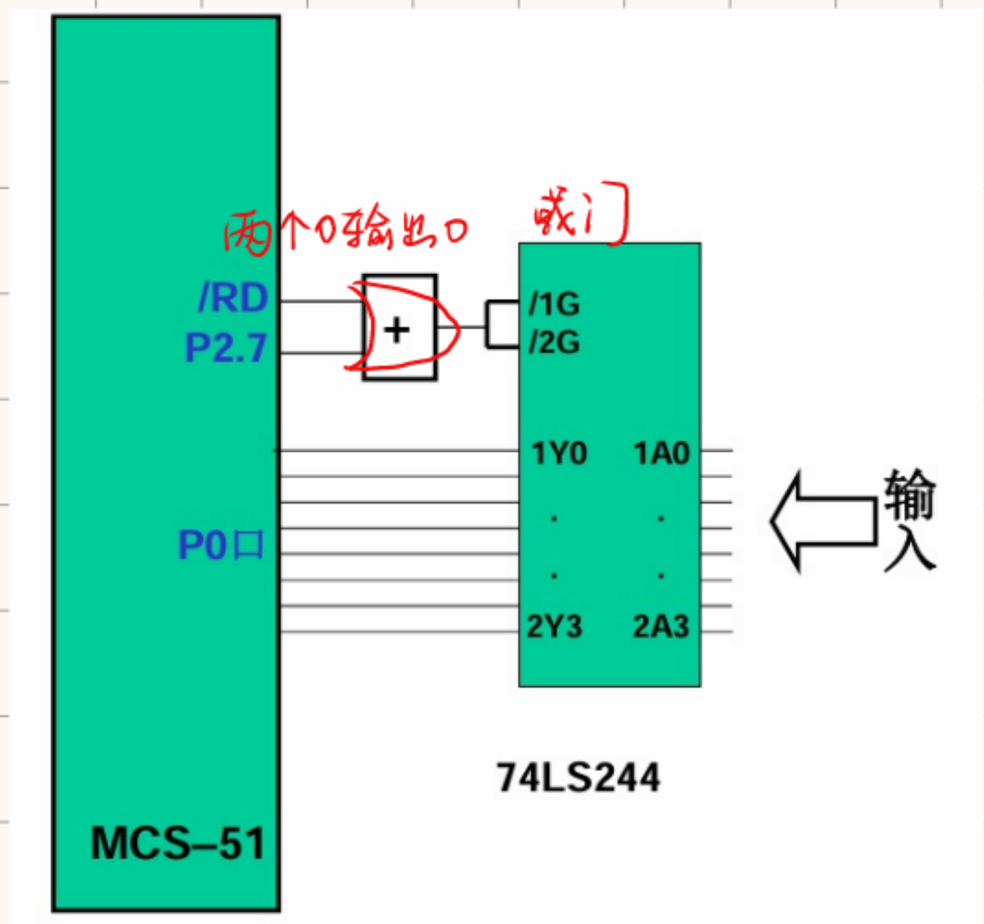
扩展 I/O 用 MOVX 指令

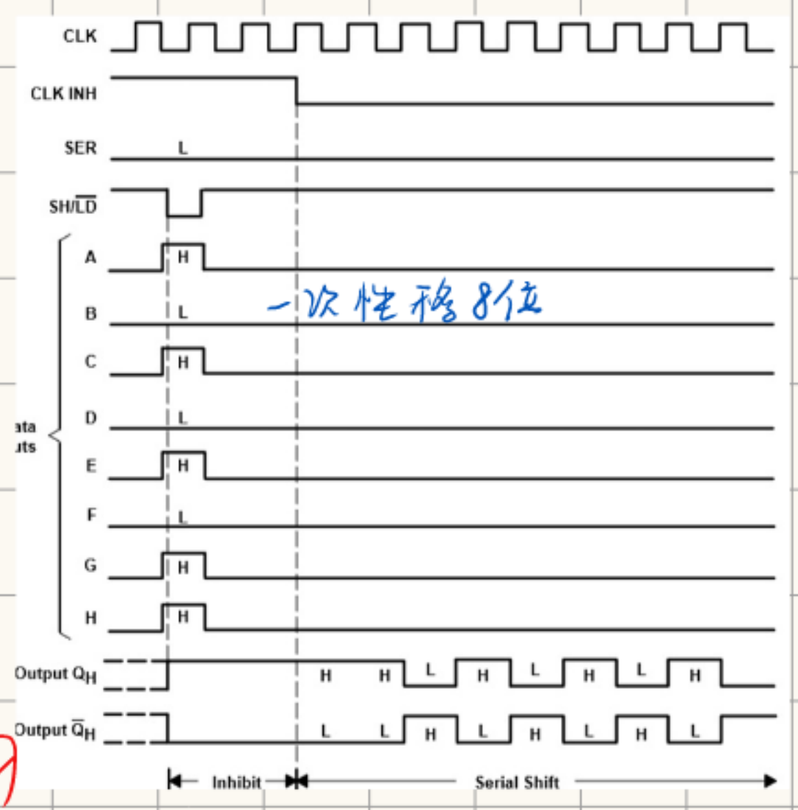
简单输出口的扩展

```
MOV  DPTR，#7FFFH  ；地址指针指向74LS377
MOV  A，#DATA      ；将输出数据送A
MOVX  @DPTR，A     ；输出数据
或
CLR  P2.7
MOV  R0，#0FFH  ；地址指针指向74LS377
MOV  A，#DATA      ；将输出数据送A
MOVX  @R0，A       ；输出数据
```



拓展输入口



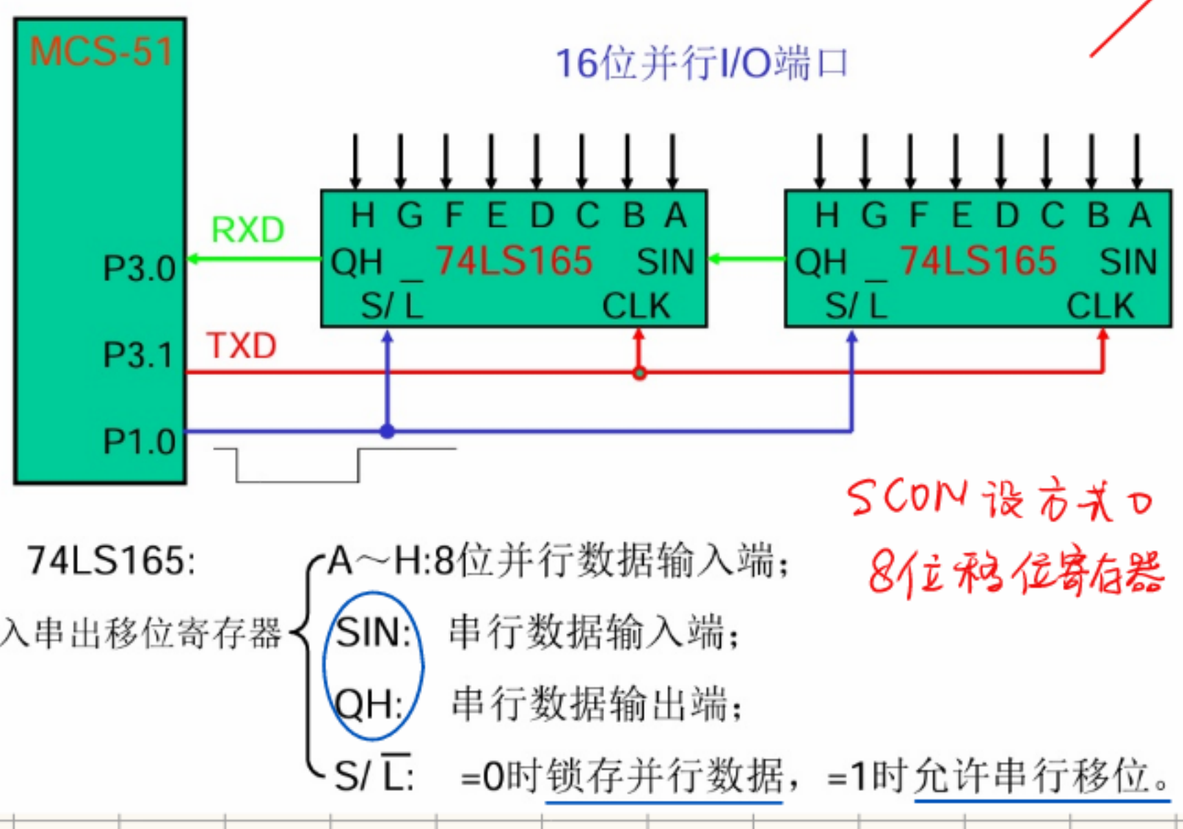


8.3 串行拓展 (占用 I/O 口 较少)

UART一般用于较长距离MCU与MCU间通信，速率较低； UART 串行通信接口

串行接口一般用于一块线路板内部MCU与外围芯片间的通信，速率较高； TX发 RX收

UART通信只定义了单个字符的数据格式；串行接口则定义了完整的地址读写格式。

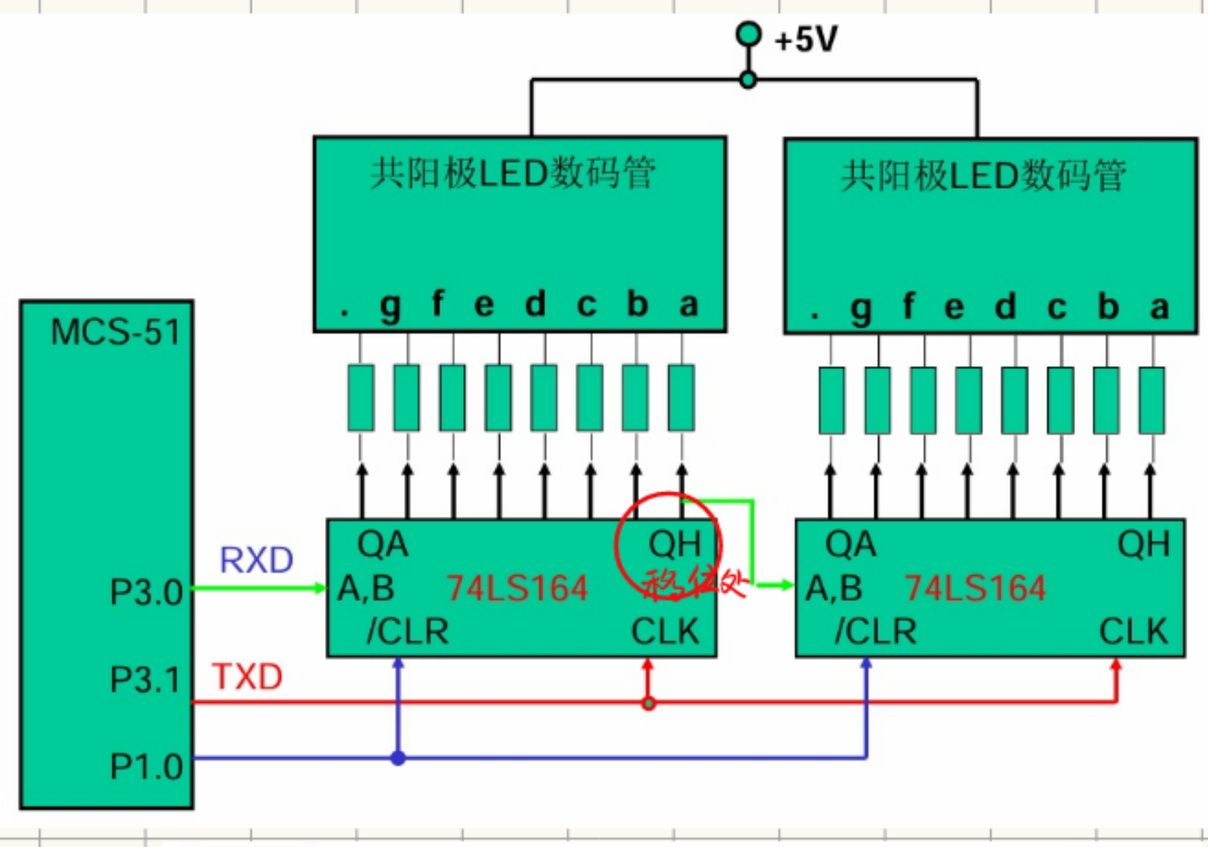


74LS165:

并入串出移位寄存器
- A～H:8位并行数据输入端；
- SIN: 串行数据输入端；
- QH: 串行数据输出端；
- S/$\overline{L}$: =0时锁存并行数据，=1时允许串行移位。

SCON 设方式0

8位移位寄存器



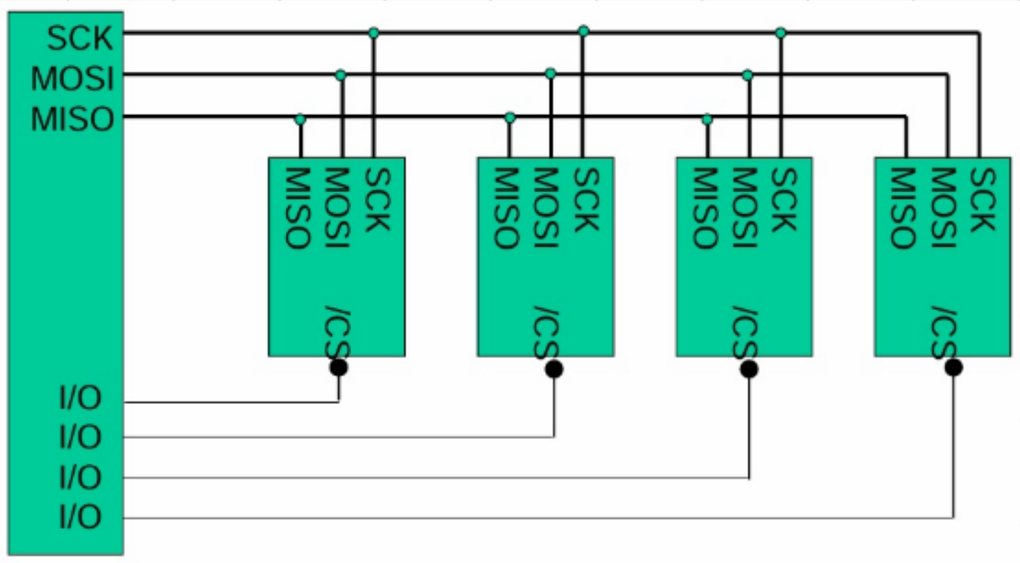
8.3.2 SPI 三总线结构

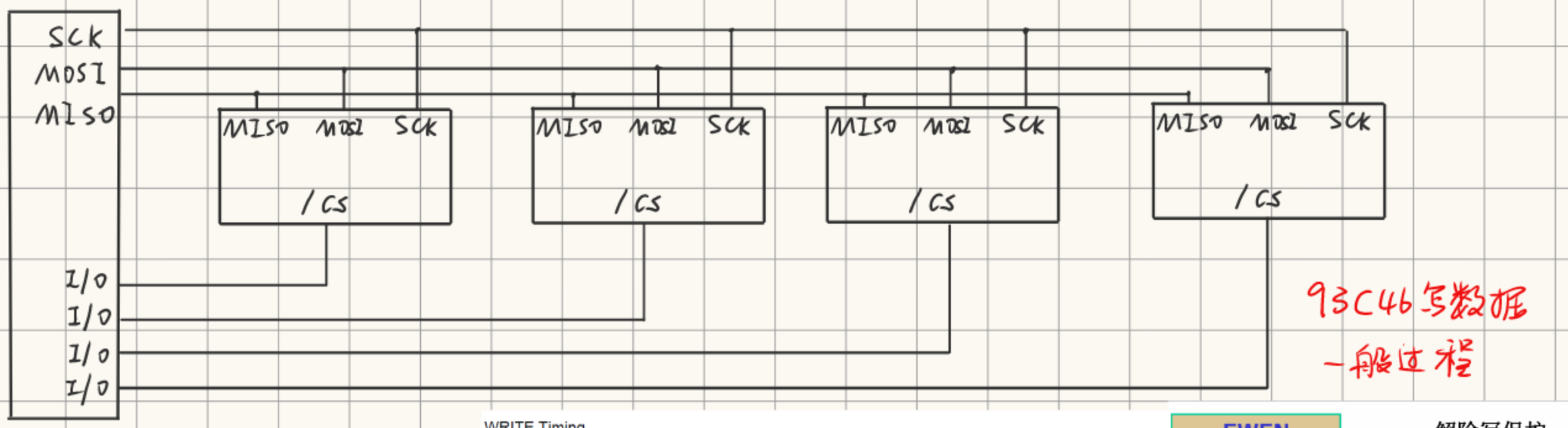
功能：同时拓展输入输出

① SCK：同步位移脉冲

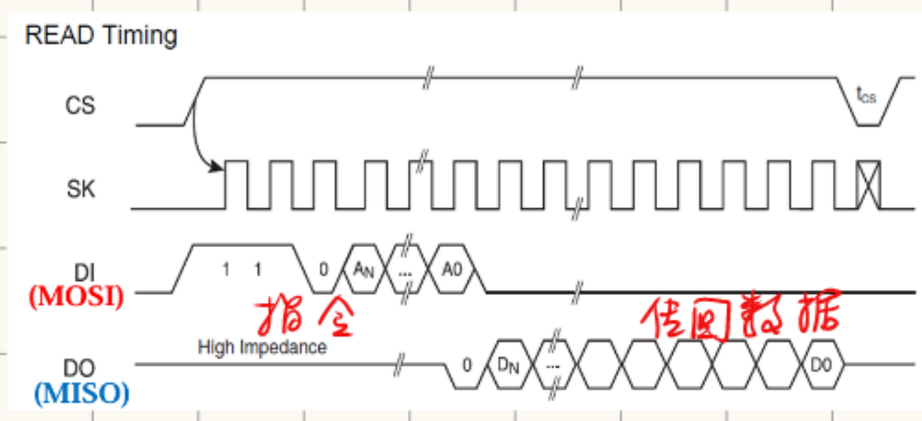
② MOSI：主机输出，从机输入 0 → I

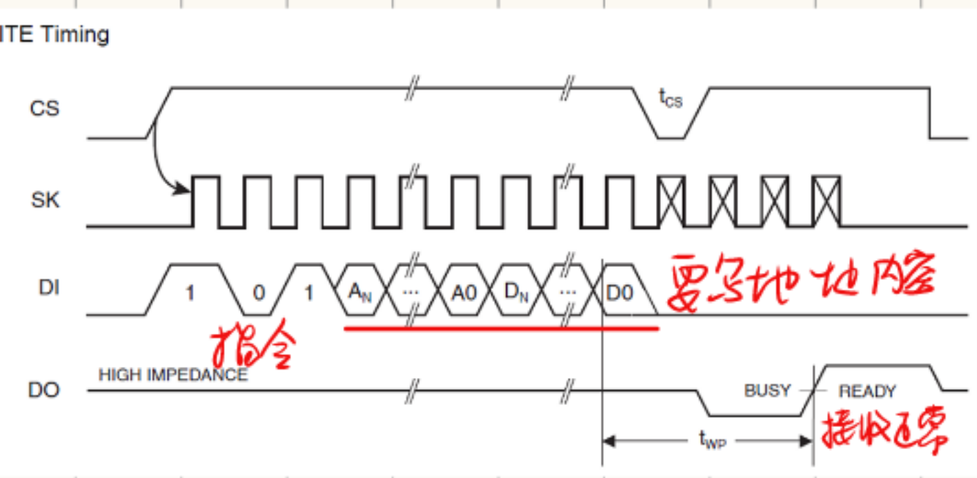
③ MISO：主机输入，从机输出 I ← 0

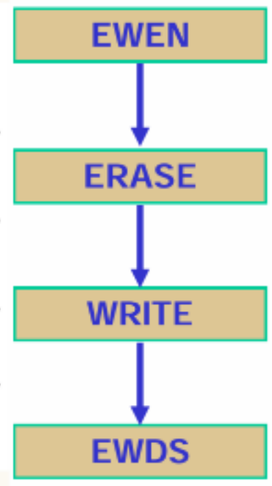




93C46写数据一般过程







解除写保护

擦除原来的数据

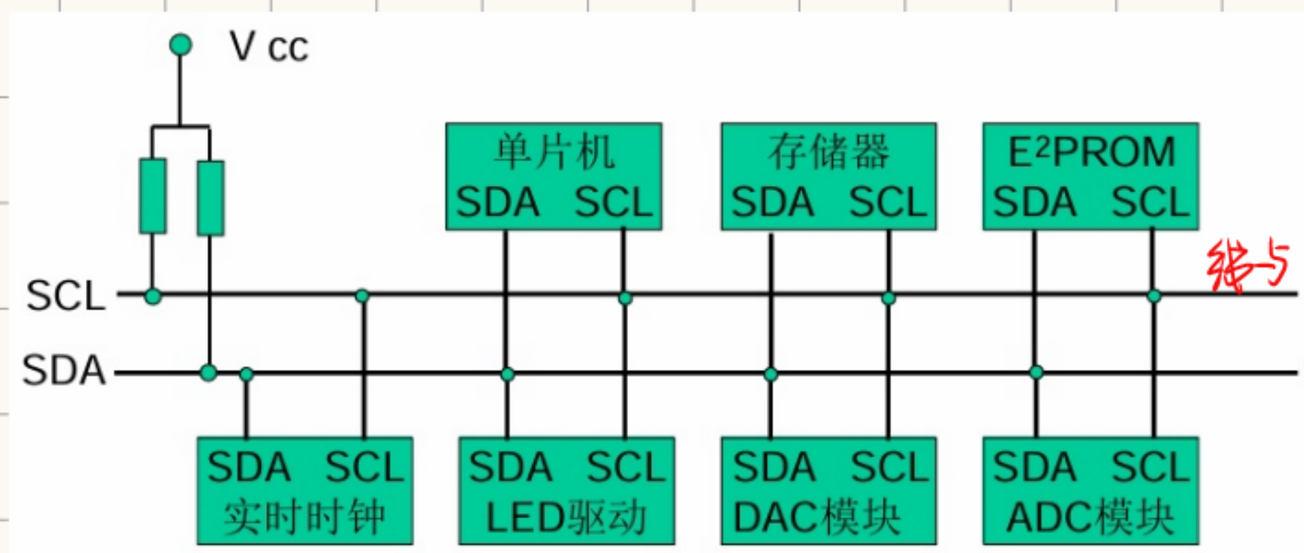
写入新数据

开启写保护

8.3.3 I^2C 总线 二线制 (同步,有clock) 基本通用

① SCL 时钟线 低电平输出至SDA，高电平读取SDA数据 同步

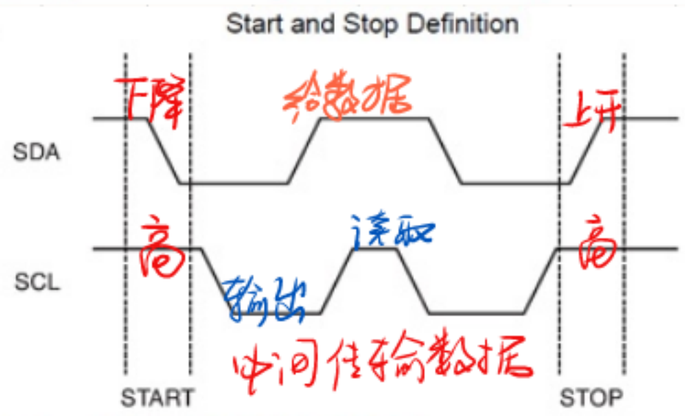
② SDA 数据线



I^2C总线工作原理

- I^2C总线采用两线制：数据线SDA、时钟线SCL。
- I^2C总线为同步总线，数据的传送与时钟信号完全同步。
- I^2C总线采用主从方式，由主控器寻址从器件，启动总线，产生时钟脉冲。
- 所有与SDA、SCL总线相连的器件（单片机和外围器件）都应当具备I^2C总线接口。
- I^2C总线接口器件在出厂时按照芯片的型号都具有唯一的8位地址（4位内部硬件地址、3位引脚定义的外部地址和1位方向地址）。

I^2C总线的Start和Stop时序



推论：中间区域

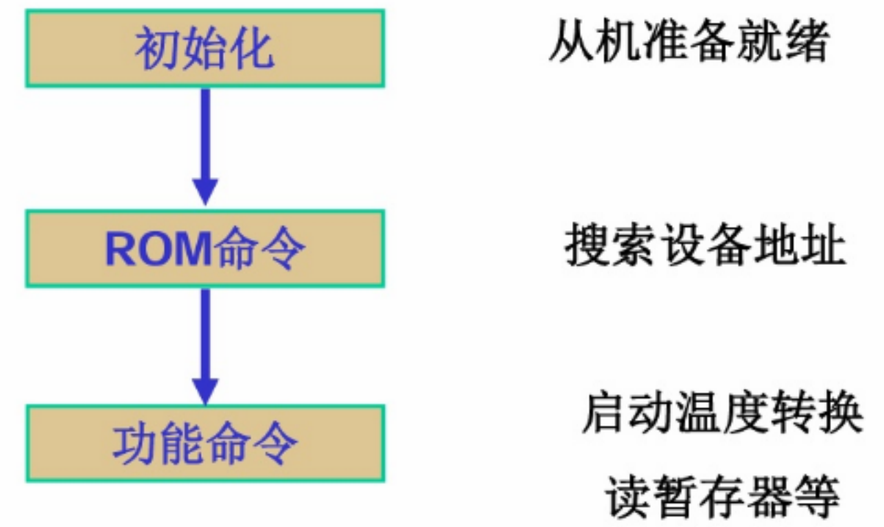
在传输数据时，当SCL高电平，SDA不能变化。

因SCL由主机控制，所以：

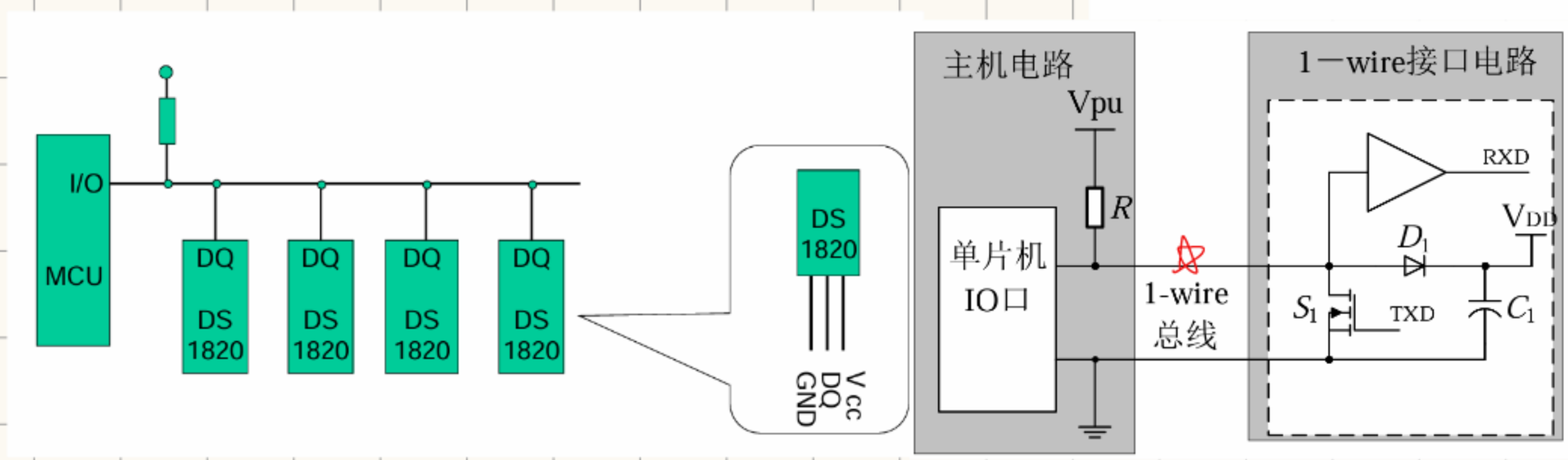
主机在SCL低电平时输出数据至SDA；

在SCL高电平时读取SDA数据。

SCL低电平 SDA数据才可能改变



8.3.4 单总线 (1-wire) 接口扩展系统



① 主机往从机发1

② 从机往主机发数据

③ 复位

④ 设备存在信号

通过时序区分

第二篇 ARM Cortex-M 部分

第九章 ARM 基础

1、 地址线32根，32位寻址 地址空间4GB 32位 RISC处理器

可以处理：
- 字节 8 bits
- 半字 16 bits
- 字 32 bits
- 双字 64 bits

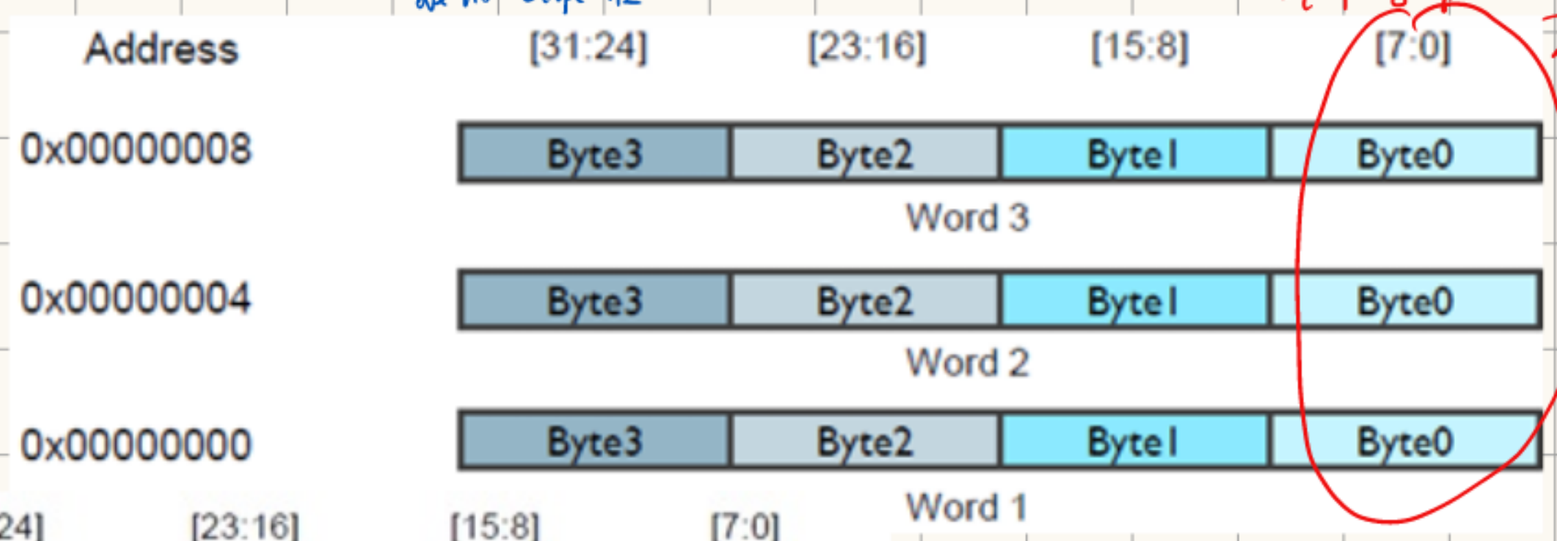
2^{10} K 1024　2^{20} M 1024K　2^{30} G 1024M

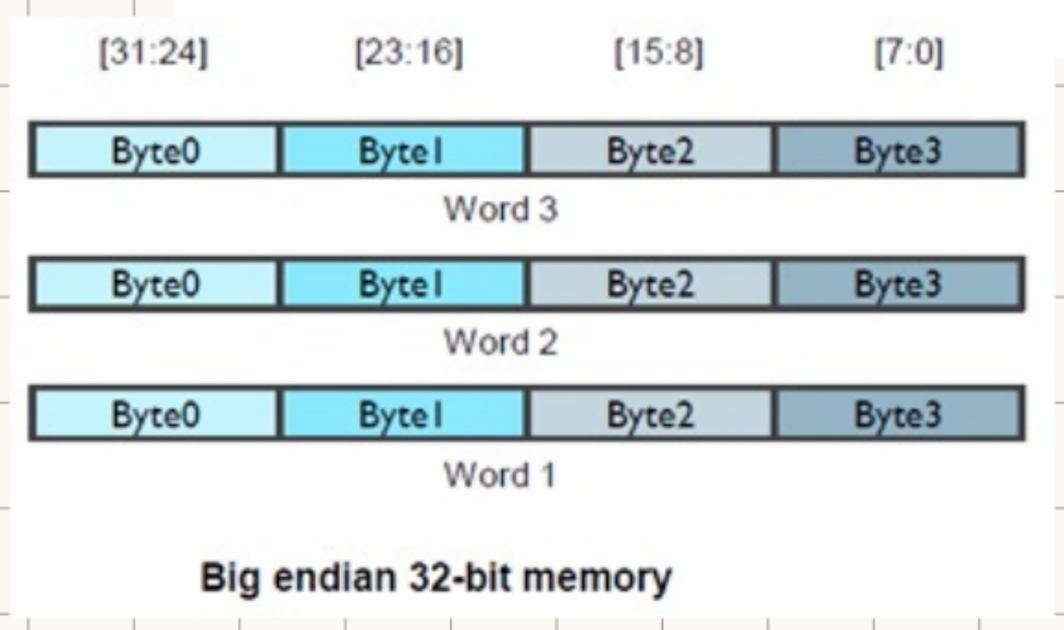
80C51 一个字 16 位，无半字，字节 8 位

2、 Cortex-M 数据存储采用 小端模式　51大多采用小端模式　DW采用 大端模式

最低 Byte 在 bit0 ~ bit7

4个字节　最低位





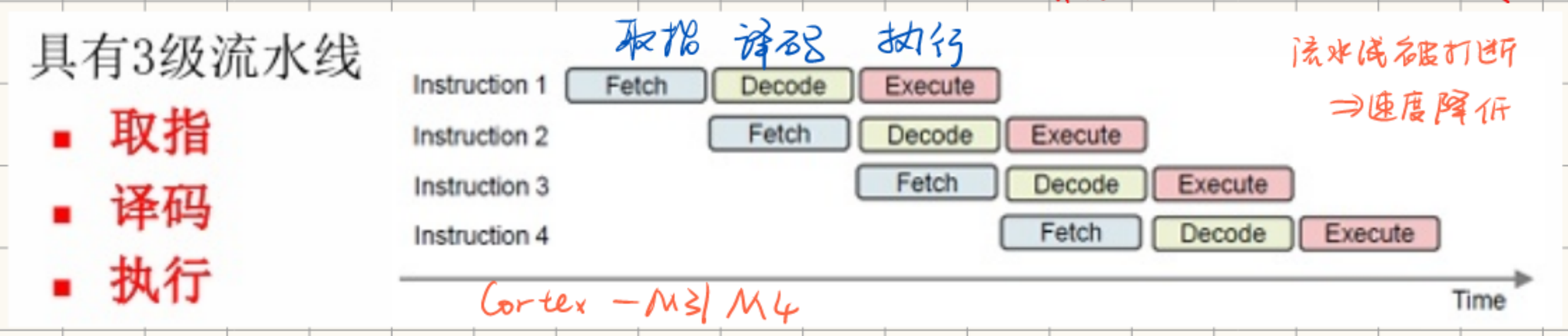
对齐传输：字变量的地址必须4的整数倍

半字变量地址必须是2的整数倍

3、 指令集 Thumb2：同时支持ARM指令(32位)和Thumb(16位)

☆ thumb是16位，thumb2 16位、32位混用

兼顾了指令密度和指令速度



- 某些指令执行多个周期，此情况下流水线暂停；
- 跳转到分支后流水线清空；
- 可一次读取两条指令(16-bit 指令)

经典：取指 → 译码 → 执行 → 访存 → 写回

4、通用寄存器：

R0 ~ R7　低寄存器 (16位指令)

R8 ~ R12　高寄存器 (32或16位指令)

✓ R13 (SP)　栈指针 { MSP 主栈指针 / PSP 进程栈指针 }

R14 (LR)　链接寄存器　用于函数或子函数调用返回地址的保存

✓ R15 (PC)　程序计数器　读返回当前地址 +4　51不能改PC，Cortex可以通过R15改PC的值

xPSR (标志位)　程序状态寄存器

存储器区　哈佛结构　4GB空间

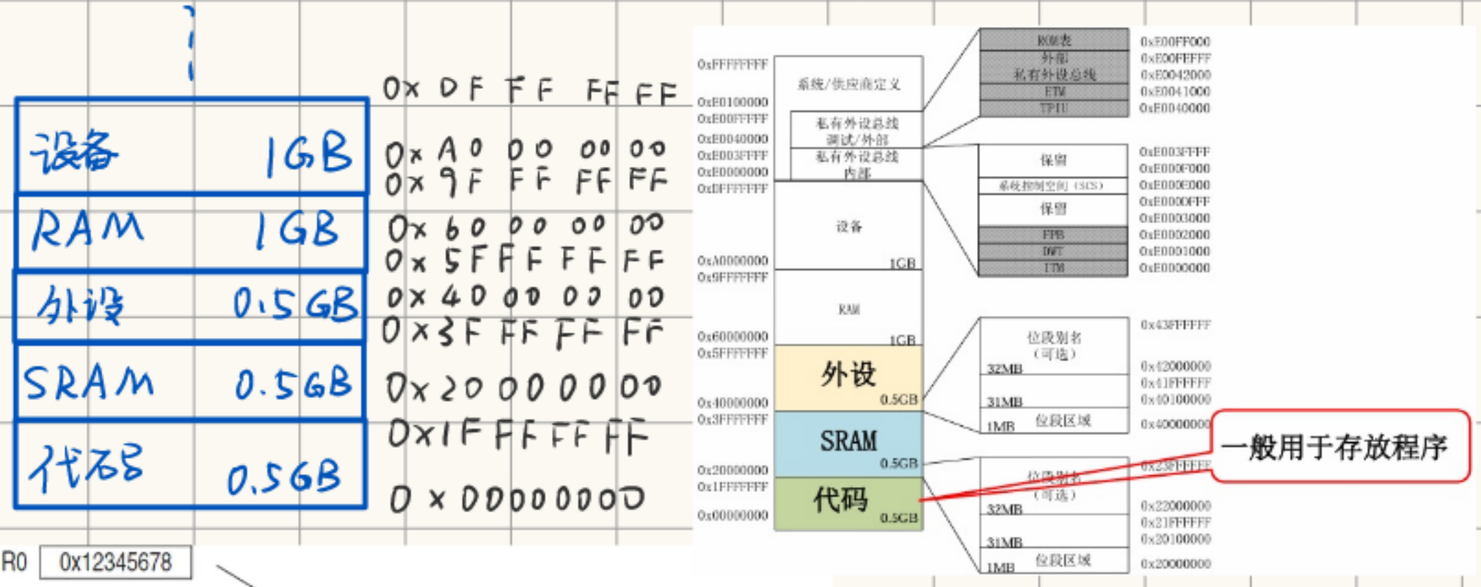
5、根据典型用法，分为：

➢程序代码访问区（如CODE区）

➢数据访问区 （如RAM区）

➢外设

➢处理器的内部控制和调试部件



6、 堆栈区

向下生长的满栈模型

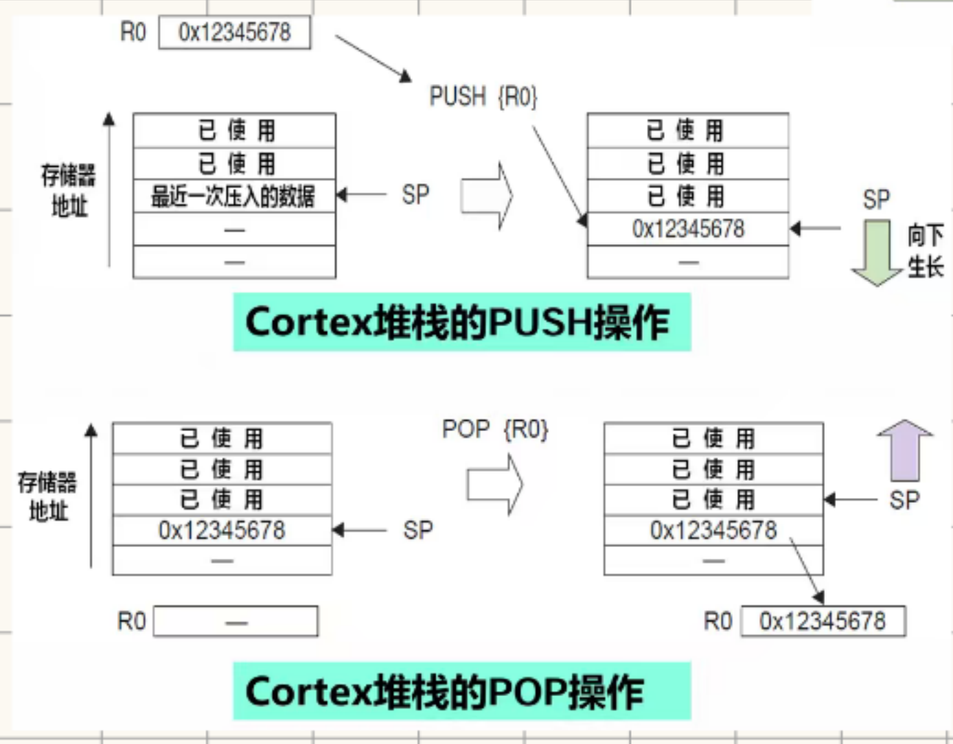
PUSH: ① SP减4

② 存入数据R0

POP ① 取出数据R0

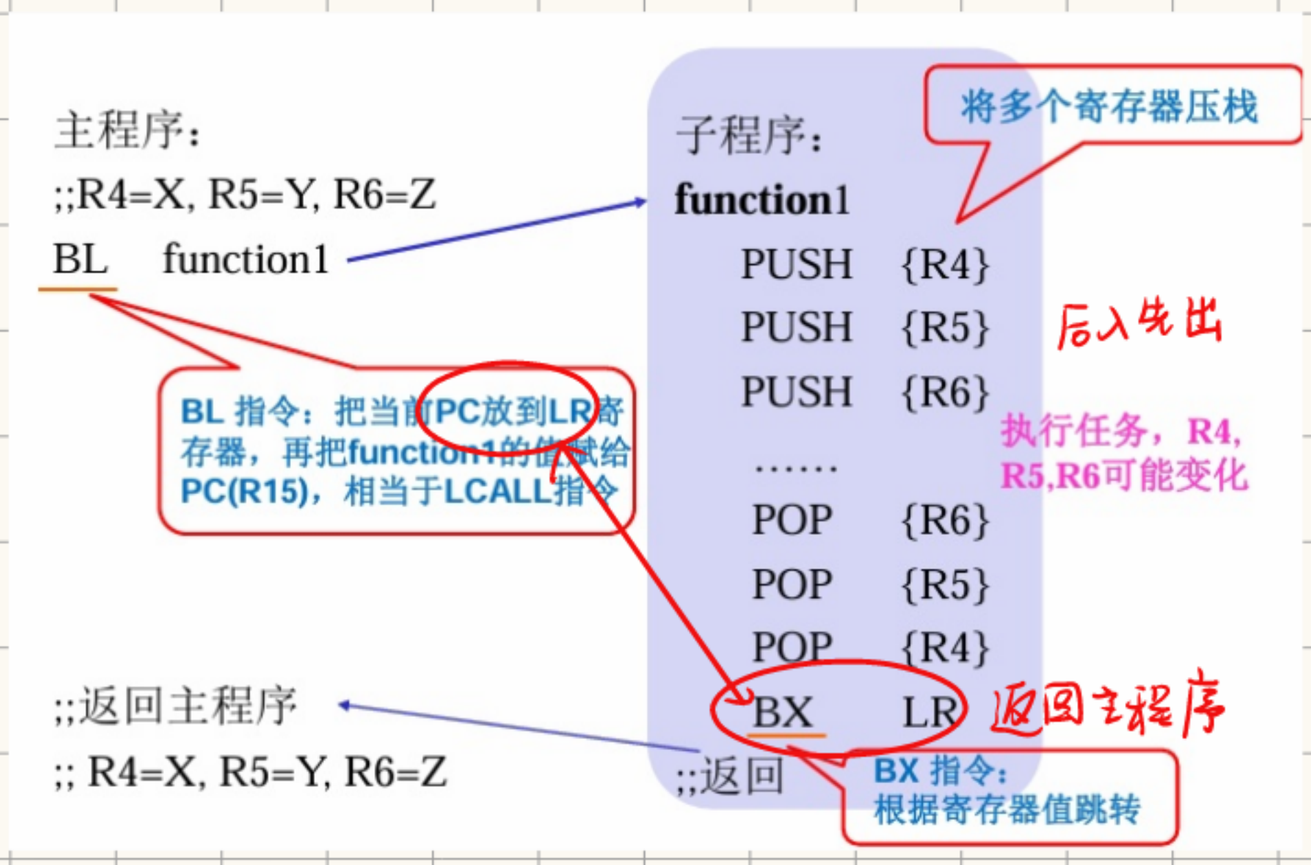
② SP加4

一次压32位



7、 中断向量 中断控制器 (NVIC)

抢占优先级 /响应优先级



push {R4-R6, LR}

pop {R4-P6, PC}

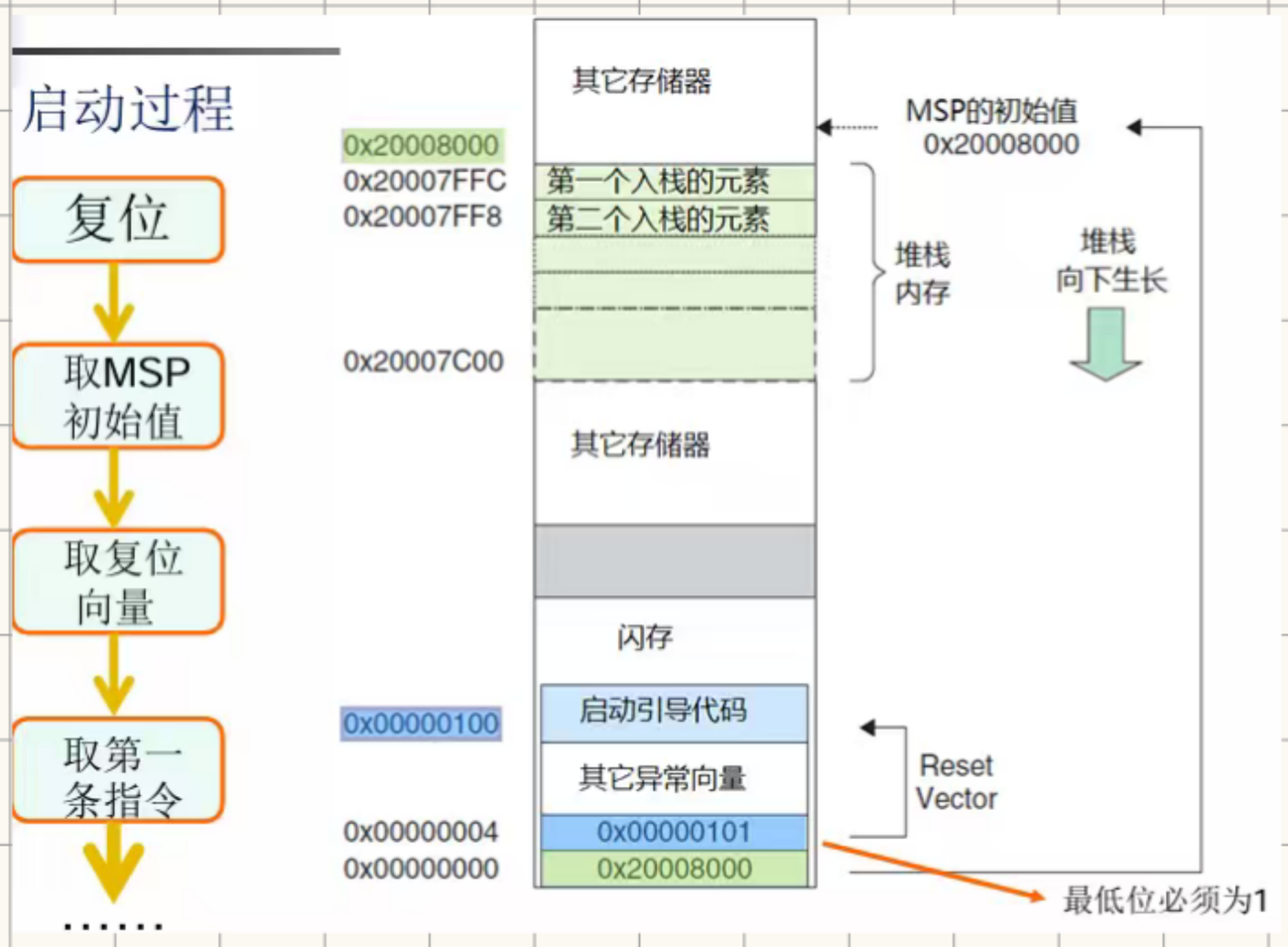
反着POP

POP PC 是把回去地址给PC，节省语句

= POP LR

+ BX LR

8、 复位流程



9. 指令集　32位或16位 RISC指令集

指令格式：

标号：操作码 操作数1 操作数2

指令后缀

S 更新APSR标志

EQ 相等执行

NE 不等

LT 少于

GT 多于

(1) 处理器内部数据转移指令

MOV R1, R2

MOVW R4, #0x1234 放低16位，高16位清零

MOVT R4, #0x5678 放高16位

(2) 存贮器访问指令

LDRB R1, [R2,#2]

例：R2 = 0x20000000

从地址(R2+2)读取1个字节(Byte)到R1。

执行后：

R1 = 0x20000002 = 0x55

STRB R1, [R2,#4]

例：R1 = 0xAA

R2 = 0x20000000

执行后：

0x20000004 = 0xAA

将R1最低8位写入地址(R2+4)。

前序

以一个寄存器作为存贮器的基准地址（指针），与一个立即数相加，其和作为访问存贮器的地址，这种指令称为“前序”寻址。

示例	功能描述
LDRB Rd, [Rn, #offset]	从地址Rn+offset处读取一个字节送到Rd
LDRH Rd, [Rn, #offset]	从地址Rn+offset处读取一个半字送到Rd
LDR Rd, [Rn, #offset]	从地址Rn+offset处读取一个字送到Rd
LDRD Rd1, Rd2, [Rn, #offset]	从地址Rn+offset处读取一个双字(64位整数)送到Rd1（低32位）和Rd2（高32位）中。Offset长度：16位指令时5位；32位指令时12位
STRB Rd, [Rn, #offset]	把Rd中的低字节存储到地址Rn+offset处
STRH Rd, [Rn, #offset]	把Rd中的低半字存储到地址Rn+offset处
STR Rd, [Rn, #offset]	把Rd中的低字存储到地址Rn+offset处
STRD Rd1, Rd2, [Rn, #offset]!	把Rd1（低32位）和Rd2（高32位）表达的双字存储到地址Rn+offset处

怎么有这么多？

（2）存贮器访问指令：寄存器偏移（前序）

以一个寄存器作为存贮器的基准地址（指针），与另一个从索引寄存器值相加，其和作为访问存贮器的地址。从索引寄存器值在相加前，可左移0~3位。

示例	功能描述
LDRB Rd, [Rn，Rm{, LSL, #n}]	从存贮器位置Rn+(Rm<<n)处读取一个字节送到Rd
LDRH Rd, [Rn，Rm{, LSL, #n}]	从存贮器位置Rn+(Rm<<n)处读取一个半字送到Rd
LDR Rd, [Rn，Rm{, LSL, #n}]	从存贮器位置Rn+(Rm<<n)处读取一个字送到Rd
LDRSB Rd, [Rn，Rm{, LSL, #n}]	从从存贮器位置Rn+(Rm<<n)处读取一个字节并进行有符号展开。
STRB Rd, [Rn，Rm{, LSL, #n}]	往存贮器位置Rn+(Rm<<n)处存储一个字节
STRH Rd, [Rn，Rm{, LSL, #n}]	往存贮器位置Rn+(Rm<<n)处存储一个半字
STR Rd, [Rn，Rm{, LSL, #n}]	往存贮器位置Rn+(Rm<<n)处存储一个字

例：

```
LDRB    R0, [R5, R1, LSL #1];
STR     R0, [R1, R2, LSL #2];
```

后序

该方式也有一个立即数偏移值，但是在存贮器访问期间不用偏移值，仅在数据结束后更新地址寄存器（指针）。指令无需用感叹号（！）。

示例	功能描述
LDRB Rd, [Rn]，#offset	读存贮器[Rn] 处的字节到Rd,然后更新Rn为Rn+offset
LDRH Rd, [Rn]，#offset	读存贮器[Rn] 处的半字到Rd,然后更新Rn为Rn+offset
LDR Rd, [Rn]，#offset	读存贮器[Rn] 处的字到Rd,然后更新Rn为Rn+offset
LDRSB Rd, [Rn]，#offset	读存贮器[Rn] 处的字节到Rd并进行有符号展开,然后更新Rn为Rn+offset。
STRB Rd, [Rn]，#offset	往存贮器位置[Rn]处存储一个字节,然后更新Rn为Rn+offset
STRH Rd, [Rn]，#offset	往存贮器位置[Rn]处存储一个半字,然后更新Rn为Rn+offset
STR Rd, [Rn]，#offset	往存贮器位置[Rn]处存储一个字,然后更新Rn为Rn+offset

更新都是Rn

例：

```
LDRB    R0, [R5], #4;
STR     R0, [R1], #20;
```

（2）存贮器访问指令：

例1：

```
LDRB    R4，[R1]        ;;从存贮器地址[R1]读一个字节存入R4
LDR     R0，[R1,#08]!
                        ;;在读取[R1+0x8]到R0后，R1被更新为R1+0x8
                        ;;感叹号(!) 表示指令执行完成后更新存放地址的寄存器
STR     R0，[R1]        ;;将R0的内容写到[R1]地址
STRD    R0,R1,[R2,#08]! ;;将R0，R1作为双字，写到[R2+0x8]地址并更新R2
LDR     R0, [PC, #0x20] ;;利用PC偏移加载字数据到R0

LDRNE   R0, [R5], #4;   ;;APSR的Z标志不为1执行该指令

LDR     R0, [R5, R1, LSL #2] ;
```

多重存贮器访问指令：

可以读或写内存中的多个连续数据。

多重加载指令	功能描述
LDMIA Rd!, {寄存器列表}	从Rd指定的存贮器地址读取多个字到寄存器列表，地址在每次读取后加4（IA）。
STMIA Rd!, {寄存器列表}	依次将寄存器列表的值写入Rd指定的地址，地址在每次读取后加4（IA）。
LDMDB Rd!, {寄存器列表}	从Rd指定的存贮器地址读取多个字到寄存器列表，地址在每次读取前减4（DB）。
STMDB Rd!, {寄存器列表}	依次将寄存器列表的值写入Rd指定的地址，地址在每次读取前减4（DB）。

IA: Increase After　　DB:Decrease Before

(3) 跳转指令

B, BX

例:

```
start
    MOV R0,#1
    B loop
    MOV R0,#2

loop
    MOV R1,#3
```

执行过程:

执行B loop后直接跳到loop。

例:

```
R0 = func
BX R0

func
    MOV R1,#100
```

说明:

BX跳转到寄存器指定的地址。常用于函数返回

(4) IT、ITE指令

IT例子:

```
CMP R0,#0
IT EQ
MOVEQ R1,#1
```

说明:

如果R0==0

则执行:

MOV R1,#1

否则跳过该指令。

ITE例子:

```
CMP R0,#0
ITE EQ
MOVEQ R1,#1
MOVNE R1,#2
```

执行结果:

R0==0

R1=1

R0!=0

R1=2

重点以下存贮器访问指令:

```
LDRB    R4, [R1]          ;;从存贮器地址[R1]读一个字节存入R4
LDR     R0, [R1,#08]!     ;;在读取[R1+0x8]到R0后，R1被更新为R1+0x8
                          ;;感叹号(!)表示指令执行完成后更新存放地址的寄存器

STR     R0, [R1]          ;;将R0的内容写到[R1]地址
STRD    R0,R1,[R2,#08]!   ;;将R0，R1作为双字，写到
                          ;;[R2+0x8]地址并更新R2

LDR     R0,[PC,#0x20]     ;;利用PC偏移加载字数据到R0
```

使用后缀指令:

MOVS、LDRNE、ADDEQ

```
R0 = 0x11223344
R1 = 0x55667788
R2 = 0x20000000

STRD R0, R1, [R2, #8]!

地址        内容
0x20000008   44
0x20000009   33
0x2000000A   22
0x2000000B   11    ← R0 = 0x11223344

0x2000000C   88
0x2000000D   77
0x2000000E   66
0x2000000F   55    ← R1 = 0x55667788
```

10、位段操作

不使用位段操作

```
LDR     R0=0x2000 0000
LDR     R1,[R0]
ORR     R1,0x4
STR     R1,[R0]
```

使用位段操作

```
LDR     R0=0x2200 0008
MOV     R1,#1
STR     R1,[R0]
```

Alias = AliasBase + (ByteOffset × 32) + (Bit × 4)

例如要操作 SRAM 地址 0x20000000 的 bit2:

AliasBase = 0x22000000 (1)22开头

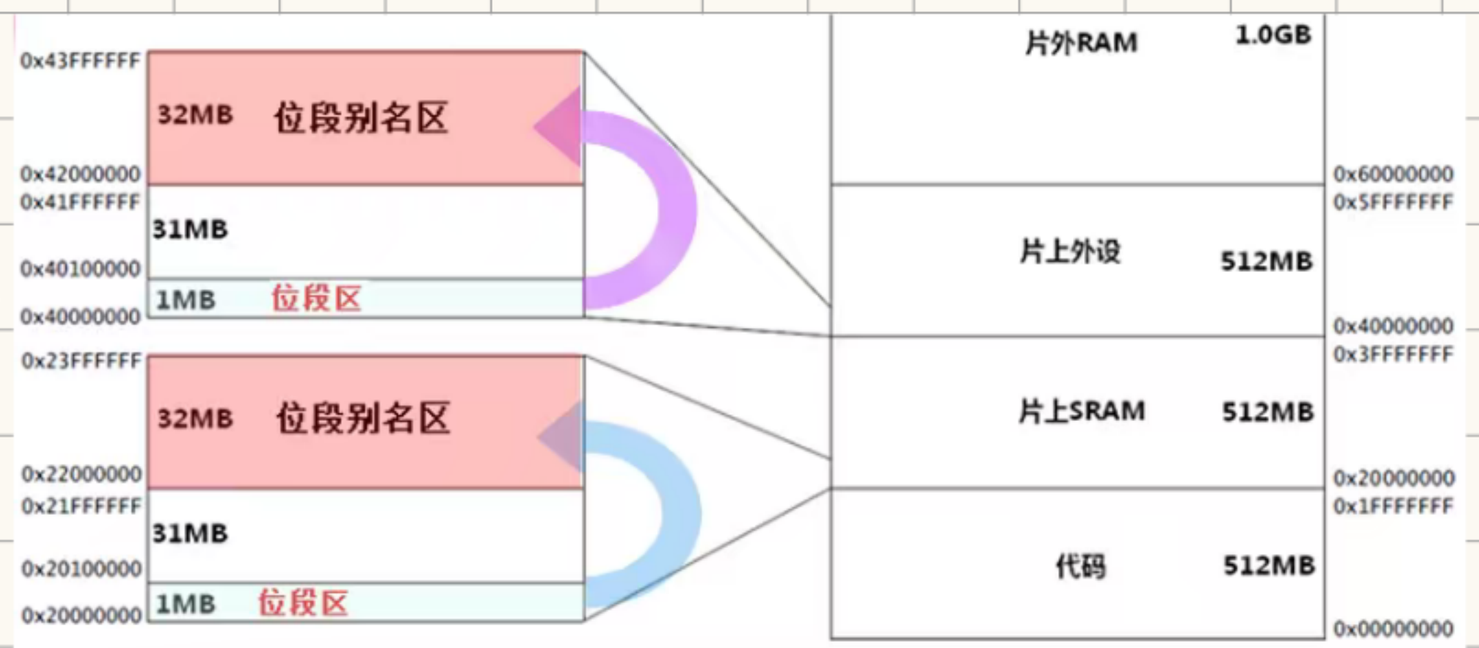
ByteOffset = 0x20000000 - 0x20000000 = 0

Bit = 2

代入公式:

Alias= 0x22000000 + (0×32) + (2×4)=

0x22000008



位段操作的优点：

- 简化编程
- 位操作是一种“原子操作”，不会被其他指令打断

“不可中断的一个或一系列操作”

例如 0x200FFFFC 第2位

偏移 = 0xFFFFC

乘32 加 2×4

0x22000000 + 32×0xFFFFC + 2×4

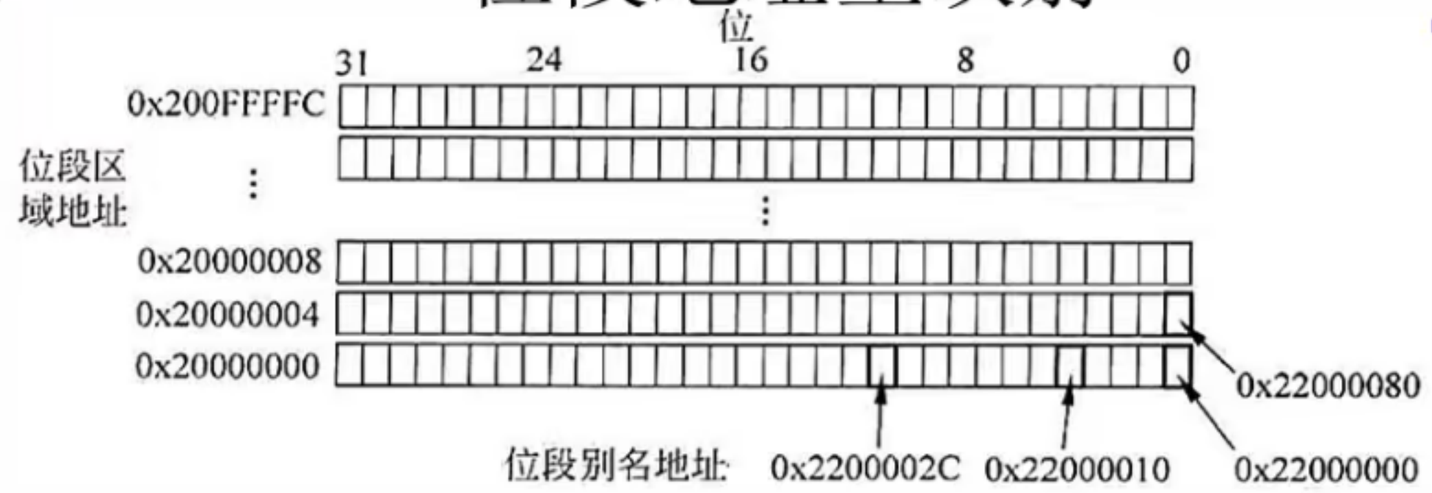
= 0x23FFFFFC

☆位地址是原子操作，不会被打断

1<<2=4 左移运算符

0000 0001 ⟶ 0000 0100

位段地址重映射



SRAM 区域的位段地址重映射

位段区域	别名等价
0x20000000 bit[0]	0x22000000 bit[0]
0x20000000 bit[1]	0x22000004 bit[0]
0x20000000 bit[2]	0x22000008 bit[0]
…	…
0x20000000 bit[31]	0x2200007C bit[0]
0x20000004 bit[0]	0x22000080 bit[0]
…	…
0x20000004 bit[31]	0x220000FC bit[0]
…	…
0x200FFFFC bit[31]	0x23FFFFFC bit[0]

第十章 ARM 编程

1. 结构体变量的地址对应关系

优点：

- 相关的设置参数放置在一个结构体中；
- 对外设操作时，将相关寄存器地址映射到结构体的成员。
- 结构体变量地址对应该结构体的首地址

```
typedef struct
{                    0x00 首地址
  __IO uint32_t MODER;
  __IO uint32_t OTYPER;
  __IO uint32_t OSPEEDR;
  __IO uint32_t PUPDR;
  __IO uint32_t IDR;
```

GPIO 寄存器映射

偏移	寄存器	31	30	29	28
0x00	GPIOA_MODER	MODER15[1:0]		MODER14[1:0]	
	Reset value	1	0	1	0
0x04	GPIOx_OTYPER (where x = A..I/)				
	Reset value				
0x08	GPIOx_OSPEEDER (where x = A..I/ except B)	OSPEEDR15[1:0]		OSPEEDR14[1:0]	
	Reset value	0	0	0	0
0x0C	GPIOA_PUPDR	PUPDR15[1:0]		PUPDR14[1:0]	
	Reset value	0	1	1	0

32位 = 4 byte

地址里面 01可以存8位

共需0 0x04 一次 ☆

```
{ #define  TRUE  1
  #undefine
```

& 按位与 | 按位或 ^ 按位异或

~ 按位非 << 左移 >> 右移

2. 通用I/O口

16位 GPIOA　16位 GPIOB　16位 GPIOC　…… 16位 GPIOI

GPIO 端口模式寄存器 (GPIOx_MODER) (x = A..I)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 2y:2y+1 **MODERy[1:0]**：端口 x 配置位 (Port x configuration bits) (y = 0..15)
这些位通过软件写入，用于配置 I/O 方向模式。
00：输入（复位状态）
01：通用输出模式
10：复用功能模式
11：模拟模式

4种模式

GPIOx_MODER寄存器用 2 个 bit 控制一个 IO 引脚，共 16 个引脚，每个引脚可配置为输入、输出、复用、模拟四种模式。

16位 GPIOA　16位 GPIOB　16位 GPIOC　…… GPIOI

寄存器名	功能
GPIOx_MODER	IO口类型设置
GPIOx_OTYPER	输出类型设置
OSPEEDER	输出速度
GPIOx_PUPDR	上拉或下拉设置
GPIOx_IDR, GPIOx_ODR	输入或输出数据
GPIOx_BSRR	位清零或置1
GPIOx_AFRH, GPIOx_AFRL	复用功能选择

配置寄存器

寄存器操作：

```
GPIOA->BSRRH= GPIO_Pin_9;      //对GPIOA.9清零
GPIOA->BSRRL= GPIO_Pin_9;      //对GPIOA.9置1
//GPIO_Pin_9预定义
#define GPIO_Pin_9             ((uint16_t)0x0200)
```

GPIO 端口置位/复位寄存器 (GPIOx_BSRR) (x = A..I)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

位 31:16 **BRy**：端口 x 复位位 y (Port x reset bit y) (y = 0..15)
这些位为只写形式，只能在字、半字或字节模式下访问。读取这些位可返回值 0x0000。
0：不会对相应的 ODRx 位执行任何操作
1：对相应的 ODRx 位进行复位
注意：如果同时对 BSx 和 BRx 置位，则 BSx 的优先级更高。

位 15:0 **BSy**：端口 x 置位位 y (Port x set bit y) (y= 0..15)
这些位为只写形式，只能在字、半字或字节模式下访问。读取这些位可返回值 0x0000。
0：不会对相应的 ODRx 位执行任何操作
1：对相应的 ODRx 位进行置位

BSRRH 清零

BSRRL 置1　☆HL反着的

采用库函数编程：

```
GPIO_ResetBits(GPIOA , GPIO_Pin_9);    //对GPIOA.9清零
GPIO_SetBits(GPIOA , GPIO_Pin_9);      //对GPIOA.9置1
```

等效的直接对寄存器操作：

```
GPIOA->BSRRH= GPIO_Pin_9;      //对GPIOA.9清零
GPIOA->BSRRL= GPIO_Pin_9;      //对GPIOA.9置1

//GPIO_Pin_9预定义
#define GPIO_Pin_9             ((uint16_t)0x0200)

GPIOA->BSRRL= GPIO_Pin_9| GPIO_Pin_10 ;
//对GPIOA.9和GPIOA.10置1
```

设置GPIO口为什么不通过ODR寄存器操作，而用BSRRH/BSRRL寄存器？

原子操作，不影响其他GPIO口；ODR一起设置，可能被中断打断

Table 8. Alternate function mapping (continued)

Port		AF0 SYS	AF1 TIM1/2	AF2 TIM3/4/5	AF3 TIM8/9/10/11	AF4 I2C1/2/3	AF5 SPI1/SPI2/ I2S2/I2S2ext	AF6 SPI3/I2Sext/ I2S3
Port B	PB0		TIM1_CH2N	TIM3_CH3	TIM8_CH2N			
	PB1		TIM1_CH3N	TIM3_CH4	TIM8_CH3N			
	PB2							
	PB3	JTDO/ TRACESWO	TIM2_CH2				SPI1_SCK	SPI3_SCK I2S3_CK
	PB4	NJTRST		TIM3_CH1			SPI1_MISO	SPI3_MISO
	PB5			TIM3_CH2		I2C1_SMBA	SPI1_MOSI	SPI3_MOSI I2S3_SD
	PB6			TIM4_CH1		I2C1_SCL		
	PB7			TIM4_CH2		I2C1_SDA		
	PB8			TIM4_CH3	TIM10_CH1	I2C1_SCL		
	PB9			TIM4_CH4	TIM11_CH1	I2C1_SDA	SPI2_NSS I2S2_WS	

对应的连线

注：并非每个引脚都有16种功能

3、中断系统

NVIC的中断优先级：

优先级包括抢占优先级和响应优先级：

- 共占4bit，哪几个bit表示抢占优先级或响应优先级可设置；
- 编号越小，优先级越高；
- 抢占优先级高的中断可以打断优先级低的中断；
- 抢占优先级相同时，两个中断同时到达，则先处理响应优先级高的中断。

例：

中断3的抢占优先级为2，响应优先级为1；

中断6的抢占优先级为3，响应优先级为0；

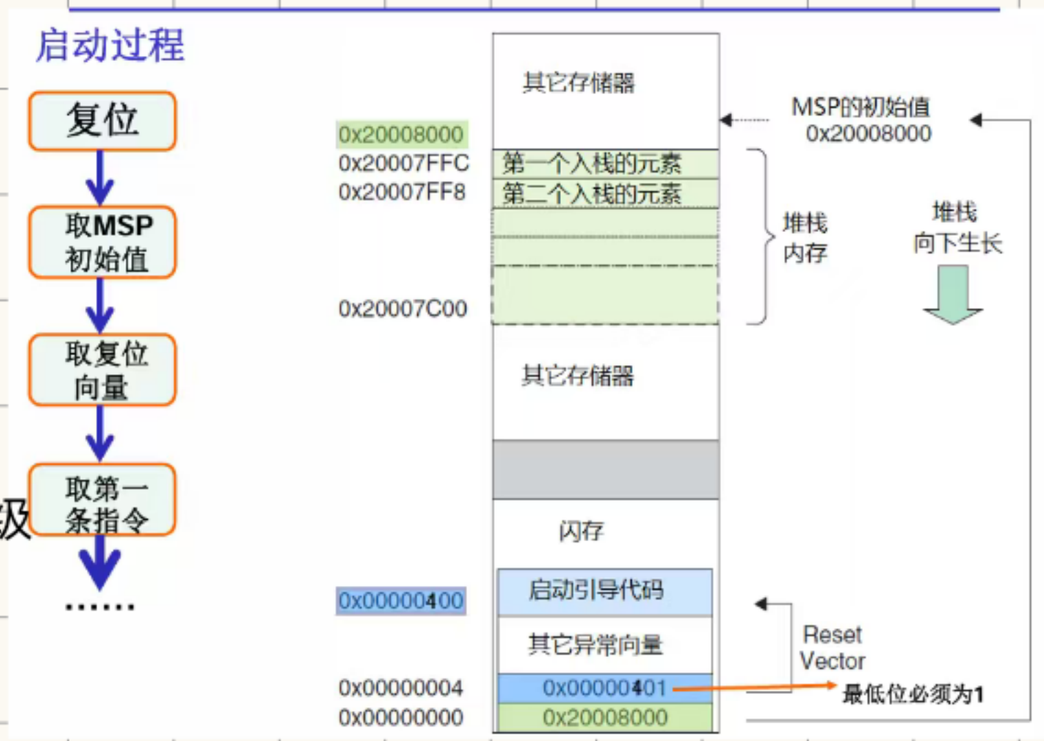
中断7的抢占优先级为2，响应优先级为0；

则

中断7和中断3可打断中断6；

中断7和中断3不可互相打断；

中断7和中断3同时触发时响应中断7；



优先级寄存器NVIC->IP的有效位数为4位Bit[7:4]，包括抢占优先级和响应优先级的位数。

具体分配情况由 SCB->AIRCR 寄存器的 bit10~8 定义

	AIRCR[10:8]	Bit[7:4]分配	分配结果
0	111	0：4	0位抢占优先级，4位响应优先级
1	110	1：3	1位抢占优先级，3位响应优先级
2	101	2：2	2位抢占优先级，2位响应优先级
3	100	3：1	3位抢占优先级，1位响应优先级
4	011	4：0	4位抢占优先级，0位响应优先级

4、定时器

- 14个定时器

定时器1和8：高级控制定时器，16位自动重装载；

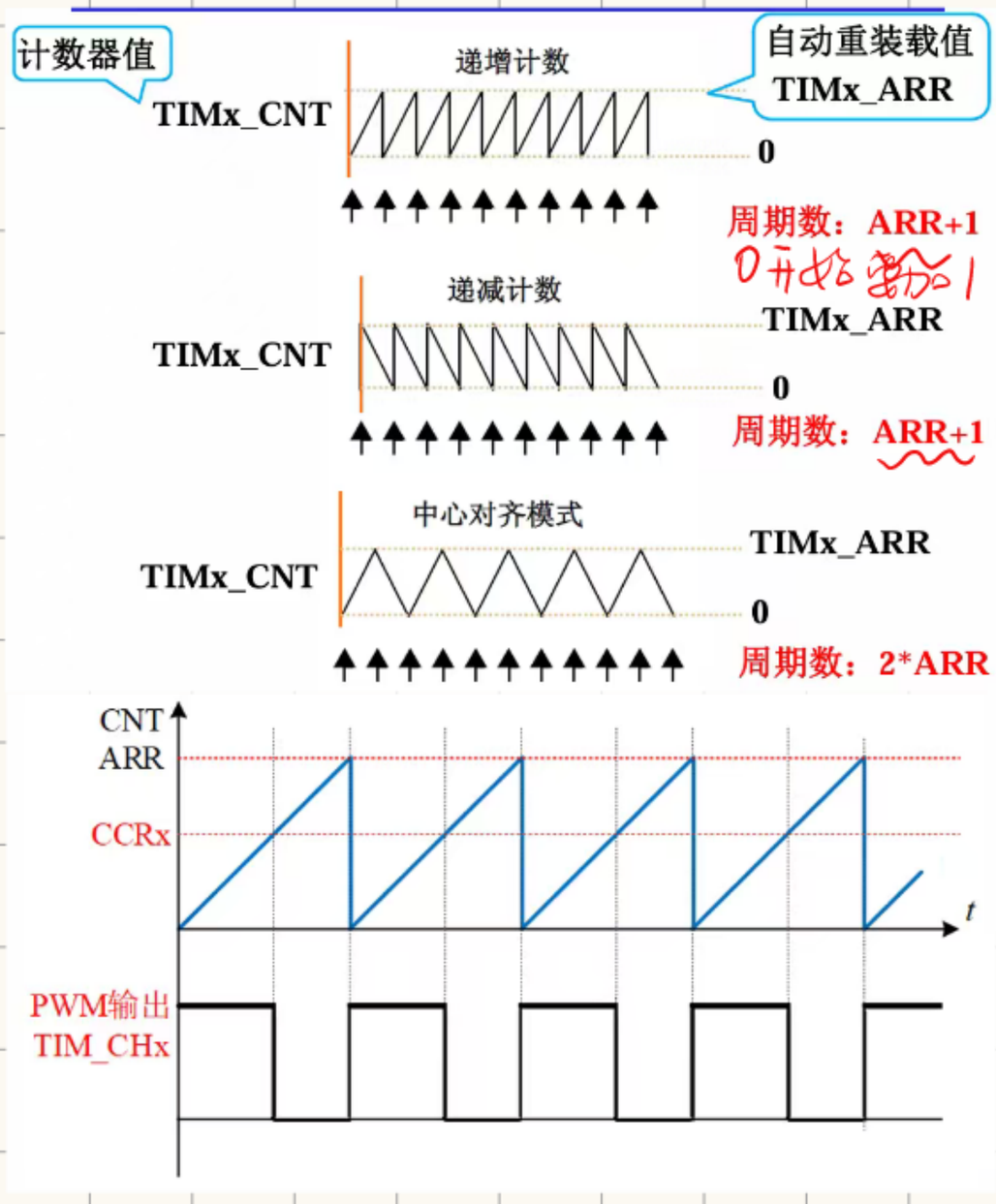
定时器2和5：通用定时器，32位自动重装载；

定时器3和4：通用定时器，16位自动重装载；

定时器6和7：基本定时器，16位自动重装载；

定时器9-14：通用定时器，16位自动重装载；

- 定时器相互完全独立，不共享任何资源；
- 功能强大，可用于定时、计数、PWM控制等；



PWM模式下的占空比设置

1、预分频设置：

TIM1->PSC=P-1;

2、定时器周期和计数方式设置；

TIM1->ARR= N-1;

3、PWM占空比设置

TIM1->CCR1=PWM;

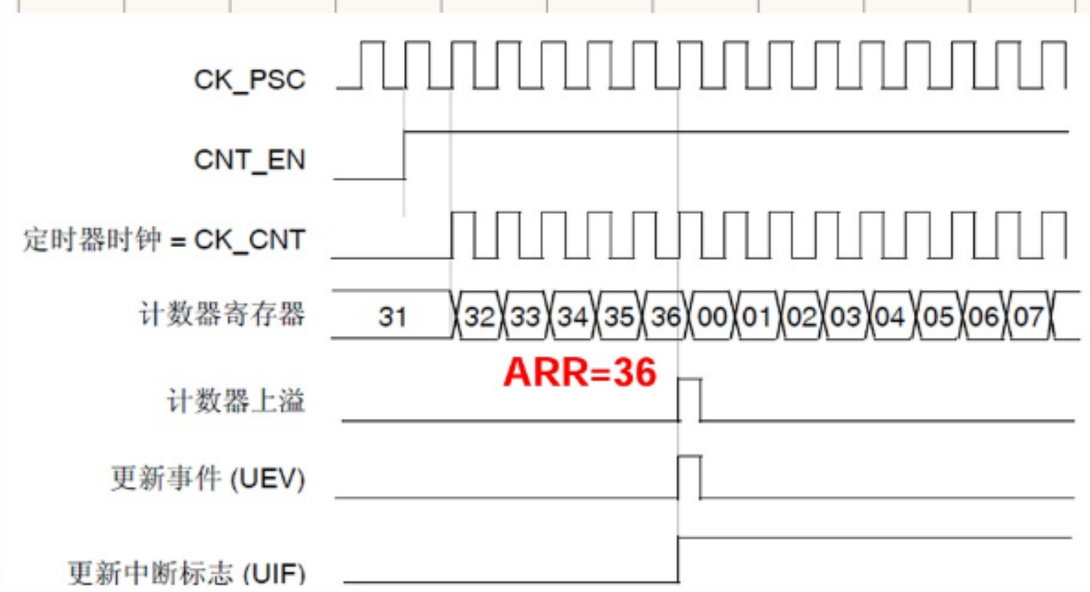
例：

假设 TIM1 时钟是 72MHz，想输出 1kHz PWM，占空比 30%：

TIM1->PSC = 72 - 1; // 72MHz / 72 = 1MHz

TIM1->ARR = 1000 - 1; // 1000个计数 = 1ms

TIM1->CCR1 = 300; // 300/1000 = 30%



PWM设计原理：

1、设置计数器向上/向下计数等工作方式；

2、设置计数周期（TIMx_ARR寄存器）；

3、设置计数比较值（TIM_CCRx, x=1,2,3,4）

5. 直接内存访问

DMA配置通常要确定 4个核心要素：

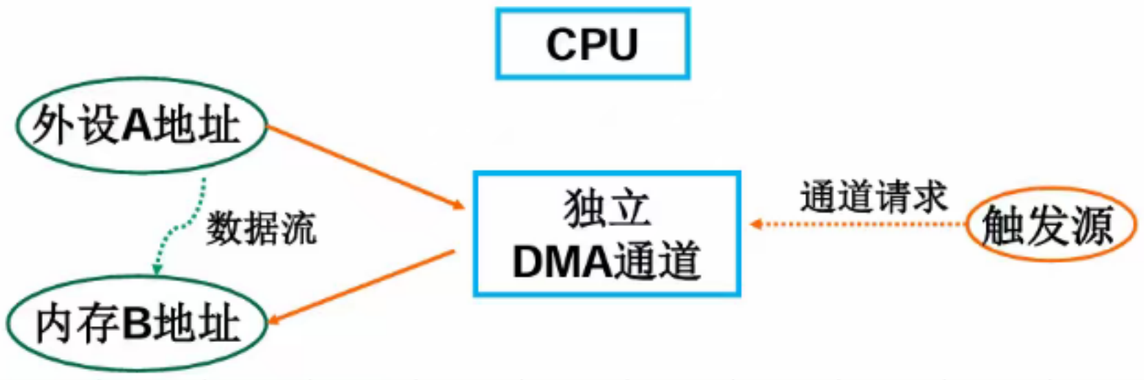
1. 数据从哪里来（源地址）
2. 数据到哪里去（目标地址）
3. 搬运多少数据（传输长度）
4. 怎么搬（传输方式）

对应 STM32 DMA 寄存器理解：

源地址　　DMA_CPAR 或 DMA_CMAR
目标地址　DMA_CMAR 或 DMA_CPAR
数据个数　DMA_CNDTR
传输方式　DMA_CCR

什么是DMA（Direct Memory Access）

- 定义和本质：是一种硬件机制，允许外设直接与内存交换数据，而无需经过 CPU 的干预。它在总线上充当“第二主控器”。
- 核心使命：将 CPU 从繁琐、机械的“数据搬运”任务中解放出来，使其能够专注于复杂的算法计算和逻辑控制，实现真正的并行处理。



DMA控制要素 —— 1.触发源 2.DMA通道 3.源地址 4.目标地址

DMA的四种传输方式：

外设到内存
内存到外设
内存到内存
外设到外设

需要设置的参数：

触发源及其通道
源地址
目标地址
传输数据类型、长度

表 36.　DMA2 请求映射

外设请求	数据流 0	数据流 1	数据流 2	数据流 3	数据流 4	数据流 5	数据流 6	数据流 7
通道 0	ADC1		TIM8_CH1 TIM8_CH2 TIM8_CH3		ADC1		TIM1_CH1 TIM1_CH2 TIM1_CH3	
通道 1		DCMI	ADC2	ADC2		SPI6_TX(1)	SPI6_RX(1)	DCMI
通道 2	ADC3	ADC3		SPI5_RX(1)	SPI5_TX(1)	CRYP_OUT	CRYP_IN	HASH_IN
通道 3	SPI1_RX		SPI1_RX	SPI1_TX		SPI1_TX		
通道 4	SPI4_RX(1)	SPI4_TX(1)	USART1_RX	SDIO		USART1_RX	SDIO	USART1_TX
通道 5		USART6_RX	USART6_RX	SPI4_RX(1)	SPI4_TX(1)		USART6_TX	USART6_TX
通道 6	TIM1_TRIG	TIM1_CH1	TIM1_CH2	TIM1_CH1	TIM1_CH4 TIM1_TRIG TIM1_COM	TIM1_UP	TIM1_CH3	
通道 7		TIM8_UP	TIM8_CH1	TIM8_CH2	TIM8_CH3	SPI5_RX(1)	SPI5_TX(1)	TIM8_CH4 TIM8_TRIG TIM8_COM

步骤2：选择数据流和通道

步骤3：编程

初始化设置：
1、串口初始化；　使能UART的DMA请求

```
USART_DMACmd(USART1, USART_DMAReq_Rx, ENABLE);
2、DMA初始化
DMA_InitTypeDef DMA_InitStructure;
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_DMA2, ENABLE);
DMA_InitStructure.DMA_Channel          = DMA_Channel_4;
DMA_InitStructure.DMA_PeripheralBaseAddr  设置对应的DMA通道
                                       = (uint32_t)(&USART1->DR);
DMA_InitStructure.DMA_Memory0BaseAddr  设置DMA的内存地址
                                       = (uint32_t)(&RS232_buff);
DMA_InitStructure.DMA_DIR              设置DMA的外设地址
                                       = DMA_DIR_PeripheralToMemory;
DMA_InitStructure.DMA_BufferSize       设置DMA的方向
                                       = REC_BUFF_SIZE;
                                       设置DMA的数据长度
```

6. 串行通信，接口编程

1、设置相应的IO，IO类型选特殊功能（AF）；
2、在IO口复用功能表中找到相应的功能号并设置；
3、初始化串行口参数；

Port B	PB4	NJTRST		TIM3_CH1			SPI1_MISO	SPI3_MISO
	PB5			TIM3_CH2		I2C1_SMBA	SPI1_MOSI	SPI3_MOSI I2S3_SD
	PB6			TIM4_CH1		I2C1_SCL		
	PB7			TIM4_CH2		I2C1_SDA		
	PB8			TIM4_CH3	TIM10_CH1	I2C1_SCL		
	PB9			TIM4_CH4	TIM11_CH1	I2C1_SDA	SPI2_NSS I2S2_WS	

8051 DW 列表时为大端模式，其他情况小端，cortex-m3/4 小端

asm

```
地址      内容
1000H     12H
1001H     34H
```

再比如：

asm

```
ORG 2000H
TAB: DW 5678H, 9ABCH
```

存储结果：

asm

```
地址      内容
2000H     56H
2001H     78H
2002H     9AH
2003H     BCH
```

示例：32位数据 0x12345678

C

```
uint32_t x = 0x12345678;
```

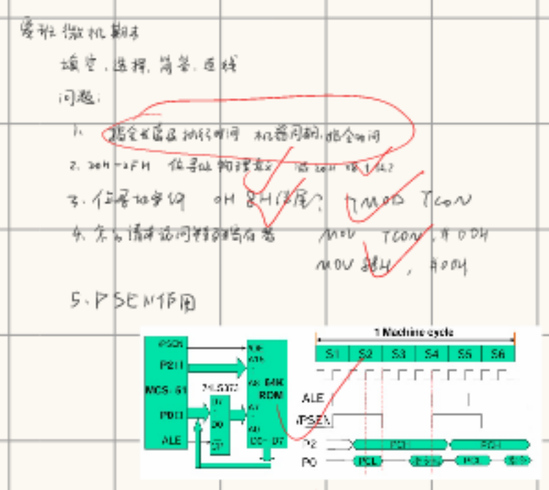
存放在地址 `0x20000000` ：

地址	数据
0x20000000	78H
0x20000001	56H
0x20000002	34H
0x20000003	12H

即：

低地址		高地址	
0x20000000	0x20000001	0x20000002	0x20000003
78H	56H	34H	12H

小端：
低位低地址
高位高地址



专题整理

① 单字节双周期

INC DPTR

MOVX A, @DPTR
MOVC A, @A+PC

RETI, RET
JMP @A+DPTR

② 6个特定地址

复位 0000H

$\overline{INT0}$ 0003H

T0 000BH

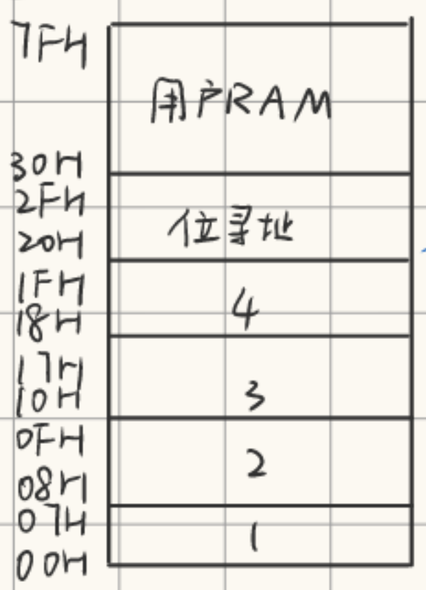
$\overline{INT1}$ 0013H

T1 001BH

串行口 0023H

(T2 002BH)

③ 内部RAM



④ PSW

7 CY 进位
6
5 F0 用户标记
4 AC 辅助进位
3 RS1
2 RS0
1 OV 溢出
0 P 奇偶校验

⑤ I/O口

P0 P1 P2 P3

双向 准I/O

(需上拉)

8个TTL 4个TTL

8位地址/数据 8位地址 RXD TXD $\overline{INT0}$ $\overline{INT1}$ T0 T1 $\overline{WR}$ $\overline{RD}$

⑥ R0 地址

@R0 R0地址里的数据

⑦ 只能对A:

ADD、ADDC、SUBB
ANL、ORL、XRL
XCH、XCHD
DA A, CPL A
RL、RR、RLC、RRC
SWAP A
JZ、JNZ 判断 A=?0

CLR A、C、bit

⑧ 伪指令

ORG < >
< > END < >
< > EQU < >
< > DL < >
< > DB < >

⑨ PCON

D7 ... D1 D0
SMOD PD IDL

PD=1 掉电保护
IDL=1 待机状态
> 都打开进入掉电保护状态

IE 中断允许

EA ES ET1 EX1 ET0 EX0
CPU允 串行允 T1允 外中1 T0允 外中0

RS1 RS0 R0-R7

0 0 00H-07H
0 1 08H-0FH
1 0 10H-17H
1 1 18H-1FH

位寻址 对应00-7FH

位寻址
80H-FFH, 末尾 0H 8H
能位寻址

IE 中断优先级

PS PT1 PX1 PT0 PX0

TMOD: 定时器工作方式

B_7 B_6 B_5 B_4 | B_3 B_2 B_1 B_0
GATE C/$\overline{T}$ M_1 M_0 | GATE C/$\overline{T}$ M_1 M_0
T_1 | T_0

M_1 M_0
0 0 13位计数器
0 1 16位计数器
1 0 8位自动重装载
1 1 T0分2个8位计数器

TCON: 控制寄存器

TF1 TR1 TF0 TR0 IE1 IT1 IE0 IT0
T1溢出 T1启动 T0 外中断1 0 1 0 1 0